

云原生 5G 核心网中网元安全连续性与切片调度技术研究

姓名： ***

学号： *****

学院： 网络空间安全学院

2025 年 5 月



云原生5G核心网中网元安全连续性与切片调度技术研究

*
*
*

北京理工大学

中图分类号：TQ028.1

UDC 分类号：540

云原生 5G 核心网中网元安全连续性与切片调度技术研究

作 者 姓 名	***
学 院 名 称	网络空间安全学院
指 导 教 师	*****
行业合作导师	*****
答辩委员会主席	*****
申 请 类 别	工学硕士
学 位 领 域	网络与信息安全
学位授予单位	北京理工大学
论文答辩日期	2025 年 5 月

Research on Network Function Security Continuity and Slice Scheduling Techniques in Cloud-Native 5G Core Networks

Candidate Name:	*****
School or Department:	Cyberspace Science and Technology
Faculty Mentor:	*****
Industry Collaboration Mentor:	*****
Chair, Thesis Committee:	*****
Degree Applied:	Master of Engineering
Major:	Network and Information Security
Degree by:	Beijing Institute of Technology
The Date of Defence:	May, 2025

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下独立完成的研究成果。文中所撰写内容符合以下学术规范（请勾选）：

☒ 论文综述遵循“适当引用”的规范，全部引用的内容不超过 50%。

☒ 论文中的研究数据及结果不存在篡改、剽窃、抄袭、伪造等学术不端行为，并愿意承担因学术不端行为所带来的一切后果和法律责任。

☒ 文中依法引用他人的成果，均已做出明确标注或得到许可。

☒ 论文内容未包含法律意义上已属于他人的任何形式的研究成果，也不包含本人已用于其他学位申请的论文或成果。

☒ 与本人一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此声明。

签 名：

日 期：

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：

① 学校有权保管、并向有关部门送交学位论文的原件与复印件；

② 学校可以采用影印、缩印或其它复制手段复制并保存学位论文；

③ 学校可允许学位论文被查阅或借阅；

④ 学校可以学术交流为目的，复制赠送和交换学位论文；

⑤ 学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名：

日 期：

导师签名：

日 期：

摘要

随着我国 5G 网络的加速商用,网络切片凭借其在统一物理基础设施上按需构建多个逻辑网络的能力,成为支撑多样化业务场景、满足差异化服务需求的关键支撑。以 Kubernetes 为代表的云原生技术通过其容器化、微服务架构和自动化运维等特性,为核心网切片的服务解耦和弹性部署提供了创新解决方案,并通过基于角色的权限控制、命名空间等机制也增强多租户切片运维的安全性。然而,现有的云原生技术在面对复杂的控制面网络功能的状态保持与调度需求方面仍存在诸多不足,如何在动态环境中保障会话认证、用户鉴权等关键安全功能的控制面网元的状态安全,以及如何实现满足安全隔离需求的切片智能调度,成为亟待解决的问题。

为此,本文设计并实现了一种面向云原生核心网控制面网元的跨节点热迁移机制与网络切片智能调度框架,旨在提升关键网元的状态安全与服务连续性、优化切片调度灵活性、提高整体运维效率,为保障切片服务质量和降低运维成本提供技术支撑。

首先,针对控制面网元状态复杂、冷启动导致安全会话中断的问题,本文设计了一种基于“克隆-恢复”的网元热迁移机制。该机制通过扩展 Kubernetes 容器运行时接口(CRI)并构建网元热迁移控制体系,实现了控制面网元容器及关联非容器状态的增量保存与一致性恢复,从而支持状态继承的快速热迁移。实验结果表明,在 400 服务请求/秒的负载下,本文方案服务迁移后终端请求成功率在 1.10 秒内恢复至 99% 以上,总迁移耗时低于 2.23 秒。该机制显著缩短了停机时间,保障了安全上下文的平滑继承,降低了因网元重配置导致的会话中断和安全风险,有效提升了控制面安全功能的服务连续性。

其次,为提升切片调度的灵活性与安全隔离需求,本文构建了以控制面信令时延最小化和资源负载均衡为优化目标的切片调度模型,将调度问题建模为马尔可夫决策过程,提出 Deep Reinforcement Slice Scheduling (DRSS) 算法实现调度策略的自适应学习与优化。实验表明,DRSS 通过多维状态感知,有效降低控制路径时延、均衡节点负载,并通过精细资源分配增强切片间的隔离性,为安全运行提供保障。

最后,基于上述迁移机制与 DRSS 调度算法,本文实现了一个具备热迁移协同能力的 Kubernetes 智能调度原型系统。系统集成状态采集、策略决策与动作执行,实现集群级的动态网元调度与切片编排。在中电某所测试网络集群的验证中,本方案在资源

利用率、节点负载均衡、终端注册时延及控制信令稳定性等指标上均优于 **Kubernetes** 原生方案。研究成果通过保障安全关键网元的连续运行和优化切片资源分配，展现了其在构建高效、可靠、安全的云原生 5G 核心网中的工程价值。

关键词：云原生；核心网安全；热迁移；容器编排；智能运维

Abstract

With the accelerated commercialization of 5G networks, network slicing, by virtue of its capability to construct multiple on-demand logical networks on a unified physical infrastructure, has become a key enabler for supporting diverse service scenarios and meeting differentiated service requirements. Cloud-native technologies, represented by Kubernetes, offer innovative solutions for service decoupling and elastic deployment in core network slicing through features like containerization, microservices architecture, and automated operations, while also enhancing the security of multi-tenant slice operations and maintenance (O&M) via mechanisms such as Role-Based Access Control (RBAC) and namespaces. However, existing cloud-native technologies still exhibit shortcomings in addressing the state persistence and scheduling demands of complex control plane Network Functions (NFs). Ensuring the state security of control plane NFs responsible for critical security functions like session authentication and user authorization in dynamic environments, and achieving intelligent slice scheduling that meets security isolation requirements, have become pressing issues.

To this end, this thesis designs and implements a cross-node hot migration mechanism for control plane NFs and an intelligent network slice scheduling framework tailored for cloud-native 5G core networks. The research aims to enhance the state security and service continuity of critical NFs, optimize slice scheduling flexibility, improve overall O&M efficiency, and provide technical support for ensuring slice Quality of Service (QoS) and reducing O&M costs.

Firstly, to address the challenges of complex state in control plane NFs and security session interruptions caused by cold starts, this thesis proposes a "Clone-Restore" based NF hot migration mechanism. By extending the Kubernetes Container Runtime Interface (CRI) and constructing an NF hot migration control system, this mechanism achieves incremental saving and consistent recovery of both containerized and associated non-containerized states of control plane NFs, thereby enabling rapid hot migration with state inheritance. Experimental results show that under a load of 400 requests per second (RPS), the terminal request success rate recovers to over 99% within 1.10 seconds post-migration, with the total migration time

under 2.23 seconds. This mechanism significantly reduces downtime, ensures the smooth inheritance of security contexts, mitigates session interruptions and security risks arising from NF reconfiguration, and effectively enhances the service continuity of control plane security functions.

Secondly, to enhance slice scheduling flexibility and meet security isolation requirements, this thesis constructs a slice scheduling model optimizing for minimized control plane signaling delay and resource load balancing. The scheduling problem is modeled as a Markov Decision Process (MDP), and a Deep Reinforcement Slice Scheduling (DRSS) algorithm is proposed to achieve adaptive learning and optimization of scheduling policies. Experiments demonstrate that DRSS, through multi-dimensional state awareness, effectively reduces control path delay, balances node loads, and enhances inter-slice isolation via fine-grained resource allocation, thereby ensuring their secure operation.

Finally, based on the aforementioned migration mechanism and DRSS scheduling algorithm, this thesis implements a Kubernetes-based intelligent scheduling prototype system with hot migration coordination capabilities. The system integrates state collection, policy decision-making, and action execution to achieve cluster-level dynamic NF scheduling and slice orchestration. Validation on a test network cluster demonstrates that the proposed solution outperforms the native Kubernetes scheduler in key performance indicators such as resource utilization, node load balancing, terminal registration delay, and control signaling stability. By ensuring the continuous operation of security-critical NFs and optimizing slice resource allocation, the research outcomes highlight its engineering value in building efficient, reliable, and secure cloud-native 5G core networks.

Key Words: Cloud-Native; Core Network Security; Live Migration; Container Orchestration; AIOps

目录

第 1 章	绪论	1
1.1	研究背景与研究意义	1
1.2	国内外研究现状及发展趋势	3
1.2.1	虚拟网络功能调度	3
1.2.2	网络切片智能编排策略	4
1.2.3	容器/VM 热迁移机制	5
1.3	研究目标和成果	6
1.4	论文组织结构	7
第 2 章	涉及的理论和技术基础	10
2.1	云原生技术	10
2.1.1	云原生的定义与核心原则	10
2.1.2	云原生技术的典型应用场景	11
2.2	Kubernetes 容器编排技术	12
2.2.1	Kubernetes 架构与核心组件	13
2.2.2	Pod 与容器运行时	14
2.2.3	Kubernetes 调度机制	16
2.3	5G 核心网架构与网元功能	18
2.3.1	5G 核心网体系结构	18
2.3.2	核心网网元功能与交互	19
2.3.3	5G 网络切片技术	19
2.4	本章总结	21
第 3 章	云原生 5G 核心网网元热迁移策略	22
3.1	引言	22
3.2	相关工作	23
3.2.1	Pod Eviction	23
3.2.2	rsync	24

3.2.3	RDMA	24
3.2.4	KubeVirt Live Migration	25
3.2.5	问题总结	25
3.3	方法原理与创新分析	25
3.3.1	技术方案探索	25
3.3.2	热迁移方案设计	30
3.4	技术方案实现	36
3.4.1	热迁移架构设计	36
3.4.2	状态提取	38
3.4.3	容器运行时接口拓展	40
3.4.4	热迁移 API 与控制器	43
3.5	实验分析	46
3.5.1	实验环境	47
3.5.2	评价指标	48
3.5.3	实验过程	49
3.5.4	实验结果分析	51
3.6	本章总结	55
第 4 章	云原生 5G 核心网智能切片编排算法	56
4.1	引言	56
4.2	问题建模与算法设计	57
4.2.1	场景定义	57
4.2.2	问题建模	60
4.3	算法设计	64
4.3.1	智能体调度架构	66
4.4	实验评估	67
4.4.1	环境设定	68
4.4.2	实验结果	70
4.5	本章总结	74

第 5 章 面向云原生 5G 核心网的智能调度器	75
5.1 引言	75
5.2 智能调度器设计	75
5.2.1 总体架构	76
5.2.2 关键技术实现	77
5.3 实验评估	79
5.3.1 实验设置	80
5.3.2 评估指标	81
5.3.3 实验结果与分析	83
5.4 本章总结	87
结论	89
参考文献	91
攻读学位期间发表论文与研究成果清单	96

插图

图 1.1	论文组织结构图	9
图 2.1	云原生架构示意图	11
图 2.2	Kubernetes 架构示意图	13
图 2.3	Pod 结构示意图	14
图 2.4	Pod 生命周期	15
图 2.5	Kubernetes 调度流程示意图	17
图 2.6	5G 核心网架构示意图	18
图 2.7	网络切片示意图	20
图 3.1	基于扩展 Pod Spec 的热迁移流程	27
图 3.2	基于 ClonePod 的热迁移流程	28
图 3.3	基于 PodMig 的热迁移流程	29
图 3.4	Pod 热迁移核心流程	31
图 3.5	全量备份、差异备份与增量备份的逻辑对比	33
图 3.6	Pod 恢复时序图	35
图 3.7	Pod 热迁移架构示意图	37
图 3.8	CRI 扩展交互流程	40
图 3.9	热迁移控制器 Reconcile 循环	45
图 3.10	热迁移 Pod 状态转换	46
图 3.11	实验负载服务器	48
图 3.12	低负载-SR 成功率	52
图 3.13	低负载 - 节点 CPU	52
图 3.14	中负载-SR 成功率	52
图 3.15	中负载 - 节点 CPU	52
图 3.16	高负载-SR 成功率	52
图 3.17	高负载 - 节点 CPU	52
图 3.18	不同负载迁移时间对比	54
图 4.1	集群拓扑示例	59

图 4.2	算法调度控制循环	66
图 4.3	控制面信令负载生成过程	69
图 4.4	不同 UE 负载下网元资源消耗	69
图 4.5	不同调度策略的奖励值对比	70
图 4.6	不同调度策略的平均资源利用率对比	71
图 4.7	不同调度策略各资源利用率对比	71
图 4.8	不同调度策略的不均衡度对比	72
图 4.9	不同策略偏好下的表现	73
图 5.1	智能调度器架构	76
图 5.2	状态采集模块流程图	78
图 5.3	动作执行模块流程	80
图 5.4	平均资源利用率与负载不均衡度	83
图 5.5	资源利用率	84
图 5.6	平均资源利用率对比	84
图 5.7	AMF-SMF 网元 RTT 和 UE 注册时延的核密度估计	86
图 5.8	响应稳定性对比	87

表格

表 3.1	热迁移 API 设计方案对比	30
表 3.2	快照策略相关符号表	34
表 3.3	实验硬件资源	47
表 3.4	实验软件资源	47
表 3.5	AMF 负载测试配置	50
表 3.6	不同迁移方案在不同 UE 规模下的性能对比	54
表 4.1	控制面网元分类	58
表 4.2	符号说明表	61
表 4.3	集群节点配置	68
表 5.1	实验集群节点配置	81
表 5.2	三类切片中各网元的资源请求与迁移代价	82
表 5.3	调度延迟对比	85

第 1 章 绪论

1.1 研究背景与研究意义

随着第五代移动通信技术（5G）的快速发展和全球商用部署，通信网络正经历从传统架构向高度灵活、智能化方向的深刻变革。5G 网络以其高带宽、低时延、高可靠性和大规模连接能力，支撑了增强型移动宽带（eMBB）、超可靠低时延通信（URLLC）和海量机器类通信（mMTC）等多样化业务场景^[1]。这些场景对网络性能提出了前所未有的要求，例如 URLLC 场景下需要毫秒级时延和 99.999% 的可靠性，mMTC 场景下需要支持每平方公里百万级设备的连接密度^[2]。更重要的是，多样化的业务场景，特别是 URLLC 和 mMTC，对网络的安全性、隔离性和服务的可信保障提出了远超以往的严苛要求。传统的核心网架构和运维方式在面对如此复杂的业务需求，尤其是在动态变化的环境下保障安全和性能时，逐渐显露出灵活性不足、资源利用率低以及运维效率有限的瓶颈。

为应对这些挑战，5G 核心网引入了控制面与用户面分离（Control and User Plane Separation, CUPS）、服务化架构（Service-based architecture, SBA）和网络切片等关键技术。CUPS 通过将控制面和用户面解耦，实现了网元功能的模块化与独立部署，显著提升了网络的灵活性；SBA 基于微服务理念，将核心网功能分解为多个独立的服务单元，通过标准化的接口实现松耦合与高效协作；网络切片技术则允许运营商根据业务需求动态创建虚拟化网络实例，从而满足不同场景的差异化服务质量要求，同时也为实现不同业务间的安全隔离提供了基础。通过与网络功能虚拟化（Network Function Virtualization, NFV）基础设施结合，5GC 能够为普通用户、应用提供商和垂直行业提供网络切片、边缘计算等新型业务能力，实现从传统通信管道向社会化信息服务使能者的转型^[3]。

与此同时，云原生技术的兴起为 5G 核心网的部署与运维提供了全新的解决方案^[4]。云原生技术以容器化、微服务架构和自动化运维为核心，结合 Kubernetes 等容器编排平台，能够实现资源的弹性伸缩、服务的动态调度和高可用性部署。在 5G 核心网中，云原生技术的应用使得网络功能可以以轻量级容器的形式运行，显著提高了部署效率和资源利用率。云原生的 5G 核心网架构利用容器化和微服务技术，提高了系统的灵活性、扩展性和资源利用效率，为 5G 网络快速部署、按需弹性伸缩和持续

升级演进奠定了技术基础^[5]，容器化和微服务架构也为实现更细粒度的安全策略部署和故障隔离提供了技术可能。

在 5G 核心网云原生演进过程中，控制面网元作为网络“大脑”，其部署方式与运行状态对整个网络性能和高可用性以及安全保障能力至关重要。控制面网元（如 AMF、SMF）承载着用户认证、鉴权、会话管理、移动性管理等关键安全功能，并处理大量敏感信令。合理的控制面网元部署策略可以降低信令传输时延，提高用户接入和切换效率^[6]。而一旦控制面网元发生故障或所在节点需要维护，将对核心网运行稳定性造成打击，甚至可能导致安全会话中断和安全策略执行的短暂失效。在 4G EPC 时代，核心网可靠性主要通过冗余和故障转移机制保障。而在 5G 时代，容器虚拟化技术的灵活性结合云原生平台的调度能力，为网元的快速迁移提供了技术可行性，从而有效保障 5G 服务的连续性^[7-8]。

然而，现有以 Kubernetes 为代表的云原生基础设施在支持有状态网元迁移和复杂网络切片场景的动态调度方面仍存在不足，原生集群高可用保障和服务调度策略在核心网调度场景面临如下问题：

（1）请求动态性与资源配置失配。5G 核心网中，服务请求呈现出显著的时空动态特征——同一时间段不同服务类型的请求量差异较大，同一服务在不同时间的请求强度也具有明显波动性。面对这种动态变化，传统静态资源放置策略虽然可以减少部署开销，但无法根据负载变化灵活调整，易导致资源浪费或性能瓶颈，限制系统弹性与响应效率，这种资源配置失配也可能间接影响到安全资源的按需分配和弹性伸缩。

（2）QoS 优化与负载均衡之间的冲突。网络切片场景下，控制链路的时延优化与节点负载的均衡分配难以兼顾。现有调度策略往往偏向于某一优化目标，难以在系统资源利用率与服务质量保障之间取得平衡，导致整体调度效果不稳定，无法满足运营级部署要求。在安全层面，这种冲突可能导致为高安全等级切片预留的隔离资源被不当占用，或关键安全功能的性能受到影响。

（3）原生调度器感知能力不足。Kubernetes 原生调度器的策略较为单一，仅基于粗粒度的 CPU 和内存资源分配，无法充分感知网络拓扑、业务负载动态变化以及切片特定的 QoS 需求，更缺乏对切片安全等级、隔离需求等安全相关属性的细致感知，导致其难以应对 5G 核心网中复杂多变的业务场景和差异化的安全保障需求。

传统运维方式依赖静态配置和人工干预，难以应对 5G 网络业务负载的快速变化与多样化需求，也无法满足动态安全策略的敏捷部署与快速响应。相比之下，人工智

能驱动的智能运维技术，特别是深度强化学习（Deep Reinforcement Learning, DRL）算法，能够通过实时感知网络状态和动态优化决策，显著提升调度效率和资源利用率。为此，本文聚焦于深度融合云原生技术、热迁移机制与智能调度算法，构建适应 5G 核心网动态运维需求的解决方案。

1.2 国内外研究现状及发展趋势

近年来，随着 5G 核心网向云原生架构加速演进，围绕网络功能虚拟化、网络切片、资源调度与服务迁移等关键技术，国内外学术界和工业界展开了广泛而深入的研究^[9-10]。在多租户、多业务并发承载的新型网络形态下，如何实现核心网功能的高效部署、按需迁移和智能编排，并在此基础上保障各租户和业务的安全隔离与数据安全，已成为当前网络架构设计的重要课题。尽管已有大量研究聚焦于用户面功能优化与调度，然而针对控制面网元，尤其是其容器化部署与迁移策略的系统性研究仍相对薄弱^[11]。特别是从保障关键安全功能连续性和切片安全隔离的角度出发的研究尚不充分。为进一步厘清本研究选题的定位与创新点，本文从以下几个方向梳理国内外相关研究工作。

1.2.1 虚拟网络功能调度

网络功能虚拟化将原本依赖专用硬件的网络功能以软件形式部署在通用服务器上，实现了网络功能的灵活部署和快速迭代^[3]。国内外学者普遍将 VNF 部署规划视为一个组合优化问题，目标是在满足服务性能指标的前提下最小化资源开销^[12-15]。Zu 等人对 NFV 环境下服务功能链资源管理的研究现状进行了综述^[16]。他们系统梳理了服务功能链的组链、部署、调度和弹性伸缩等环节的优化方法，总结了各类 VNF 调度与资源分配框架与算法，分析了当前存在的问题并展望了未来的发展方向。Z Liu 等人^[17]针对虚拟网络中对时延敏感的业务功能链上的虚拟网络功能的映射、调度和路由问题，提出了一种 VNF 调度算法 D3QN，结合 Dueling Networks 和 Double DQN 优化 VNF 映射、调度和路由，降低 8% 的 SFC 拒绝率，适用于时延敏感场景。Zhu 等人^[18]提出了一种面向容器云异构工作负载的优先级感知调度框架 PA-CCWS，通过工作负载特征分析、TOPSIS 优先级量化及结合 DDQN 的深度强化学习，优化了资源利用率、负载均衡和 SLA 符合率，相较于 PSOS、KCSS 等算法展现出更优的性能和收敛稳定性。X. Huang 等人^[19]提出了一种预测性在线服务链和资源调度方案 POSCARS，

将复杂的全局优化问题解耦为多个可并行求解的子问题，在仅依赖有限未来信息的前提下实现近似最优调度决策，通过可调节的系统指标权衡实现了队列稳定性保证。Thien 等人采用改进的多目标遗传算法（NSGA-III）来优化 VNF 的放置和调度^[20]。他们将 VNF 映射与调度建模为同时最小化网络链路时延、各虚拟机处理负载以及处理时延的三目标优化问题，并提出 VNF-NSGA-III 算法，在同构和异构数据中心环境下的实验表明该算法优于其他启发式和元启发式方法。Jia 等人针对随机到达的 SFC 请求和节点失效场景，提出了基于强化学习的可靠性感知服务链动态调度算法^[21]。该方法在 5G 网络中通过强化学习智能决策 VNF 部署与迁移，实现了服务链的高可靠调度，在保证端到端时延约束的同时提升了请求成功率。Guo 等人设计了一个深度强化学习框架，实现优先级权重延迟约束下的 VNF 协同调度^[22]。该方法针对不同服务请求的紧急程度分配调度优先权，显著提高了多服务功能链环境下请求的接收率，降低了平均处理延迟并改善了负载均衡。这些优化方法大多聚焦于性能和资源效率，对核心网场景下控制面网元迁移和调度过程中可能引入的安全风险或如何保障隔离性的讨论较少。

1.2.2 网络切片智能编排策略

网络切片的编排过程需在多个维度动态分配资源，确保每类业务切片的 QoS 目标得到满足。M. Gharbaoui 等人^[23]针对面向低时延业务的网络切片编排问题，提出了一种支持重试机制的多策略资源编排框架，该框架综合考虑处理时延与网络传输时延两个维度协调调度计算、存储与网络资源，设计了三种编排策略用于在不同业务目标下选择最优资源路径。Han 等人提出了面向多业务租户的网络切片编排策略，通过建立不耐烦租户多队列排队模型优化切片的接入控制^[24]。该方法结合队列论推导不同租户请求的丢弃率和资源占用情况，并利用近似算法实现切片资源的动态优化，提高了多租户环境下切片接受率和系统吞吐量。Moreira 等人^[25]提出了一种基于机器学习的网络切片架构特征智能编排方法，通过在 SFI2 架构中嵌入 ML 代理和联邦学习机制，优化切片资源分配与功能编排，并开发了分布式 ML 代理的攻击预防安全机制，展示了数据驱动方法在网络切片过程中的重要性和应用效益。Kovacevic 等人研究了多域环境下的网络切片映射编排机制，提出了一种延迟均衡策略来在跨域资源分配中保证时延的一致性^[26]。他们设计了分层的编排框架，将端到端切片映射到多个域的物理资源，并通过调整各域资源分配使不同域间的切片通信时延得以均衡，从而满足严

格的服务时延要求。N Salhab 等人^[27]提出了一种利用机器学习技术进行 5G 网络切片资源编排的框架，通过回归树分类与预测、背包问题形式化的准入控制与切片调度，以及基于深度强化学习的自适应资源管理器，优化了资源利用率和网络吞吐量，在基于 OpenAirInterface 的实验中优于静态切片和随机切片方法。Mijumbi 等人^[28]构建了一种面向服务请求的切片生命周期管理框架，结合启发式部署算法和策略驱动的资源回收机制，实现了对 eMBB 与 URLLC 切片的差异化支持。M. Gharbaoui 等人针对面向低时延业务的网络切片编排问题，提出了一种支持重试机制的多策略资源编排框架。Z. Saleh 等人^[29]提出一种结合 DQN 的切片资源编排模型 DDQN-PER，将切片调度建模为多步决策过程，能够根据实时网络负载动态调整切片资源占比。该模型在仿真中展现出优于传统基于启发式算法的切片编排方法的性能。

1.2.3 容器/VM 热迁移机制

Pod 与虚拟机的热迁移机制是实现服务连续性、故障恢复与负载重分布的重要基础。Raseena M. Haris^[30]对网络环境下虚拟机实时迁移过程中的内存迁移挑战进行了系统综述，指出高可用服务对迁移中断时间、迁移数据量和总耗时提出了严格要求。该文总结了当前主流 LVM 技术在内存预拷贝、增量迁移与压缩传输等方面的优化策略，并深入分析了多策略组合优化在提升迁移效率、降低网络负载方面的潜力。在容器环境中，Andrei Vagin 等人^[31]主导开发了 CRIU (Checkpoint/Restore In Userspace) 框架，支持 Linux 进程的运行时快照与恢复，为 Pod 级别迁移提供了底层机制支撑。Biswas 等^[32]基于 OpenStack 平台设计了 VNF 在线迁移机制，结合 Open vSwitch 实现状态同步，有效降低了迁移过程中的数据丢失概率。R. Nishtala 等人^[33]针对 KVM 虚拟机迁移中的中断问题，提出了一种改进的预拷贝算法，通过动态预测脏页写入行为减少传输轮数，显著缩短迁移总时长。针对 CPU 密集型服务刷存速度快于快照传输速度的问题，Stoyanov 等人^[34]提出了一种高效的 Linux 容器热迁移方法，通过利用 CRIU 的“图像缓存/代理”功能，避免使用二级存储，显著减少了容器迁移过程中因内存修改频繁而导致的迁移时间和应用停机时间。Poggiani 等人^[35]提出了一种在多集群 Kubernetes 边缘系统中实现多容器 Pod 热迁移的解决方案，利用 Liko 开源项目^[36]建立集群间对等关系，迁移单容器和多容器本地状态实例，无需修改 Kubernetes 标准 API，并通过实验验证了其在边缘计算中的有效性和实用性。Ahmad 等人设计了 Neutrino，一个面向边缘部署的快速一致性蜂窝控制面架构^[37]。Neutrino 通过异步复

制结合基于重放的机制在边缘维护用户状态副本，实现了对控制面故障的容错，在提供可靠接入的同时将控制面时延显著降低，相比传统集中式方案大幅提升了控制信号处理的速度与一致性。此外，Ahmad 等人在此基础上提出了 CellClone 架构，引入多活控制面副本以进一步加速故障恢复^[38]。CellClone 结合定制的快速状态同步协议，在出现控制面节点故障时将控制信令延迟减少了 2.8 倍以上，在无故障情况下性能与现有方案相当但可靠性大幅提升，使得自动驾驶、虚拟现实等边缘应用的超低时延通信成为可能。

1.3 研究目标和成果

本研究旨在针对云原生 5G 核心网智能运维中的关键问题，提出系统性解决方案，本文主要成果如下：

（1）设计并实现了基于 Clone-Restore 的网元热迁移方案

本研究提出了一种针对云原生 5G 核心网有状态网元的热迁移方案，通过对 Kubernetes 基础设施的扩展，实现了 Pod 运行时中容器状态与非容器状态的增量克隆与恢复，赋予了云原生集群高效的 Pod 热迁移能力。该方案在迁移时刻对源 Pod 进行状态检查点，并在目标节点启动克隆 Pod 恢复运行，从而将控制面网元无缝转移至新节点。该方案特别关注了 AMF 等承载关键安全功能的网元，通过状态的完整克隆与恢复，确保了用户认证会话、安全上下文等敏感信息的连续性，避免了因迁移导致的安全状态丢失和潜在的安全风险。为协调迁移过程中的资源调度和状态同步，开发了定制的智能迁移控制器，集成于 Kubernetes 控制平面，实现对控制面 Pod 迁移的自动触发和全过程管控。实验测评表明，该迁移机制能够将控制面服务中断时间控制在秒级以内，显著提升控制面运维效率和安全功能的服务连续性。

（2）提出了基于 DQN 的切片智能调度算法

本研究针对网络切片控制面网元编排问题，设计了一种基于深度强化学习的调度编排算法，综合优化控制面路径时延和节点负载均衡。算法通过马尔可夫决策过程建模切片调度问题，利用改进的深度 Q 网络生成高效的调度与迁移动作。算法结合离线训练和在线决策，适应基础网络环境的动态变化，同时优化网络切片信令时延并提高集群资源负载度。通过对网络拓扑和多维资源状态的感知，该算法在调度决策时能更好地避免不同切片间的资源冲突，为实现更可靠的切片安全隔离提供了基础。此外，本章在模拟环境中对算法进行了全面的测试和性能评估，通过与基线算法的比较证明

了策略的有效性和优越性。

(3) 设计并实现了面向核心网运维场景的 K8s 智能切片调度器

本研究在 (1) 中实现的扩展热迁移能力的 Kubernetes 集群基础上以 (2) 中实现的调度算法为核心设计了一套智能调度器架构。该调度器通过感知资源使用、网络拓扑以及业务负载等信息, 综合考虑服务时延和资源负载均衡两个目标, 对控制面 Pod 的调度策略进行优化。研究采用改进的启发式算法实现多目标调度决策, 使调度器能够根据实时网络状态动态选择网元部署节点。在仿真和实际测试中构建部署 5G 网络切片任务测试场景, 围绕资源利用率、负载均衡、调度延迟、信令处理效率和响应稳定性五个维度开展了性能对比实验。结果显示, 相比 Kubernetes 默认调度策略, 智能调度器能够更充分利用系统资源, 显著降低负载不均衡问题, 同时优化关键路径信令时延和用户终端 (User Equipment, UE) 注册延迟, 具备更高的时延稳定性。热迁移方案通过动态增量快照和状态克隆技术, 将网元迁移中断时间控制在秒级以内, 保障了服务的连续性。

1.4 论文组织结构

本文分为六个章节, 每章内容安排如下:

第一章为绪论。介绍了研究的背景与意义, 分析了国内外研究现状, 明确了研究目标和成果, 并阐述了论文的组织结构, 为后续研究奠定了基础。

第二章阐述论文涉及的理论和技术基础。首先, 介绍了云原生技术的核心原则及其在 5G 核心网部署中的高弹性优势; 其次, 分析了 Kubernetes 容器编排技术的架构、Pod 管理、网络模型和调度机制, 阐明其在支持动态调度和热迁移中的关键作用; 最后, 聚焦 5G 核心网架构, 探讨了控制面与用户面分离、服务化架构及关键网元功能, 并通过网络切片技术分析 5G 对低时延和高可靠性的需求。本章指出现有技术的局限性, 为后续研究提供了方向。

第三章提出云原生核心网网元热迁移策略。设计并实现了一种云原生 5G 核心网有状态网元热迁移方案, 通过扩展 Kubernetes 的 CRI 等组件, 实现 Pod 状态的增量克隆与恢复, 并设计网元热迁移控制体系确保状态一致性。实验对比了 Clone-Restore 方案与传统 Delete-Recreate 方案, 验证了其在迁移时延、会话连续性和资源效率方面的有效性。

第四章提出了云原生 5G 核心网智能切片调度算法。以马尔可夫决策过程为理论

基础，设计了一种基于深度强化学习的调度策略，优化网元调度与迁移决策。该策略感知多维度集群资源和节点时延变化，适应核心网控制面网元的动态运行环境，同时提升切片端到端时延和集群负载均衡度。此外，本章还构建了一个虚拟实验环境，对提出的切片调度策略进行全面测试和评估，并通过与基线方法的比较，证明了策略的有效性，为电信运营商优化切片网元调度提出了新的解决方案。

第五章设计并实现了面向云原生 5G 核心网的智能调度器。通过整合前两周章的研究成果，设计并实现了用于核心网运维的 Kubernetes 集群调度器，拓展了 Kubernetes 的调度策略和调度动作。智能调度器集调度与迁移决策于一体，包含状态采集、策略决策和动作执行三大核心模块。实验验证了其在资源利用率、负载均衡、调度延迟和响应稳定性等方面的性能优势，展现了适应多变业务负载的能力。

第六章为结论。总结了论文的主要研究成果和创新点，分析了研究的局限性，并提出了未来研究方向，为云原生 5G 核心网智能运维的进一步发展提供展望。

论文整体架构如图 1.1所示。

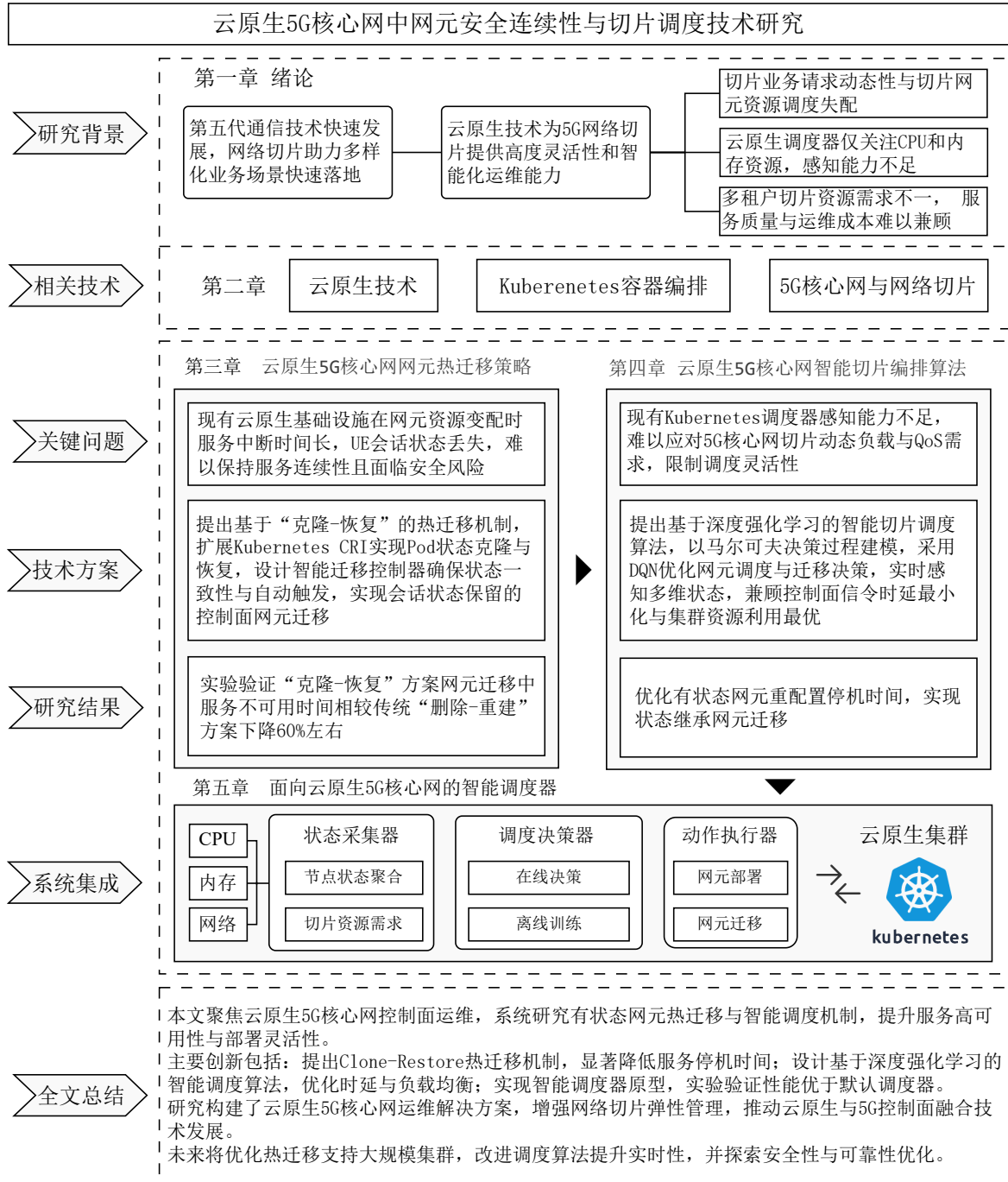


图 1.1 论文组织结构图

第 2 章 涉及的理论和技术基础

2.1 云原生技术

云原生（Cloud Native）作为一种先进的软件开发与部署范式，其核心目标在于充分发掘并利用云计算环境的固有优势，显著提升应用系统的可扩展性、弹性和可靠性。根据云原生计算基金会（Cloud Native Computing Foundation, CNCF）的权威定义，云原生旨在通过运用容器化封装、微服务架构拆分以及声明式 API 驱动等关键技术手段，构建并运行能够在现代动态环境中弹性伸缩的应用^[39]。云原生技术的蓬勃发展，为以 5G 核心网为代表的电信领域的自动化、智能化运维演进提供了坚实的技术支撑，有效满足了其高并发处理、低时延响应和高可用保障的核心需求^[40]。

2.1.1 云原生的定义与核心原则

云原生技术的精髓在于运用一系列现代化的技术栈与设计哲学，以实现应用全生命周期的高效管理，涵盖快速开发、敏捷部署与自动化运维等多个维度^[41]。其关键原则主要体现在以下几个相互关联的方面：

首先，**容器化**是云原生的基石。应用及其所有运行时依赖被统一封装在标准的容器镜像中，这种机制确保了从开发、测试到生产环境的高度一致性。以 Docker^[42]为代表的容器技术凭借其轻量级虚拟化特性，不仅实现了资源的有效隔离与应用的便捷迁移，同时也显著降低了传统虚拟化带来的额外资源开销。在此基础上，**微服务架构**应运而生。它倡导将复杂的单体应用细粒度地拆分为一组小型的、功能内聚且松散耦合的服务单元。每个微服务聚焦于特定的业务能力，并通过标准化的 API 接口进行通信。这种架构模式极大地增强了系统的模块化程度，使得各服务能够独立地进行开发、部署、扩展和维护，从而提升了整体的灵活性和可维护性。

为了有效管理由大量容器和微服务构成的复杂系统，云原生引入了**声明式 API**。开发者或运维人员通过声明式配置文件精确描述系统的期望终态，而非具体的操作步骤。随后，自动化控制系统便会持续监控系统当前状态，并自动执行必要的操作以确保系统状态与声明保持一致。这种基于状态而非过程的管理方式，显著降低了系统运维的复杂度，并为动态调整和自动化管理提供了基础。基于声明式 API 和自动化管理能力，云原生系统得以实现高效的**弹性扩展**。系统能够依据实时的负载状况，自动增加或减少服务实例，通过水平扩展（Horizontal Scaling）的方式灵活应对业务流量的

峰值和低谷，从而保障服务的持续可用性和性能稳定^[43]。

最后，整个应用生命周期通过**自动化运维**实践得以优化。结合持续集成/持续部署（Continuous Integration/Continuous Deployment, CI/CD）流水线以及先进的监控告警工具，云原生环境能够实现应用代码的自动化构建、测试、部署乃至故障自愈，最大限度地减少了人工干预的需求和潜在错误，降低了运维成本。

云原生架构的一个典型形态如图 2.1 所示^[44]，直观地展示了容器层、微服务层以及编排管理层之间的协同工作关系。容器层提供标准化的运行时环境，微服务层承载具体的业务逻辑实现，而上层的编排工具则扮演着“大脑”的角色，负责资源的智能调度、服务的生命周期管理以及整体系统状态的维护。这些核心原则共同塑造了云原生技术体系，为构建和运维现代分布式系统提供了强大的理论指导和实践框架，尤其适用于处理高动态性、高并发和高可靠性要求的复杂应用场景^[44]。在本文所关注的 5G 核心网领域，云原生技术的应用使得网络功能能够实现敏捷部署与弹性伸缩，这为后续章节探讨的网元热迁移和智能调度等高级运维技术奠定了不可或缺的基础。

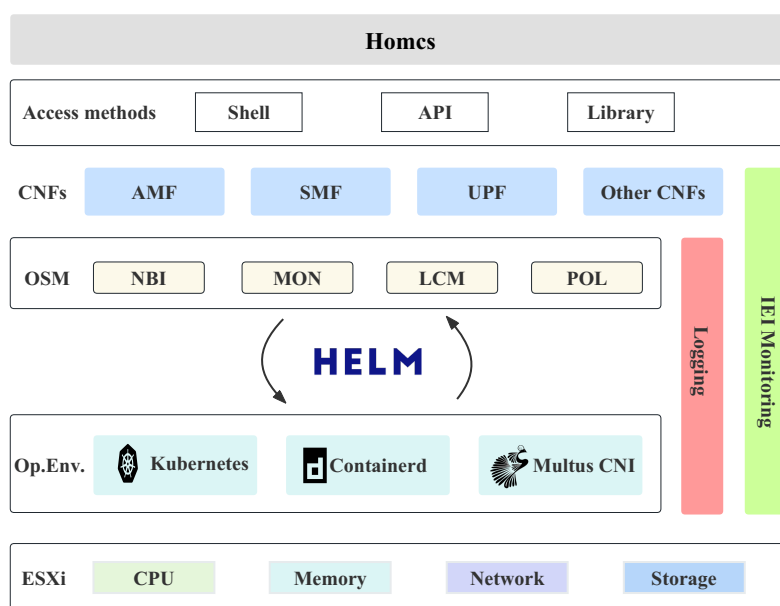


图 2.1 云原生架构示意图

2.1.2 云原生技术的典型应用场景

凭借其卓越的灵活性与效率优势，云原生技术已渗透到众多行业和领域，并在以下几个典型场景中得到了广泛且深入的应用：

其中最为广泛的是在**云计算与微服务架构**领域。云原生已成为现代云计算平台上

构建和部署应用的标准模式。AWS 和阿里云等国内外公有云头部厂商都大规模采用 Kubernetes（简称 K8s）等云原生技术来管理其庞大的微服务集群，通过动态调度数以万计的容器实例，高效应对全球范围内用户的海量并发访问请求，确保了服务的稳定性和用户体验^[45-46]。

在电信领域，特别是 5G 核心网部署方面，云原生技术正扮演着日益关键的角色。面对 5G 业务多样化（eMBB、URLLC、mMTC）带来的差异化服务质量需求以及网络流量的剧烈波动，传统网络架构难以为继。5G 核心网设计伊始便积极拥抱云原生理念，通过将核心网网元（AMF、SMF、UPF 等）进行容器化封装，并借助服务化架构和 Kubernetes 等编排平台进行管理，实现了网络功能的高弹性扩展、按需部署和自动化运维。这不仅支持了网络切片（详见第2.3.3节）的动态创建与资源隔离，也显著提升了网络部署的灵活性和资源利用效率。例如，中国移动等领先运营商已基于 Kubernetes 成功构建了云原生 5GC 商用网络，验证了该技术路径的可行性与优越性^[47]。

云原生技术不仅是本文研究背景的重要组成部分，更是后续技术方案得以实现的关键赋能者。具体而言，5G 核心网网元的容器化部署和微服务化改造，为第三章的 Pod 热迁移工作提供了必要的可行性基础，使得服务状态的捕获与恢复成为可能，进而支持服务的无中断或低中断迁移。同时，Kubernetes 平台提供的开放 API 和可扩展架构，以及其内建的调度框架，也为第四章设计基于深度强化学习的 DQN 智能调度算法和第五章实现定制化的 K8S 调度器扩展提供了基础平台与交互接口。通过引入更智能的调度策略，有望克服默认调度器在复杂 5G 场景下的局限性，从而显著提升核心网资源的分配效率和整体服务质量。

2.2 Kubernetes 容器编排技术

在云原生技术体系中，容器编排是实现应用自动化部署、扩展和管理的核心环节。Kubernetes 作为其中的佼佼者，已成为业界公认的容器编排标准^[48]。它提供了一个健壮的、可扩展的平台，用于管理容器化的工作负载和服务，极大地简化了分布式系统的运维复杂性。本节将系统阐述 Kubernetes 的关键技术，包括其系统架构、核心对象 Pod、网络模型以及调度机制，为后续章节中 5G 核心网网元热迁移和智能调度的技术实现奠定基础。

2.2.1 Kubernetes 架构与核心组件

Kubernetes 采用分布式主从架构，逻辑上分为控制平面（Control Plane）和由工作节点 Node 组成的数据平面，如图 2.2 所示。控制平面负责集群的全局决策和状态管理，而数据平面则运行具体的应用容器。

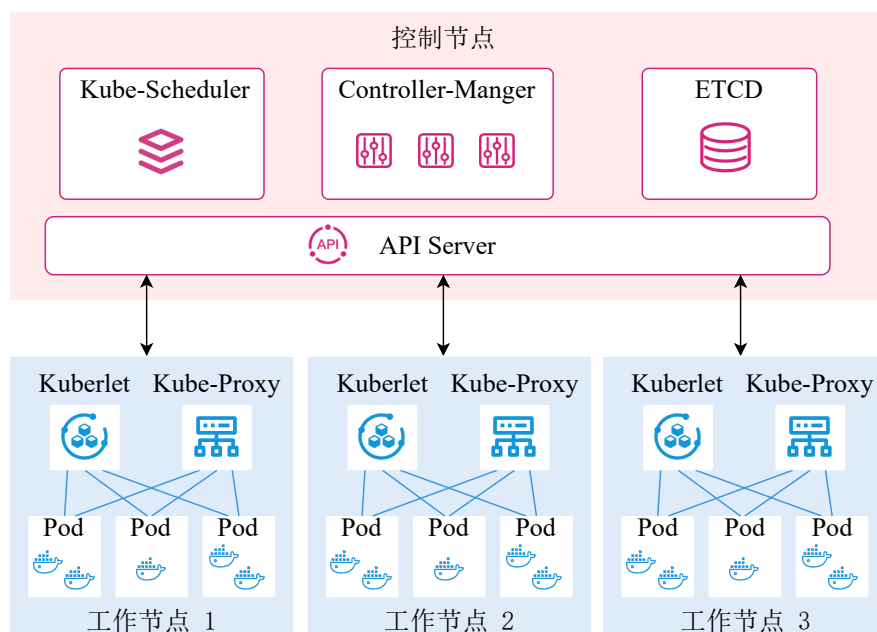


图 2.2 Kubernetes 架构示意图

控制平面由多个核心组件协同工作，以维持集群的期望状态。核心组件包括 kube-apiserver、etcd 和 kube-controller-manager。其中，kube-apiserver 是 Kubernetes API 的统一入口，处理 REST 请求并验证数据对象；etcd 是一个高可用分布式键值存储，持久化存储集群配置和状态信息，构成集群的“事实来源”；kube-controller-manager 运行节点控制器、副本控制器等多个控制器，通过 API Server 监控集群状态并驱动实际状态向期望状态收敛。此外，若集群部署在云平台上，cloud-controller-manager 可与云服务提供商交互，管理负载均衡器和存储卷等资源。调度器 kube-scheduler 负责 Pod 的节点分配，其机制将在第 2.2.3 节详细讨论。

数据平面的每个工作节点运行 kubelet 和 kube-proxy，并依赖容器运行时执行任务。kubelet 是节点上的代理，与控制平面通信，负责 Pod 的创建、监控和销毁；kube-proxy 维护网络规则，实现 Service 的通信和负载均衡；容器运行时则负责镜像拉取和容器生命周期管理，其细节将在下一小节阐述。这种分布式架构赋予 Kubernetes 高可用性、可扩展性和自动化管理能力^[48]，为 5G 核心网的高并发场景提供了可靠的运行

环境。

2.2.2 Pod 与容器运行时

Pod 是 Kubernetes 中最小的部署与调度单位，其结构和运行特性直接影响迁移方案的设计与实现。本小节首先分析 Pod 的组成结构与生命周期，进而对比 Pod 与传统虚拟机和独立容器在架构与状态上的差异，阐明 Pod 在云原生环境下的独特需求及热迁移面临的挑战，为第 3 章的 AMF 网元迁移提供技术背景。

2.2.2.1 Pod 结构

Pod 的主体通常由一个或多个容器组成，这些容器共享 Pod 级别的网络命名空间和存储资源，能够通过本地主机进行直接通信，并共享数据卷以实现数据的持久化存储。容器的共享机制使得 Pod 内的各个容器能够紧密协作，共同完成特定的应用功能，如 Web 服务器与日志收集器的组合应用。除了容器本身之外，Pod 还包含元数据和运行时配置两个重要组成部分，这些内容在热迁移过程中同样需要被精准地保存和恢复。

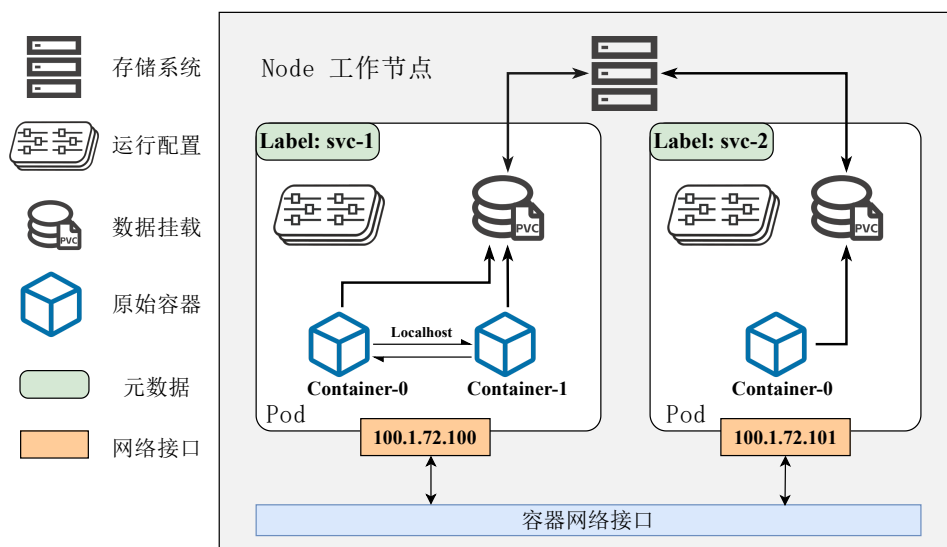


图 2.3 Pod 结构示意图

Pod 的元数据主要由标签 (Labels) 和注解 (Annotations) 构成，存储于 Kubernetes API 服务器，与 Pod 生命周期紧密绑定。标签以键值对形式标识 Pod 属性，用于调度、服务发现和负载均衡；注解则存储非结构化信息，供外部工具解析，直接影响集群管理和迁移状态的维护。运行时配置则涵盖 Pod 的动态状态，包括网络状态 (IP 地址、端口映射、TCP 会话)、存储状态 (挂载卷) 以及环境配置 (环境变量、启动命令、镜

像版本)。这些配置赋予 Pod 显著的动态特性, 使其运行时状态远超单一容器进程。

相较于传统虚拟机, Pod 的运行时状态更加轻量和集成。虚拟机通过 Hypervisor 技术^[49]虚拟完整操作系统, 迁移需处理大量内存快照和磁盘镜像, 数据量大且过程复杂。而 Pod 依托容器技术, 共享宿主内核, 仅封装应用及其依赖, 状态数据量较小, 迁移效率更高。然而, 这种轻量设计对网络和存储状态的实时捕获与一致性恢复提出了更高要求。相比独立容器, Pod 的运行时状态更复杂且依赖集群环境^[50]。独立容器仅关注自身进程和文件系统, 迁移时只需捕获内部状态快照; 而 Pod 作为多容器协作单元, 其状态包括多容器协作关系、与集群服务的交互以及元数据, 热迁移需全面捕获这些复合状态, 确保功能一致性。

2.2.2.2 Pod 生命周期

Pod 的生命周期涵盖从创建到终止的全过程, 由控制平面和工作节点的组件协同管理。图 2.4展示了 Pod 的状态转移流程。当用户提交 Pod 创建请求时, Pod 进入 *Pending* 状态, 等待调度器分配节点或拉取容器镜像。调度完成后, Pod 转入 *Running* 状态, 表明至少一个容器正在运行。若所有容器正常终止且无重启策略, Pod 进入 *Succeeded* 状态; 若容器因故障终止且无法重启, 则进入 *Failed* 状态。若因节点通信故障无法获取状态, Pod 被标记为 *Unknown*。

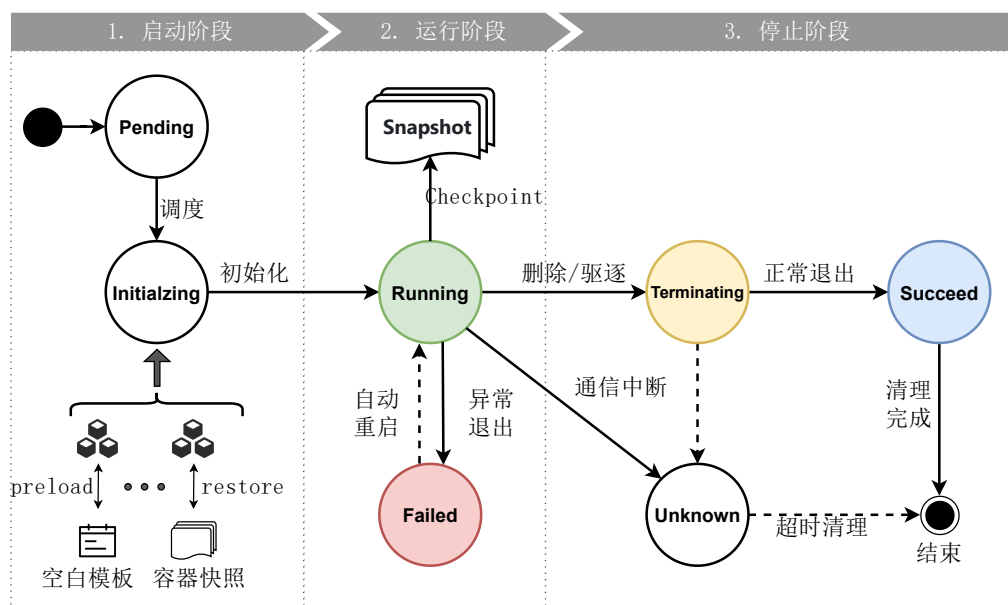


图 2.4 Pod 生命周期

Pod 生命周期由多个组件共同协调。API Server 处理用户请求, 控制状态转换; 调度器根据资源需求、节点状态和亲和性规则选择运行节点; kubelet 在节点上管理容器

运行，包括启动、重启和停止等操作。然而，Kubernetes 设计中 Pod 被视为短暂实体，故障时通常通过删除重建而非修复来实现高可用性。这种策略在 5G 核心网有状态应用中会导致状态中断和大量失败请求，显著影响 QoS。因此，Pod 热迁移成为保障服务连续性的关键技术，需优化默认创建机制以支持状态迁移。

在 Kubernetes 的设计中，Pod 被视为相对短暂和易于替换的实体^[50]。集群管理策略通常是在容器或 Pod 失败时创建新的 Pod，而不是试图修复。这种设计哲学支持了高可用性和可伸缩性，但对于核心网网元等带状态运行的工作负载来说，删除重建的过程将会导致状态中断从而引发大量失败请求，这会对服务 QoS 造成显著影响。因此 Pod 热迁移对于云原生集群上的有状态应用高可用性保障有着不可或缺的作用。

Pod 内容器的实际运行由节点上的容器运行时负责。为了解耦 kubelet 与具体的容器运行时实现，Kubernetes 定义了容器运行时接口（**Container Runtime Interface, CRI**）^[51]。CRI 是一套基于 gRPC 的服务接口规范，kubelet 通过调用 CRI 接口来管理容器和镜像的生命周期，而无需关心底层的容器技术细节。这使得 Kubernetes 可以支持多种符合 CRI 标准的容器运行时。**Containerd** 是一个业界广泛采用的、从 Docker 项目中分离出来的、并已在 CNCF 毕业的核心容器运行时。它专注于容器生命周期管理的核心功能，具有轻量、健壮和可扩展的特点，是当前 Kubernetes 环境中主流的容器运行时选项之一。

2.2.3 Kubernetes 调度机制

Kubernetes 调度器（kube-scheduler）的核心职责是将新创建的 Pod 分配到集群中最合适的可用节点上运行。理解其调度机制对于优化资源利用率、满足应用性能需求以及实现高级调度策略至关重要。Kubernetes 原生调度流程如图 2.5 所示^[52-53]，调度器通过 API Server 获取待调度的 Pod 对象，并根据集群状态和调度策略进行节点选择。

调度过程通常分为三个主要阶段：

1. **过滤（Filtering）**：在此阶段，调度器遍历集群中的所有可用节点，应用一系列预定义的“断言（Predicates）”规则来筛选出能够满足 Pod 运行条件的节点。这些条件可能包括：节点是否有足够的计算资源、是否满足 Pod 请求的特定标签（Node Selector）、是否满足亲和性（Affinity）或反亲和性（Anti-affinity）规则、是否能容忍节点的污点（Taints）等。无法满足任一条件的节点将被过滤掉。
2. **评分（Scoring）**：对于通过过滤阶段的所有候选节点，调度器会应用一系列“优

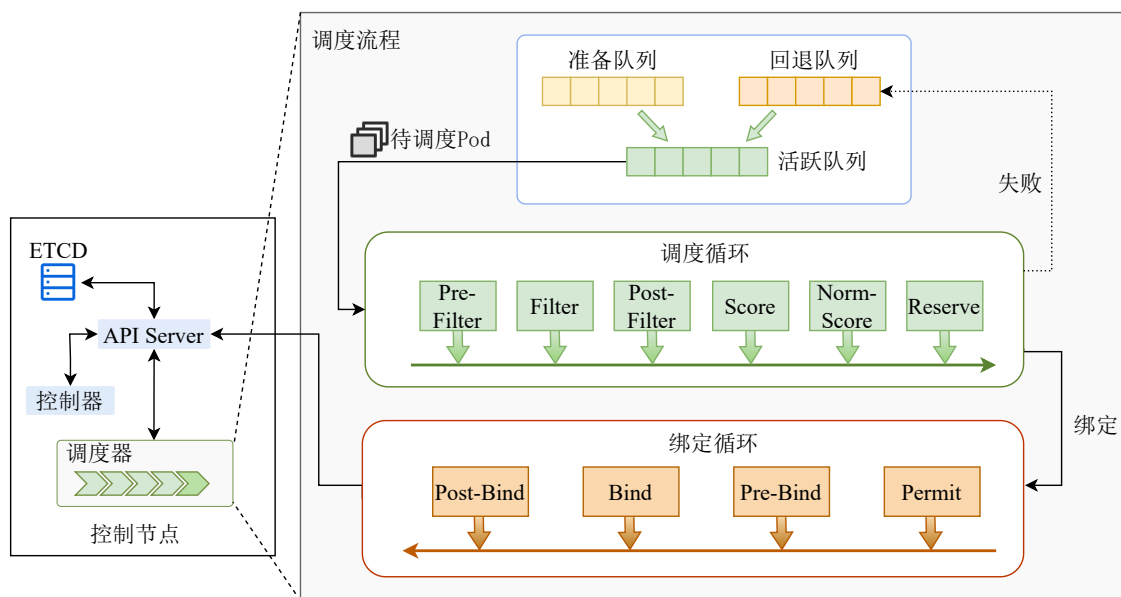


图 2.5 Kubernetes 调度流程示意图

选（Priorities）”规则（在新版本中称为 Score 插件）对它们进行打分。每个规则会给节点赋予一个分数，表示该节点对于运行该 Pod 的“适合程度”。评分规则旨在优化某些目标，例如资源利用率的均衡、优先部署到已存在相关镜像的节点、满足 Pod 间的亲和性要求等。调度器会汇总所有规则的分数，得到每个节点的最终得分。

3. **绑定 (Binding):** 调度器从评分结果中选择得分最高的节点，并将 Pod 与该节点进行绑定。绑定操作实际上是调度器通知 API Server 更新 Pod 对象的“nodeName”字段。一旦绑定成功，目标节点上的 kubelet 就会接管该 Pod，开始拉取镜像并创建容器。

Kubernetes 的默认调度器采用了一套通用的过滤和评分规则，旨在实现相对均衡和高效的资源分配。例如，在评分阶段，常见的策略如 **LeastRequestedPriority** 会优先选择请求资源总量较少的节点，以期望将负载分散到各个节点；而 **BalancedResourceAllocation** 则倾向于选择 CPU 和内存使用率都比较均衡的节点。然而，这些通用策略往往是基于节点当前的资源请求量或使用量进行决策，可能无法充分考虑到应用的特定需求、网络拓扑的影响、或者集群整体的能耗等复杂因素。这正是本论文后续章节引入基于深度强化学习的智能调度算法以优化 5G 核心网场景下资源调度效果的研究动机所在。

2.3 5G 核心网架构与网元功能

5G 核心网（5GC）是 5G 网络的关键组成部分，负责用户接入、会话管理、数据转发及服务质量（QoS）保障，其架构设计直接影响网络性能和服务能力。本小节首先介绍 5GC 的体系结构，重点分析控制面与用户面分离（CUPS）和服务化架构（SBA）的核心理念，随后阐述关键网元的功能与交互，最后探讨网络切片技术及其对 QoS 的支持。通过系统分析 5GC 的架构特性，为第 3 章的网元热迁移和第 4 章的智能调度研究奠定技术基础。

2.3.1 5G 核心网体系结构

5GC 采用模块化与高灵活性的架构设计，通过 CUPS 和 SBA 实现高效的资源管理和网络优化，以满足 5G 网络高吞吐量、低时延和高可靠性的需求^[2]。图 2.6 展示了 5GC 的整体架构及其网元间的交互关系。

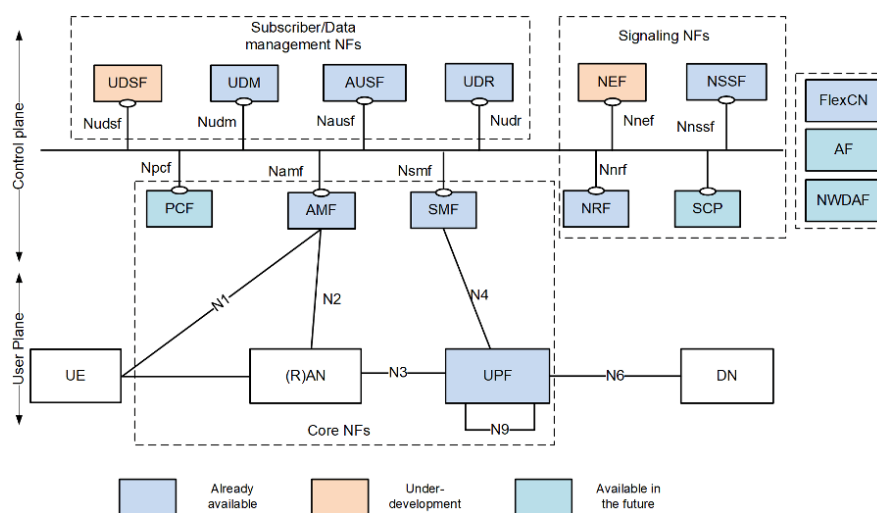


图 2.6 5G 核心网架构示意图

控制面与用户面分离（CUPS）是 5GC 架构的核心特性，通过将控制面和用户面功能解耦，显著提升网络灵活性和性能。控制面负责信令处理，包括用户接入、会话管理和移动性管理，而用户面专注于数据转发，支持高吞吐量和低时延传输。CUPS 通过标准化接口实现控制面与用户面的协同，允许两者独立扩展和部署。例如，控制面网元可集中部署以优化管理效率，用户面网元则可分布式部署至边缘节点以降低时延^[2]。这种分离设计为云原生环境下的动态调度提供了技术基础，特别是在第 3 章中，CUPS 支持舍弃了冗余的会话状态管理需求，支持 AMF 等控制面网元的热迁移，保障服务连续性。

服务化架构（SBA）进一步增强了 5GC 的模块化和可扩展性。SBA 基于服务化思想，将网元功能分解为独立的服务，通过 RESTful API 和 HTTP/2 协议的服务接口进行通信。网元通过网络功能存储库（NRF）动态注册和发现服务，实现灵活的服务调用。例如，AMF 通过 SBA 接口与 SMF 协作完成会话建立，提升系统可维护性和扩展性^[2]。SBA 的松耦合特性为云原生部署提供了便利，尤其在第 3 章的网元热迁移中，支持网元间的动态协调；在第 4 章和第 5 章的网络切片调度中，SBA 通过细粒度资源分配优化 QoS 保障。

CUPS 和 SBA 的结合为 5G 网络切片技术提供了架构支持。网络切片通过在统一物理基础设施上按需创建逻辑网络，满足多样化的业务场景和 QoS 需求^[2]。例如，低时延切片可优先分配边缘节点资源，高吞吐量切片则优化带宽分配。

2.3.2 核心网网元功能与交互

5GC 包含多个关键网元，协同工作以支持用户接入、会话管理和数据传输，主要包括 AMF、SMF、UPF、UDM、AUSF 和 UDR。AMF（接入与移动性管理功能）负责 UE 的注册、连接管理和移动性管理，通过 NGAP 协议与 gNB（the next Generation Node B，下一代基站）交互，通过 NAS 协议与 UE 通信，是热迁移（第 3 章）的核心对象。SMF（会话管理功能）管理 PDU 会话的建立、修改和释放，与 UPF 协作处理用户数据转发。UPF（用户平面功能）负责数据包的路由和转发，支持流量优化和 QoS 保障，常部署于边缘以降低时延。UDM（统一数据管理）存储用户数据和签约信息，AUSF（认证服务器功能）执行用户认证，UDR（统一数据仓库）提供数据存储服务。这些网元通过 SBA 接口或专用接口交互，共同实现 5G 服务^[2]。

AMF 的高并发特性对服务连续性要求极高。在云原生部署中，AMF 以 Pod 形式运行，其迁移需确保会话状态不中断，同时调度需满足低时延和高吞吐量需求。网元间的动态交互进一步凸显了热迁移和智能调度的重要性。

2.3.3 5G 网络切片技术

网络切片（Network Slicing）是 5G 网络的重要特性，允许网络运营商在统一的物理基础设施上逻辑隔离地部署多个虚拟网络实例，以满足不同业务场景的特定 QoS 需求^[54]。网络切片通过网络功能虚拟化（NFV）和云原生技术实现资源灵活分配，支持服务差异化^[2]。

网络切片的设计理念是将物理网络划分为多个逻辑网络，每个逻辑网络称为一个

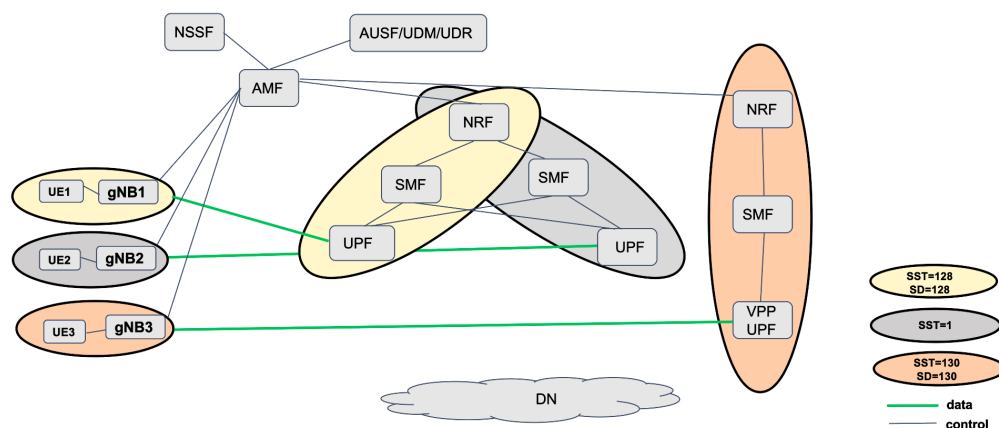


图 2.7 网络切片示意图

切片（Slice），如图 2.7所示。每个切片可以根据特定的业务需求进行定制，支持不同的 QoS、带宽、延迟和可靠性要求。切片之间相互独立，资源隔离，避免了传统网络中资源竞争和干扰的问题。

根据 3GPP 标准^[2]，网络切片按照业务特性，一般可分为增强型移动宽带（eMBB）、超可靠低时延通信（URLLC）和大规模机器类通信（mMTC）三大典型类型。eMBB 切片提供高速率、高容量的数据传输服务，适用于高清视频、虚拟现实等场景；URLLC 切片则保证极低时延和高可靠性，适用于自动驾驶、工业控制等时延敏感业务；mMTC 切片则支持海量设备连接，适用于物联网（IoT）和智慧城市建设。QoS 分类标准明确了各类切片的时延、可靠性、带宽等指标要求，为不同的业务应用提供了清晰的网络质量保障框架^[55-57]。5G 网络切片技术不仅提高了网络资源的使用效率，还为新型业务模式的实现提供了技术支撑。

5G 网络切片的部署模式主要包括完全隔离、部分共享和完全共享三种形式，以平衡资源效率和服务隔离需求^[2]。完全隔离模式为每个切片分配独立的核心网网元和资源，适用于对隔离性要求极高的场景，但资源利用率较低。完全共享模式则允许多个切片共享所有网元和资源，适用于资源受限场景，但可能导致服务间的干扰。部分共享模式是实际部署中的折中方案，通常共享部分资源以提升效率，同时隔离关键网元以保障 QoS。例如，中国移动在实际部署中采用“控制面独立 + 用户面集中化”的模式，为政务专网、车联网和高校 5G 校园网部署多个切片^[58-59]。在此模式下，各切片的 AMF 和 SMF 独立运行以确保控制面隔离，而 UPF 集中部署以优化用户面资源利用率。

2.4 本章总结

本章系统阐述了论文研究涉及的相关理论和技术背景，为后续的 5G 核心网智能运维研究奠定了基础。第 2.1 节介绍了云原生技术的核心原则和应用场景，指出其在 5G 核心网云原生部署中的高弹性优势。第 2.2 节分析了 Kubernetes 容器编排技术，详细探讨了其架构、Pod 与容器运行时、网络模型和调度机制，阐明 Kubernetes 在支持动态调度和热迁移中的关键作用。第 2.3 节聚焦 5G 核心网架构，介绍了控制面与用户面分离、服务化架构以及关键网元功能，并通过网络切片技术分析 5G 对低时延和高可靠性的需求。

通过上述分析，云原生技术和 Kubernetes 为 5G 核心网的云原生部署提供了高效的运行环境。Kubernetes 的 Pod 管理和调度机制为核心网运维提供了更大的灵活性，5G 核心网的 CUPS 和 SBA 架构则通过模块化设计使其以云原生环境作为基础计算架构成为可能。网络切片技术的多样化 QoS 要求进一步凸显了热迁移和智能调度的必要性。然而，现有技术仍存在一定局限性：热迁移方案在处理大内存场景时迁移时延较高，而 Kubernetes 默认调度器基于静态规则，难以适应 5G 核心网动态负载和多目标优化需求。这些不足为后续研究提供了方向。

第3章 云原生 5G 核心网网元热迁移策略

3.1 引言

随着 5G 技术的快速发展和广泛部署，云原生架构凭借其高可用性、灵活性和动态扩展能力，已成为 5G 核心网部署的重要技术支撑。云原生架构通过容器化、微服务和自动化调度等技术，为网络功能虚拟化（NFV）提供了高效的实现方式，能够显著提升核心网的资源利用率和运维效率。然而，5G 核心网作为支持高吞吐量、低时延和大规模连接的关键基础设施，不仅对服务连续性和系统可靠性提出了极高的要求，更关键的是，核心网中的许多网元承载着用户的认证、鉴权、会话管理等关键安全功能，其安全连续性（Security Continuity）的保障至关重要。尤其是在动态负载均衡、故障恢复和节点维护等场景中，如何在不中断服务、不丢失关键安全上下文的情况下实现网元的高效迁移，成为保障核心网高可用性与运行安全性的核心挑战^[60]。

传统云原生基础设施通常采用 Delete-Recreate 策略进行 Pod 迁移，即通过销毁源节点上的 Pod 并在目标节点重新创建来完成调度。然而，该策略因无法保留 Pod 的运行状态，导致服务中断时间较长，并引发会话重建的高开销。对于 5G 核心网而言，这不仅仅是服务质量下降的问题，更可能导致安全漏洞和风险。以接入与移动性管理功能（AMF）为例，其负责管理用户设备（UE）的注册、连接、移动性会话，并处理 NAS 安全上下文、用户认证信息等敏感数据。在迁移过程中，若 AMF 的状态丢失，不仅导致 UE 需要重新发起连接，引发业务中断和性能下降，更重要的是，关键安全状态的丢失可能迫使用户重新进行完整的认证流程，增加了信令开销，也为潜在的中间人攻击或会话劫持提供了短暂的窗口期。例如，在测试场景中，当 500 个 UE 同时接入时，Delete-Recreate 方案的恢复时间可达 35 秒，严重影响用户体验和网络的整体服务质量，同时也暴露了安全连续性保障的脆弱性。

针对传统迁移策略在保障安全连续性方面的局限性，本研究提出了一种基于“Clone-Restore”的热迁移方案，旨在实现 5G 核心网网元的带状态迁移，显著降低服务中断时间、优化资源利用效率，并确保关键安全上下文的平滑继承。该方案通过扩展容器运行时接口（CRI）并设计专用的热迁移控制器，实现了 Pod 运行时状态的高效捕获、传输和恢复，能够在迁移过程中完整保留网元的内存状态、文件系统数据和网络连接状态，从而维持网元功能及其承载的安全机制的连续性。为全面验证所提

方案的性能和适用性，本研究在中国电科某研究所的测试网络环境中搭建了 5G 独立组网（SA）核心网原型系统，以 AMF 网元为测试对象，开展了多场景的实验评估。实验结果表明，Clone-Restore 方案在服务恢复时间、资源开销、QoS 保障以及安全状态保持等方面均优于传统方案，为云原生环境下 5G 核心网的高可用性与安全运维提供了有效解决方案。

本章内容安排：第3.2节详细分析现有迁移技术的原理及其在 5G 核心网场景中的局限性；第3.3节系统阐述 Clone-Restore 热迁移方案的设计原理和创新点；第3.4节介绍关键技术的实现细节；第3.5节通过实验设计和性能评估验证所提方案的有效性；第3.6节总结本章的研究成果和后续工作方向。

3.2 相关工作

在云原生环境下，Pod 作为 Kubernetes 集群的最小调度单位，承载了容器化应用的运行时环境，其热迁移技术的研究对于实现高可用性至关重要。现有 Pod 重调度方法主要分为两大类：基于删除重建的迁移和基于内存迁移的热迁移。前者通过销毁和重新创建 Pod 完成调度，后者则尝试在迁移过程中保留运行时状态，以减少服务中断时间。本节从 5G 核心网高可用性的需求出发，分析了几种典型迁移方案的原理、适用场景及其局限性，为后续提出 Clone-Restore 方案奠定理论基础。

3.2.1 Pod Eviction

Pod Eviction 是 Kubernetes 提供的一种原生调度机制，通常用于节点资源优化、故障恢复或集群维护场景中的 Pod 迁移。其工作原理是通过 Kubernetes API 服务器向源节点的 Kubelet 守护进程发送驱逐请求，逐步终止目标 Pod 并释放其占用的 CPU、内存和存储等计算资源。随后，集群调度器根据 Pod 的资源需求（Resource Limit）、标签信息（Label Info）以及节点亲和性规则，在集群中选择合适的目标节点并重新创建 Pod。该过程完全依赖 Kubernetes 控制平面的自动化调度和 Pod 重启机制，具有实现简单、部署便捷的优点，广泛应用于无状态应用的动态调度。

然而，Pod Eviction 的局限性在于其本质上是一种 Delete-Recreate 策略，无法保留 Pod 的内存数据、文件句柄和网络连接等运行时状态。在迁移过程中，Pod 被销毁后需从初始状态重新启动，导致服务不可用时间较长。对于 5G 核心网的有状态网元，其运行时状态涉及复杂的协议交互和会话管理，状态丢失将引发 UE 重连或会话中断，

显著降低服务质量。因此，Pod Eviction 难以满足 5G 核心网对高可用性和低时延的严格要求。

3.2.2 rsync

rsync 是一种高效的文件同步工具，可通过网络传输 Pod 的容器文件系统数据以实现迁移。其工作流程为：在源节点暂停 Pod 运行后，利用 rsync 算法比较源节点和目标节点的文件差异，仅将变更部分同步至目标节点。同步完成后，目标节点基于相同的容器镜像和配置重新启动 Pod，恢复应用运行。rsync 的优点在于支持增量同步，能够有效减少数据传输量，且实现方式简单，适用于文件系统较大的场景。此外，rsync 无需复杂的硬件支持，易于在通用云原生集群中部署。

然而，rsync 的局限性在于其仅能同步文件系统数据，无法捕获 Pod 的内存状态、进程上下文或网络连接状态。对于 5G 核心网网元，运行时状态通常存储在内存中，rsync 无法满足带状态迁移的需求。此外，当文件系统规模较大或网络带宽受限时，rsync 的同步过程耗时较长，导致服务中断时间显著增加。因此，rsync 难以满足 5G 核心网热迁移场景对低时延和高可靠性的要求。

3.2.3 RDMA

远程直接内存访问（Remote Direct Memory Access, RDMA）是一种高性能网络技术，通过支持 RDMA 的网络接口卡（如 InfiniBand 或 RoCE）实现端到端的低时延数据传输。RDMA 允许源节点的内存数据直接传输到目标节点的内存，绕过 CPU 和操作系统内核的数据拷贝，从而实现高吞吐量和低延迟的数据迁移。在 Pod 迁移场景中，RDMA 可用于快速传输内存状态，理论上适合延迟敏感的 5G 核心网应用，能够有效缩短迁移过程中的服务中断时间。

尽管 RDMA 在传输性能上具有显著优势，但其应用受到多方面限制。首先，RDMA 需要专用硬件支持（如高性能网卡和交换机），部署成本较高，且对网络基础设施的要求严格，通用云原生集群难以满足。其次，RDMA 仅能传输内存数据，无法处理 Pod 的文件系统、容器运行时状态或网络连接，需与其他工具结合使用。此外，当迁移涉及大规模 Pod 或多节点并发迁移时，网络带宽可能成为瓶颈，进一步限制其适用性。因此，RDMA 在 5G 核心网热迁移中的实用性较低。

3.2.4 KubeVirt Live Migration

KubeVirt Live Migration 是 Kubernetes 扩展项目 KubeVirt 提供的虚拟机实时迁移功能，可作为 Pod 热迁移的参考方案。其基于 QEMU/KVM 虚拟化技术，通过内存预复制（Pre-copy）和停机复制（Stop-and-copy）两个阶段实现迁移。在预复制阶段，虚拟机的内存数据被逐步传输到目标节点，同时源节点保持运行；当内存差异足够小时，进入停机复制阶段，暂停源虚拟机，快速传输剩余状态并在目标节点恢复运行。KubeVirt 通过 Kubernetes API 封装了该过程，便于集群管理，支持虚拟机的无缝迁移。

尽管 KubeVirt Live Migration 在虚拟机场景中表现良好，但其直接应用于 Pod 热迁移存在显著局限性。首先，虚拟机与容器的运行时特性差异较大，虚拟机的资源开销和迁移耗时远高于轻量级的 Pod。其次，该方案依赖底层 hypervisor，增加了系统的复杂性和维护成本。此外，KubeVirt 的设计目标是虚拟化环境，而非云原生容器场景，其迁移流程无法直接适配 5G 核心网网元的轻量级需求。因此，该方案难以满足本研究的热迁移要求。

3.2.5 问题总结

综合以上分析，现有 Pod 迁移方案在 5G 核心网热迁移场景中均存在显著局限性。Pod Eviction 和 rsync 因无法保留运行时状态，导致服务中断时间长，难以满足有状态网元的连续性要求；RDMA 虽然在内存传输性能上表现优异，但其高成本和通用性不足限制了实际应用；KubeVirt Live Migration 则因虚拟化开销和复杂性不适合容器场景。5G 核心网网元的热迁移需同时捕获持久化数据和运行时数据，并确保迁移过程中的低时延和高可靠性。现有方案难以全面满足这些需求，为此，本研究提出了一种基于 Clone-Restore 的热迁移方案，通过高效的状态捕获和恢复机制，解决传统方案的不足，为云原生环境下 5G 核心网的高可用性提供创新性解决方案。

3.3 方法原理与创新分析

3.3.1 技术方案探索

在云原生环境下，Kubernetes 作为 5G 核心网部署的核心基础设施，其高可用性直接影响网络服务的稳定性和用户体验。热迁移技术通过支持 Pod 的跨节点动态调度，能够在节点故障、负载均衡或维护场景中实现网元迁移，从而保障核心网的连续性和可靠性。然而，5G 核心网网元具有复杂的有状态特性，涉及 UE 上下文、协议状

态和会话管理的实时交互，传统迁移策略难以满足其对低时延、高可靠性和状态一致性的严格要求。为此，本研究设计了一种基于“Clone-Restore”的热迁移方案，旨在通过扩展 Kubernetes API 体系，实现 Pod 的带状态迁移，满足 5G 核心网高可用性的需求。

为确保热迁移技术在云原生 5G 核心网场景中的有效性，设计方案需满足以下目标：

(1) **API 一致性：**Kubernetes API 是集群管理和用户交互的核心接口，热迁移功能的 API 设计需遵循 Kubernetes 的设计准则，与现有 API 体系保持风格一致，避免引入过多复杂性，确保长期维护和优化的可行性。同时，API 需提供必要的字段，支持 5G 核心网网元的特殊需求。

(2) **最短停机时间：**5G 核心网对服务质量要求极高，特别是在延迟、吞吐量和网络切片性能方面。热迁移过程需尽量减少服务中断时间，理想情况下实现无感知迁移或极短停机时间，确保有状态网元在跨节点迁移时不丢失会话状态，维持服务连续性。

(3) **底层兼容性：**热迁移功能需与 Kubernetes 的底层组件无缝集成，避免潜在冲突。迁移过程应支持可编程、可调度和可管理的特性，与云原生生态充分融合，适应核心网的动态运维需求。

(4) **易用性：**为便于手动操作或自动调度，热迁移方案需提供灵活的调用接口，支持用户或自定义调度器通过简单配置触发迁移，同时确保操作的透明性和可控性，提升核心网运维效率。

在 Kubernetes 中，所有资源均通过 API 定义，集群的调度、管理和运维操作均依赖 API 服务器的控制。因此，Pod 热迁移的全流程需通过扩展 Kubernetes API 实现，API 设计成为本研究的核心环节。以下分析了三种理论上可行的 API 设计方案，从实现复杂度、系统集成性、兼容性和 5G 核心网需求的契合度等方面进行评估，提出了本研究的创新性解决方案。

3.3.1.1 基于扩展 Pod Spec 的方案

从实现复杂度的角度出发，扩展 Pod Spec 字段是最直接的热迁移 API 设计方案。具体而言，通过在 Pod 的 Spec 字段中添加迁移相关信息，即可指示 Kubelet 执行迁移操作。如图 3.1 所示，迁移流程如下：用户创建包含源 Pod 引用的目标 Pod 并调度至

目标节点；目标节点的 Kubelet 根据引用信息克隆源 Pod 的容器环境，复制其文件系统和运行时状态；迁移完成后，源 Pod 被删除，目标 Pod 接管服务。

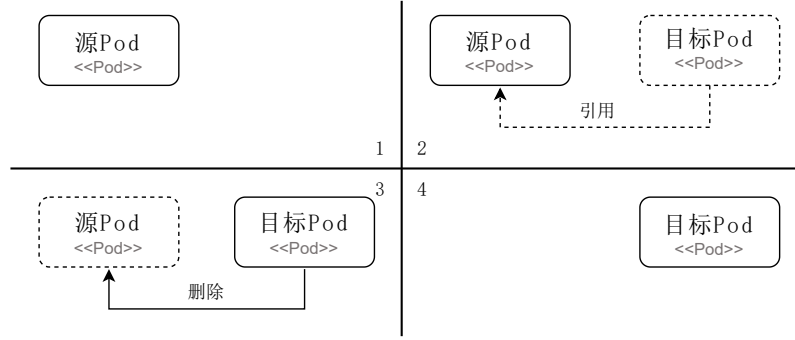


图 3.1 基于扩展 Pod Spec 的热迁移流程

该方案的优点在于实现简单且高度灵活。Kubelet 可直接从 Pod Spec 中解析迁移指令，无需额外的资源请求或状态监控，且迁移字段作为 Pod 规范的可选属性，完全兼容现有 Kubernetes API 体系。此外，用户可通过标准 Pod 创建接口触发迁移，无需修改集群控制平面，降低了开发成本。

然而，该方案存在若干局限性。首先，目标 Pod 创建会导致源 Pod 立即终止，与 Kubernetes API 对象间互不影响的设计原则相冲突，可能引发状态不一致问题。其次，该方案无法有效限制可迁移 Pod 的范围，用户可随意指定任意源 Pod，增加了误迁移的风险。例如，若源 Pod 与目标 Pod 的配置不一致，可能导致状态恢复失败。此外，为支持迁移参数配置，需在 Pod Spec 中引入额外字段，对 Pod 的原生运行机制具有一定侵入性。最后，该方案依赖用户手动创建目标 Pod，缺乏专门的控制器协调迁移流程，与调度器、节点管理等组件的集成较为复杂。在 5G 核心网场景中，AMF 等网元需高频、自动化的迁移支持，该方案的灵活性虽高，但自动化程度不足，难以满足实际需求。

3.3.1.2 基于 ClonePod 迁移任务的方案

为克服扩展 Pod Spec 方案的局限性，本研究进一步提出了一种基于迁移任务资源的方案，定义了一种新的自定义资源，称为 ClonePod。ClonePod 作为迁移任务的声明对象，负责协调 Pod 的克隆和状态转移过程。如图 3.2 所示，迁移流程如下：用户创建 ClonePod 对象并指定源 Pod 的标识；ClonePod 控制器接管迁移任务，自动创建目标 Pod 并复制源 Pod 的状态；迁移完成后，控制器删除源 Pod，释放目标 Pod 的控制权，随后 ClonePod 对象被垃圾回收。

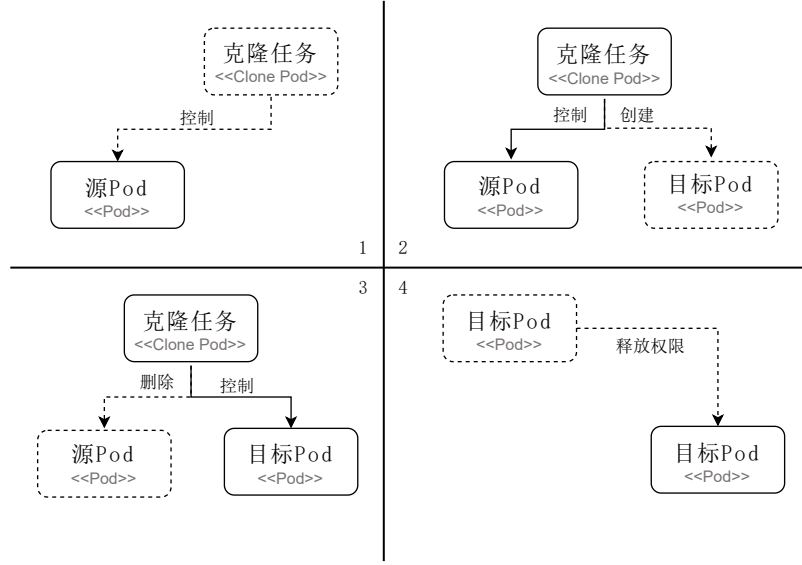


图 3.2 基于 ClonePod 的热迁移流程

与扩展 Pod Spec 方案相比，ClonePod 方案具有显著优势。首先，通过独立的迁移任务资源，避免了 Pod 间的直接依赖，降低了因配置不一致导致的迁移失败风险。其次，ClonePod 控制器负责状态跟踪和错误处理，支持复杂的容错机制，提高了迁移的可靠性。此外，用户只需创建 ClonePod 对象并配置迁移参数，无需手动构造目标 Pod，操作更加简便。ClonePod 资源还支持统一的迁移参数管理，便于在单一对象中定义源 Pod 和目标 Pod 的相关配置，增强了方案的优雅性和可维护性。

然而，ClonePod 方案仍存在一定局限性。其迁移触发依赖用户手动创建 ClonePod 对象，难以直接响应 Kubernetes 的原生事件。若需适配现有机制，则需修改调度器或节点管理组件以生成 ClonePod 对象，增加了实现成本。此外，ClonePod 作为一次性任务资源，迁移完成后即被销毁，难以支持持续的 Pod 管理需求。在 5G 核心网场景中，网元迁移需与集群的动态调度紧密集成，ClonePod 的自动化程度仍需进一步提升。

3.3.1.3 基于 PodMig API 的方案

为进一步提升热迁移方案的系统集成性和自动化程度，本研究提出了基于 PodMig API 的创新设计。PodMig 是一种自定义控制器资源，借鉴 Kubernetes 工作负载控制器的设计理念，实现了对 Pod 迁移的持续管理和自动化控制。如图 3.3 所示，PodMig 的迁移流程如下：在 Pod 创建之初或运行期间，PodMig 资源即与目标 Pod 建立所有权关系；当触发迁移事件时，PodMig 控制器自动创建目标 Pod，复制源 Pod 的状态，并在迁移完成后删除源 Pod；迁移后，PodMig 继续持有目标 Pod 的所有权，直至 Pod

或 PodMig 被显式删除。

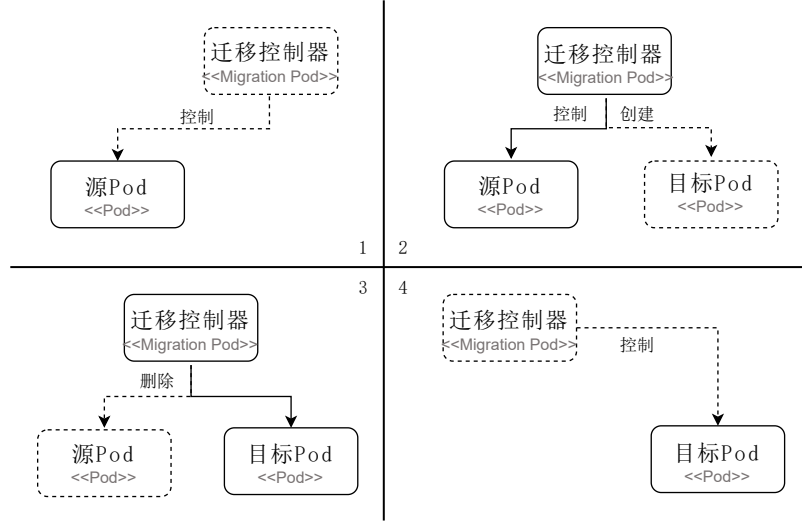


图 3.3 基于 PodMig 的热迁移流程

与 ClonePod 方案相比，PodMig 具有以下创新优势。首先，PodMig 采用持续所有权机制，严格控制 Pod 的迁移行为，避免了误迁移的发生。例如，只有 PodMig 持有的 Pod 才能被迁移，未授权的 Pod 无法被其他用户或组件随意操作。其次，PodMig 支持自动响应 Kubernetes 原生事件，无需额外适配即可与调度器、Kubelet 等组件集成，显著提升了自动化程度。此外，PodMig 的行为模式与 Kubernetes 控制器体系高度一致，支持声明式 API 和状态协调机制，确保迁移过程的可控性和可观测性。在 5G 核心网场景中，AMF 等网元需在高动态环境下实现快速迁移，PodMig 的自动化和集成性能够有效满足这一需求。

然而，PodMig 方案也存在一定局限性。首先，其灵活性较低，Pod 需在创建时即绑定 PodMig 资源，或通过后期手动变更建立所有权关系，增加了初始配置的复杂性。其次，PodMig 控制器的开发涉及对节点管理进程和容器运行时服务的修改，开发成本较高。此外，持续所有权机制可能限制某些场景下 Pod 的独立操作。尽管如此，在 5G 核心网的典型用例中，网元迁移需求较为明确，自动化和可靠性优先于灵活性，PodMig 方案的整体优势更为突出。

3.3.1.4 方案对比与小结

为全面评估上述三种方案的适用性，本研究从 API 一致性、实现复杂度、系统集成性、灵活性和 5G 核心网需求的契合度等方面进行了对比分析，结果如表 3.1 所示。

表 3.1 热迁移 API 设计方案对比

指标	Pod Spec	ClonePod	PodMig
API 一致性	中等	中等	高
实现复杂度	低	中等	高
系统集成性	低	中等	高
灵活性	高	中等	低
停机时间	中等	低	极低
错误防控	低	中等	高
5G 核心网适用性	低	中等	高

扩展 Pod Spec 方案实现复杂度最低，用户可通过简单配置触发任意 Pod 的迁移，灵活性较高。然而，其缺乏专门的控制器协调迁移流程，容易引发误迁移或状态不一致问题，与调度器等组件的集成成本较高，难以满足 5G 核心网的自动化需求。ClonePod 方案通过引入迁移任务资源和控制器，显著提高了操作便利性和错误防控能力，用户只需声明 ClonePod 对象即可完成迁移，迁移过程中的状态一致性得到保障。然而，其一次性任务模型限制了与 Kubernetes 原生事件的集成，自动化程度有限。PodMig 方案在 API 一致性、系统集成性和错误防控方面表现最佳，其控制器模式与 Kubernetes 工作负载管理逻辑高度契合，支持自动响应节点故障和维护事件，迁移过程中的停机时间极低。此外，PodMig 的所有权机制有效避免了误迁移，确保了迁移行为的可控性和可靠性。尽管其灵活性较低且开发成本较高，但在 5G 核心网的高可用性场景中，网元迁移的自动化和低时延需求优先于灵活性，PodMig 方案能够充分发挥云原生基础设施的优势。

综上所述，PodMig 方案在满足 5G 核心网热迁移需求方面具有显著优势，其创新性的控制器设计和自动化机制为网元的高效、可靠迁移提供了有力支撑。因此，本研究选择 PodMig 作为 Clone-Restore 热迁移方案的核心 API 框架，后续将围绕 PodMig 展开关键技术的实现与实验验证。

3.3.2 热迁移方案设计

为实现 Pod 层级的垂直弹性以满足网元及网络切片更具鲁棒性的高可用需求，本文提出了一种适用于核心网网元的热迁移方案。方案核心思路是将整个热迁移过程分

为克隆和恢复两个阶段，在克隆阶段通过多种手段提取目标 Pod 的快照，并在恢复阶段通过增量更新的方式减少快照与运行服务的差异性，最终达到最小停机时间。方案核心流程设计如图3.4所示。与其他方案相比，本方案状态完整性、迁移速度和与 Kubernetes 的兼容性上具有显著优势。具体实现及性能验证见 3.4 小节。

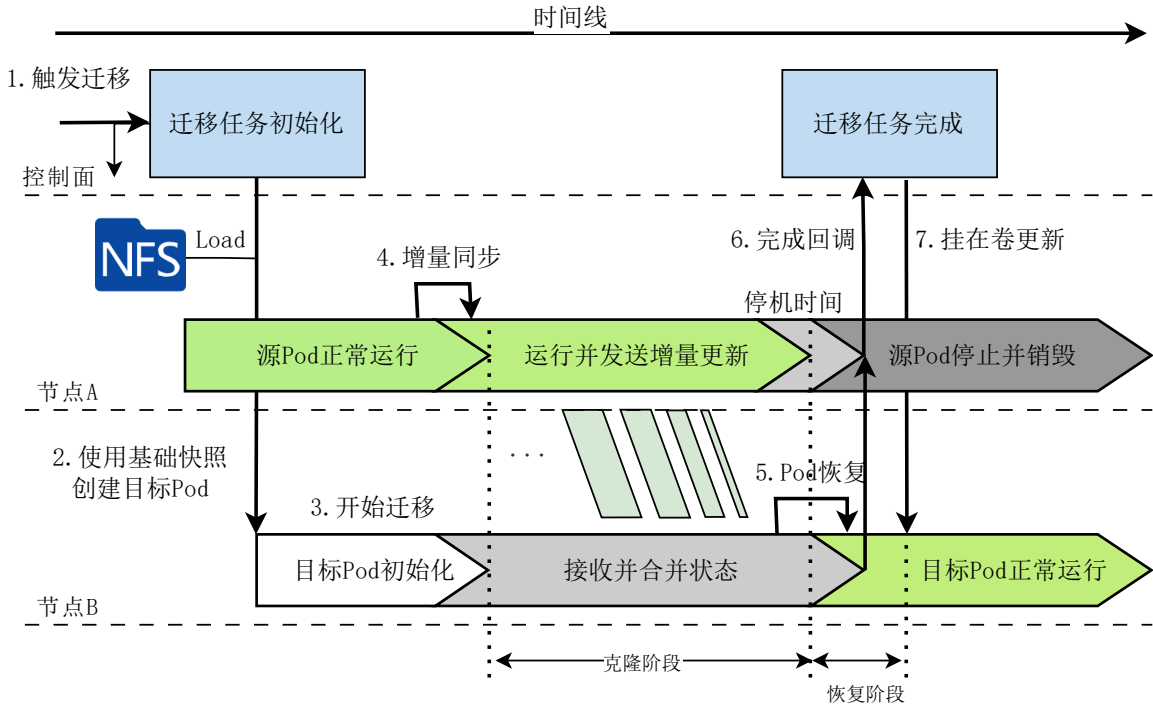


图 3.4 Pod 热迁移核心流程

3.3.2.1 Pod 状态克隆

Pod 状态克隆是热迁移方案的核心环节，通过捕获和转移 Pod 的运行时状态以实现 5G 核心网网元的跨节点无缝迁移。如第 2.2 节所述，Pod 作为 Kubernetes 的最小调度单位，其状态由多个部分组成：一是由一个或多个容器提供的计算服务运行时，包括内存数据和进程上下文；二是标签、注解等 Pod 元数据，用于描述 Pod 的属性和调度策略；三是网络接口配置、存储卷状态和环境变量等，提供 Pod 与外部环境的交互接口。在 5G 核心网场景中，网元承载复杂的会话管理和协议状态，这些状态的完整迁移直接影响服务的连续性和 QoS 保障。因此，Pod 状态的保存和恢复是热迁移技术的关键挑战，本节描述方案设计，第3.4节将进一步介绍关键技术的具体实现。

为实现高效的状态克隆，本研究在“Pre-Copy”的基础上提出了一种基于“初始全量备份 + 动态增量快照”的 Pod 状态保存策略。需要指出的是，频繁的全量备份不仅会显著增加存储资源开销，其快照文件的传输还可能占用集群内网带宽，导致服务

时延增大，特别是在高负载的 5G 核心网环境中。因此，设计合理的快照策略至关重要，需在为主备切换提供支持的同时，尽量减少对系统资源的占用。本方案通过全量备份与增量快照的结合，优化了状态克隆的实时性和迁移效率。

如图 3.4 所示，Pod 状态克隆的流程分为两个子阶段。在初始部署阶段，源节点生成一次全量备份，捕获 Pod 的完整运行时状态，包括容器内存、文件系统、配置数据和网络连接信息，生成快照文件并存储至分布式存储系统。与传统的中心化存储相比，分布式存储通过数据分片和负载均衡，能够显著提升迁移过程中的并发性和可靠性。全量备份仅在 Pod 状态发生破坏性变更时按需执行，避免了周期性备份带来的计算和存储开销。

在全量备份的基础上，本方案引入动态增量快照机制^[61]，以捕获 Pod 的实时状态变化。源节点通过 CRIU^[31]技术周期性地生成增量快照，快照记录自上次快照以来的状态变更，主要是内存页的修改。这些增量快照数据量较小，并点对点传输直接预拷贝^[62]至目标节点。为优化快照频率和数据量，本方案设计了自适应快照策略^[63]，通过 Kubernetes 自定义控制器（Custom Operator）实时监听 Pod 的运行状态，量化内存状态变化和系统负载实现高效状态克隆，根据服务内存脏页率和系统 CPU 利用率调整增量快照的终止条件^[64]。公式 (3.1) 定义了内存脏页率（Dirty Page Rate）：

$$R_d = \frac{N_d}{T_i} \quad (3.1)$$

其中， R_d 表示脏页率， N_d 为某时间间隔内累积的脏页数量， T_i 为测量时间间隔。通过解析 Linux 的 `/proc/pid/smaps` 接口， R_d 反映了内存页的修改速率，为快照频率调节提供依据。

根据论文^[30]的研究，构建快照间隔计算函数：

$$T_{\text{next}} = \max \left(T_{\min}, \min \left(T_{\max}, \alpha \cdot \frac{1}{R_d} + \beta \cdot \frac{1}{U_c} \right) \right) \quad (3.2)$$

其中， T_{next} 表示下一次快照的间隔时间， $T_{\min}$ 和 $T_{\max}$ 分别为快照周期的下限和上限； α, β 为经验参数， $\alpha = 0.6$ ， $\beta = 0.4$ ，反映内存脏页率和 CPU 利用率的相对权重； R_d 通过 Linux 的 `/proc/pid/smaps` 接口估算，反映内存状态的变更速率； U_c 由 API Server 实时采集，反映节点的计算负载。该函数确保在高负载、高变更率场景中动态提高快照频率（即减小 T_{next} ），以降低每次快照的增量数据量，保障状态一致性。例如，对于 AMF 网元在高并发 UE 注册场景下的 CPU 密集型负载，较高的 R_d 和 U_c 会

触发更频繁的快照。在系统稳定状态下，则降低快照频率（即增大 T_{next} ），减少无效的计算和存储开销。

为估算每次快照的数据规模，本方案引入增量快照数据量公式：

$$S_{\text{inc}}(t) \approx R_d(t) \times \Delta t \times P_{\text{size}} \quad (3.3)$$

其中， $S_{\text{inc}}(t)$ 表示时间 t 时的增量快照数据量， $R_d(t)$ 为当前脏页率， Δt 为快照间隔（秒，通常由公式 (3.2) 确定）， P_{size} 为页面大小，Linux 系统中通常是 4KB。

为进一步优化快照触发时机，本方案设计了快照触发阈值条件：

$$\text{Trigger} \iff (t - t_s \geq T_m) \vee (D_a \geq D_{\text{th}}) \quad (3.4)$$

其中， t 为当前时间， t_s 为上次快照时间， T_m 为最大快照间隔， D_a 为累积脏页量， D_{th} 为脏页量阈值。该条件确保快照触发兼顾时间间隔和状态变化，当时间超限或脏页量过大时启动快照生成，提升状态一致性。

为决定何时结束增量快照并进入最终同步阶段，本方案设计了终止条件：

$$\tau(D_t, D_{t-1}, N) = \begin{cases} 1, & \text{if } |D_t - D_{t-1}| < \delta \vee N \geq N_m \\ 0, & \text{otherwise} \end{cases} \quad (3.5)$$

其中， τ 表示终止函数，输出 1 时终止增量快照； D_t 和 D_{t-1} 分别为当前和前一次增量快照的脏页量； δ 为收敛阈值； N 为已生成快照次数； N_m 为最大快照次数。该条件表示当脏页量变化趋于稳定或快照次数达到上限时，终止增量快照。实验表明，在 400 UE 负载下，脏页量通常在 3-5 次快照后收敛。

当满足终止条件时，自定义控制器引导迁移进入最终同步阶段。在此阶段，控制器挂起源节点容器，完成残余状态的同步，并在目标节点恢复 Pod 运行。CRIU 技术确保了状态传输的一致性，将停机时间压缩至秒级，满足 5G 核心网低时延的严格要求。

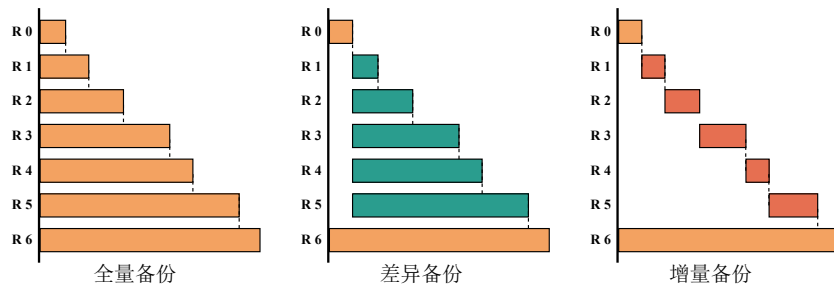


图 3.5 全量备份、差异备份与增量备份的逻辑对比

为进一步分析快照策略的性能，本方案对比了全量备份、差异备份和增量备份的特性。如图 3.5 所示，增量备份通过细粒度的快照序列，能够以更高的生成频率和更小的状态距离适应 Pod 的动态变化。相比差异备份需恢复单一累计快照，增量备份在每次快照的生成开销和恢复复杂度上更优。

综上所述，本方案通过“初始全量备份 + 动态增量快照”的策略，实现了 Pod 状态的克隆和可靠转移方案设计。快照间隔公式和双重终止判据进一步量化了动态调整逻辑，确保了状态一致性和资源优化。

本节所有公式的符号定义见表 3.2。

表 3.2 快照策略相关符号表

符号	含义
R_d, N_d, D_a	脏页率; 脏页数量; 累积脏页量
D_t, D_{t-1}, δ	当前快照脏页量; 前一次快照脏页量; 收敛阈值
D_{th}	脏页量阈值
$T_i, T_{next}, \Delta t$	测量时间间隔; 下一次快照间隔; 快照间隔
t, t_s, T_m	当前时间; 上次快照时间; 最大快照间隔
$S_{inc}(t), P_{size}, N$	增量快照数据量; 页面大小; 已生成快照次数
N_m, τ	最大快照次数; 终止函数
U_c, α, β	CPU 利用率; 脏页率权重; CPU 利用率权重

3.3.2.2 状态恢复阶段

状态恢复阶段利用克隆阶段生成的 Pod 状态快照，在目标节点精确重建运行时状态，确保网元服务的连续性和功能一致性。如第 3.3.2.1 节所述，克隆阶段通过“初始全量备份 + 动态增量快照”策略，将 Pod 的运行时状态基础快照存储于存储 NFS 中。本小节聚焦于如何通过快照整合、最终同步机制、网络与存储状态恢复降低停机时间。

恢复流程首先从 NFS 获取克隆阶段生成的全量备份快照。全量快照记录了 Pod 的完整初始状态，包括容器内存数据、文件系统内容、网络配置和 Pod 元数据。随后，目标节点加载动态增量快照，这些快照捕获了自全量备份以来 Pod 状态的实时变更。目标节点首先通过 CRIU 将全量快照加载为 Pod 的初始运行环境，随后按时间顺序逐一应用增量快照，动态重建 Pod 的最新状态。为优化恢复效率，本设计借鉴了虚拟机热迁移的预拷贝思想，在克隆阶段即通过点对点传输预同步增量快照，减少恢复阶段

的数据加载开销。

尽管动态增量快照显著降低了状态距离，目标 Pod 从重建到服务就绪的短暂时间窗内仍可能丢失最新的状态变更。为解决这一问题，本方案引入了最终同步 (last-sync) 作为恢复流程的核心创新。如图 3.6 所示，当克隆阶段满足终止条件（即脏页量收敛 $|D_t - D_{t-1}| < \delta$ 或快照次数达到 $N_{\text{snapshot}} \geq N_{\text{max}}$ ），控制器挂起源节点容器，触发最终同步。最终同步通过集群网络直接在源节点和目标节点间传输残余状态数据，绕过分布式存储系统以减少时延。

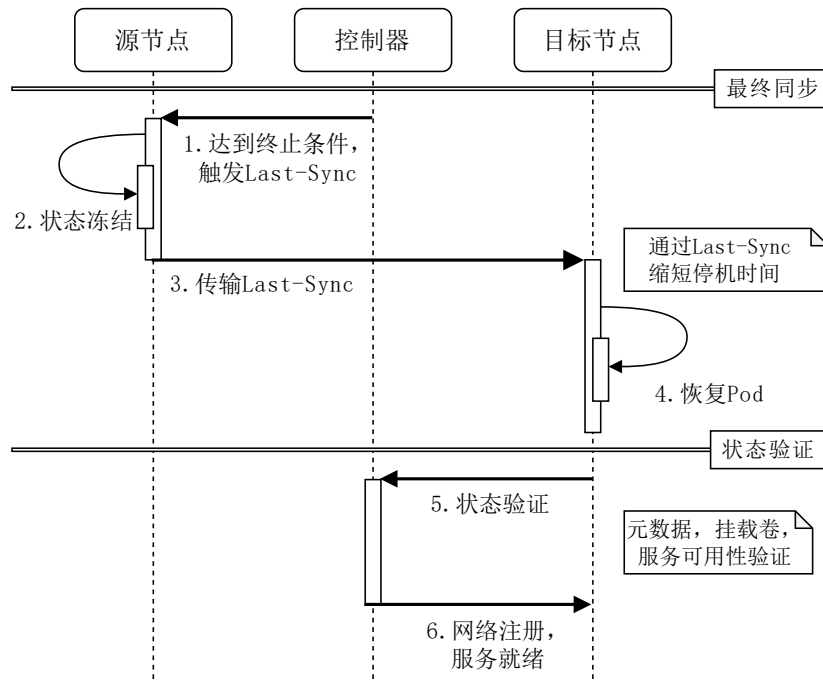


图 3.6 Pod 恢复时序图

网络恢复依赖 Kubernetes 的容器网络接口插件，目标节点动态分配与源节点一致的网络资源，包括 IP 地址、端口映射和负载均衡策略，确保 TCP 会话和 UDP 连接的连续性。为支持 5G 核心网的网络切片需求，本方案利用 Kubernetes 的自定义注解保存切片标识、QoS 参数和安全策略，确保迁移后业务逻辑的完整性和服务质量的稳定性。存储恢复则通过 Kubernetes 的持久卷（Persistent Volume, PV）和持久卷声明（Persistent Volume Claim, PVC）机制实现，目标节点挂载与源节点相同的存储卷，结合分布式存储系统的快照功能，同步克隆阶段的状态数据。Ceph 的快照机制确保了存储卷内容与 Pod 运行时状态的一致性，避免了数据丢失或不一致的风险 citeweil2006ceph。为进一步提升迁移可靠性，本方案利用探针机制验证 Pod 状态，在恢复完成后，控制器

通过执行服务就绪检查探针（readiness probe）确认目标 Pod 能够正常响应业务请求。

综上所述，本方案通过优化快照整合、最终同步机制、网络与存储状态恢复确保 Pod 状态的高效重建和可靠迁移。最终同步阶段通过最小化状态距离；网络与存储恢复机制确保了服务连续性和业务逻辑的完整性；状态验证机制进一步提升了迁移的鲁棒性。后续将详细介绍关键技术的实现细节，并通过实验进行性能评估。

3.4 技术方案实现

为了最大限度减少热迁移改进对云原生基础设施的侵入，本文基于定制资源定义（Custom Resource Definition，CRD）实现热迁移功能与核心网基础设施的集成。通过 CRD 进行集成，我们得以在云原生集群中创建系统级别的资源定义，从而获得与系统内建资源相同甚至更高的调度优先级。云原生集群中每个工作节点都通过控制进程 Kubelet 与集群中枢和其他工作节点进行通信，节点内容器调度则通过容器运行时接口进行，热迁移功能的实现也需要对这两个组件做出改进，本节首先介绍热迁移整体技术架构设计，然后对其中关键技术实现做出详细描述。

3.4.1 热迁移架构设计

图 3.7 展示了本文提出的 Pod 热迁移系统的整体技术架构。该架构以控制器为核心，结合分布式存储、节点代理、容器运行时与外部网络组件，实现了跨节点的最小中断热迁移流程。

本架构主要由控制层、执行层与存储与网络支撑层三部分组成，各部分分工协作，构成了一个具备自适应调度能力、状态持续性保障与低中断恢复特性的 Pod 级别热迁移体系。

（1）控制层

控制层主要由控制器组件及其上运行的迁移调度逻辑构成，承担全局协调、资源调度与状态感知等核心职能。控制器监听集群资源状态，接收外部触发指令，并结合当前集群拓扑、服务负载、QoS 目标等信息，决策是否进行 Pod 迁移，以及选定目标节点。控制层还负责更新 Pod 的调度策略和资源绑定关系，驱动后续的状态克隆、恢复与验证流程，是整个迁移流程的中枢控制单元。

（2）执行层

执行层以 Kubernetes 在各工作节点上的代理程序（Kubelet）构成，负责执行与热

3.4.2 状态提取

状态提取环节从源节点高效捕获 Pod 的完整运行时状态，为后续快照生成与状态恢复提供基础支撑。如第 3.3.2 节所述，状态克隆阶段依赖全量备份与动态增量快照，而状态提取阶段则负责原始数据的高效生成。Pod 的运行状态涵盖容器状态与非容器状态，需要针对不同类型的状态设计差异化的捕获机制。在 5G 核心网场景下，关键网元对状态一致性和迁移延迟具有极高要求。为此，本文提出一种基于 CRIU 与 Kubernetes API 的状态提取优化方案，提升状态提取的效率与鲁棒性。

3.4.2.1 容器状态提取

从操作系统视角看，Kubernetes Pod 内的容器本质上是一组通过命名空间（Namespace）和控制组（Cgroup）隔离的进程，共享同一个 Linux 内核。每个容器的运行时状态包括内存数据、CPU 寄存器、文件描述符、线程上下文等，状态提取的核心在于对这些进程状态的完整捕获。本方案采用 CRIU（Checkpoint/Restore In Userspace）作为容器状态提取的核心工具，其检查点机制能够高效捕获进程状态并序列化为可传输的镜像文件，为后续快照生成和恢复提供支持。

容器状态提取流程分为三个阶段：进程冻结、状态捕获和数据序列化。首先，自定义控制器根据 Pod 的根进程标识遍历其进程树，构建完整的拓扑结构，包括主进程、子进程和线程组。随后，CRIU 利用 Linux 内核的 ptrace 接口冻结所有相关进程，确保状态提取期间数据一致性，避免因并发修改导致状态不完整。冻结完成后，CRIU 通过 /proc 文件系统与 ptrace 接口获取进程运行状态信息。其中，内存状态提取是关键环节，CRIU 通过解析 /proc/PID/smmaps 分析内存布局，识别匿名内存、脏页及共享内存区域，并采用增量导出策略减少数据量。CPU 寄存器与线程状态则通过 ptrace 读取，记录如 RIP、RSP 等寄存器值及线程组关系，确保恢复时能重建原始线程上下文。文件描述符通过 /proc/PID/fd 提取，保留文件句柄及其属性。此外，Pod 可能包含初始化容器，这些容器在 Pod 启动时按序执行，仅用于初始化配置，正式运行后无需迁移。本方案通过解析 Pod Spec 字段，自动剔除初始化容器，优化迁移数据的选择性，降低传输开销。

3.4.2.2 非容器状态提取

Pod 的状态不仅限于容器，还包括元数据、网络配置、存储卷和运行时环境等非容器状态。如第 2.2 节所述，这些状态由 Kubernetes 的控制平面和容器运行时共同管

理，需要针对其特性设计差异化的提取策略，以确保迁移效率和状态一致性。本小节按状态类型分类，阐述元数据、沙箱、存储和网络的提取方法，并结合 5G 核心网需求进行优化。

元数据与运行时配置：Pod 的标签、注解和用户定义的运行时配置存储于 Kubernetes API Server 的 etcd 数据库中，具有中心化特性。本方案通过 Kubernetes API 直接查询这些数据，序列化为 JSON 格式，作为快照的一部分。提取过程无需访问源节点，降低了分布式系统的通信开销。为确保一致性，控制器在提取前锁定 API Server 的相关记录，防止迁移期间元数据的并发修改。

Pod 沙箱：Pod 沙箱由 CRI 提供共享命名空间支持，由容器运行时的 CRI 实现。以 Containerd 为例，沙箱表现为一个轻量级容器，承载共享的 Linux 命名空间。由于沙箱仅为容器运行提供锚点，其状态不影响业务逻辑，本方案选择不迁移沙箱状态，而在目标节点通过 CRI 重新创建，从而简化流程并提高效率。

持久存储状态：Kubernetes 通过持久卷和持久卷声明管理 Pod 的存储数据，支持业务逻辑与数据的分离。本方案遵循数据与业务分离的最佳实践，存储数据本身无需迁移，仅需提取 PV 和 PVC 的配置信息，通过 API Server 序列化为快照的一部分。分布式存储的多副本机制进一步保障了数据可靠性，即使源节点故障，目标节点仍可通过相同的 PV 访问数据。

网络状态：网络状态的提取是 5G 核心网热迁移的关键挑战，迁移需确保连接状态信息不丢失。本方案通过 CRIU 的 TCP_REPAIR 模式捕获网络连接状态，包括 TCP 会话的序列号、窗口大小和套接字选项。在提取过程中，源节点通过 iptables 规则临时锁定流量，防止数据包丢失；目标节点恢复后，控制器通过 CNI 插件 Flannel 重新分配 Pod IP 并绑定网络策略，确保 IP 地址、端口映射和负载均衡的连续性。为支持 5G 网络切片，本方案利用 Pod 的注解保存切片标识和 QoS 参数，序列化为快照，确保迁移后业务逻辑的完整性。

为提升状态提取的鲁棒性，本方案引入了错误重试和回滚机制。若提取过程中发生节点失联或网络中断，控制器将尝试最多三次重新加载或恢复数据。若仍失败，则通过 Kubernetes 的生命周期机制自动回滚至源节点运行状态。此外，针对密钥等中心化存储的集群组件，本方案直接在目标节点通过 API Server 附加，无需迁移，简化了流程。

综上所述，本方案通过 CRIU 和 Kubernetes API 实现了容器和非容器状态的高效

提取，满足了 5G 核心网网元对安全会话状态的一致性和可靠性状态捕获需求。这些优化设计为 Clone-Restore 方案的状态克隆和恢复提供了坚实基础，相关性能评估详见第 3.5 节。

3.4.3 容器运行时接口拓展

容器运行时接口作为 Kubernetes 架构中连接 kubelet 组件与容器运行时的核心协议，通过 Protocol Buffers (protobuf) 接口定义和 gRPC 通信机制，实现了容器生命周期管理与镜像操作的标准化。在 5G 核心网热迁移场景中，检查点与恢复 (Checkpoint/Restore, C/R) 功能对于实现 Pod 的无中断迁移不可或缺。然而 CRI 现有接口无法直接支持 C/R 操作，限制了热迁移的实现。为解决这一问题，本研究通过扩展 CRI，新增支持 C/R 的方法，从而为状态克隆和恢复提供运行时支持。本小节从协议层规范扩展、运行时系统实现以及控制平面适配三个层面展开论述，详细阐述实现过程。

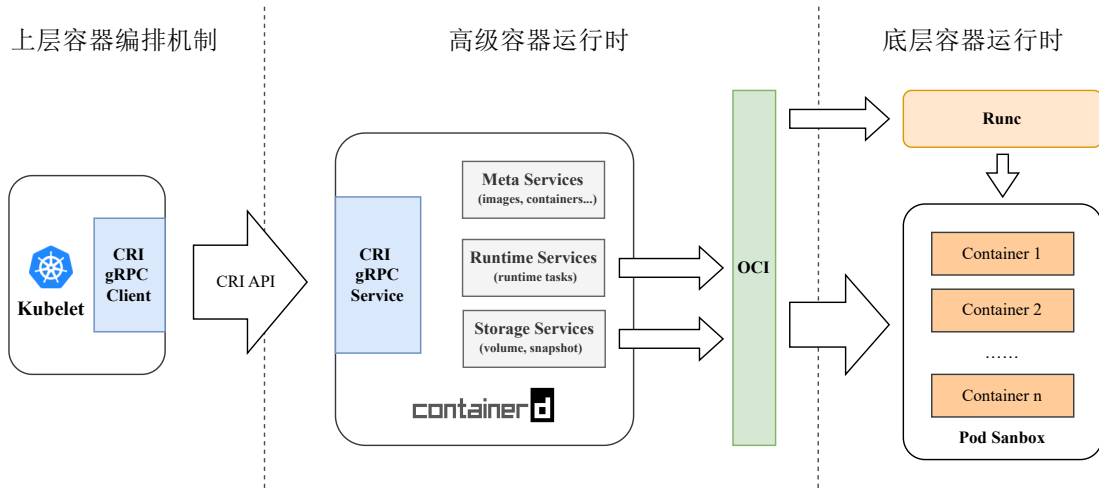


图 3.8 CRI 扩展交互流程

3.4.3.1 Protobuf 协议规范扩展

作为 CRI 接口的正式描述语言，protobuf 通过服务和消息定义实现 gRPC 远程调用规范。本研究在 CRI 现有 RuntimeService 服务基础上进行协议扩展，新增 CheckpointContainer 与 RestoreContainer 两个 RPC 方法，分别实现容器状态捕获与恢复功能。代码 3.4.3.1 展示了扩展后的消息结构定义。

```

1 // gRPC RuntimeService definition
2 service RuntimeService {
3     // [...]
4     rpc CheckpointContainer (CheckpointContainerRequest)

```

```

5     returns (CheckpointContainerResponse) {}
6   rpc RestoreContainer (RestoreContainerRequest)
7     returns (RestoreContainerResponse) {}
8   // [...]
9 }
10 message CheckpointContainerRequest {
11   string container_id = 1;
12   CheckpointContainerOptions options = 2;
13 }
14 message CheckpointContainerResponse {}
15 message CheckpointContainerOptions {
16   string checkpoint_path = 1;
17   bool leave_running = 2;
18 }
19 message RestoreContainerRequest {
20   string container_id = 1;
21   RestoreContainerOptions options = 2;
22 }
23 message RestoreContainerResponse {}
24 message RestoreContainerOptions {
25   string checkpoint_path = 1;
26 }

```

CheckpointContainerRequest 消息定义了检查点请求的参数, 包含容器标识符 (container_id)、检查点存储路径 (checkpoint_path) 以及运行时保留标志 (leave_running) 三个关键字段。其中 leave-running 参数支持动态增量快照模式, 当设置为 true 时可维持容器运行状态, 为 5G 用户面功能等实时业务提供连续服务能力。对应的 CheckpointContainerResponse 消息返回操作状态码与错误信息, 保持与现有接口的响应模式一致。类似地, RestoreContainer 的请求消息 RestoreContainerRequest 指定容器标识符和检查点路径, 响应消息 RestoreContainerResponse 确认恢复结果。

3.4.3.2 Containerd 运行时功能增强

在更新 protobuf 规范后, 需在容器运行时端实现对应的 C/R 逻辑。本研究以 containerd 为目标运行时, 其 CRI 插件内置 gRPC 服务器框架, 支持 RuntimeService 的接口扩展。为此, RuntimeService 的 Go 接口新增 CheckpointContainer 和 RestoreContainer 方法, containerd 插件通过实现这些方法提供具体功能。在协议规范扩展基础上, 本研究对 containerd 1.6 版本的 CRI 插件进行功能增强。CheckpointContainer 方法调用

containerd 的内部检查点函数，传入容器标识符和检查点路径，生成包含内存、寄存器和文件描述符的检查点文件。该实现直接复用 containerd 的 CRIU 集成，仅需调整参数传递逻辑，降低了开发成本。

RestoreContainer 方法的实现更为复杂，因容器恢复需重建运行时环境。本研究通过重构 containerd 的 StartContainer 方法，提取其核心逻辑至一个通用函数 StartOrRestoreContainer。该函数接受容器配置和可选的检查点路径，统一处理启动和恢复操作。RestoreContainer 方法调用该函数，传递检查点路径，通过 CRIU 加载快照，重建容器状态。这种复用设计不仅减少了代码冗余，还提高了实现的简洁性和可维护性，确保了 5G 核心网网元在恢复时的低时延要求。

3.4.3.3 Kubelet 控制平面适配

Kubelet 通过模块化接口与 CRI 交互，其中 RemoteRuntimeService 封装了 gRPC 客户端，附加了日志记录功能。为支持 C/R 操作，本研究在 RemoteRuntimeService 中新增 CheckpointContainer 和 RestoreContainer 方法的调用逻辑。新方法的实现与现有方法一致，通过 gRPC 客户端向 containerd 发送请求并处理响应，确保调用方式的统一性。

```

1 func (r *RemoteRuntimeService) CheckpointContainer(
2   ctx context.Context,
3   req runtimeapi.CheckpointContainerRequest
4 ) (runtimeapi.CheckpointContainerResponse, error) {
5   resp, err := r.runtimeClient.CheckpointContainer(ctx, req)
6   recordCRIOperationMetrics("checkpoint", err)
7   return resp, err
8 }

```

此外，kubelet 的 instrumentedRuntimeService 接口用于收集运行时度量指标，需同步更新以支持新方法。更新后的 instrumentedRuntimeService 在调用 RemoteRuntimeService 的基础上，记录 C/R 操作的检查点创建时间和恢复耗时等性能数据。这种分层设计保留了 kubelet 的模块化特性，便于调试和维护，同时为 5G 核心网的高可靠性需求提供了可观测性支持。

3.4.3.4 小结

通过对 CRI 的 protobuf 规范、containerd 插件和 kubelet 的系统扩展，本研究成功集成了检查点与恢复功能，为热迁移提供了运行时支持。新增的 CheckpointContainer

和 `RestoreContainer` 方法与 CRI 现有框架高度兼容，`protobuf` 规范的调整确保了接口的标准化，`containerd` 插件的复用设计优化了实现效率，`kubelet` 的模块化更新提升了系统的可维护性。这些扩展不仅满足了 5G 核心网网元对无中断迁移的需求，还为 Kubernetes 生态的热迁移功能提供了通用支持。后续将在第3.5节通过实验验证 C/R 操作的性能表现。

3.4.4 热迁移 API 与控制器

K8S 作为基底的云原生基础设施交互采用声明式 API 模型（Declarative API），对于集群的一切操作交互都通过声明式 API 进行。操作者通过 API 描述资源需要达到的期望状态，而资源由当前状态向期望状态转移的具体过程由相应的控制器（Operator）负责管控。声明式 API 的这种机制与命令式 API（直接给出具体动作指令）形成鲜明对比。本小节将详细介绍热迁移 API 和热迁移控制器的实现。

3.4.4.1 热迁移 API

Kubernetes API Server 是集群的统一入口，通过身份验证、RBAC 权限控制和操作审计确保安全性，所有组件均通过 API Server 获取期望状态并执行协调。如第3.3.2.1节所述，热迁移依赖检查点与恢复操作生成快照，而这些操作需通过 API 触发并存储于 `etcd` 数据库。声明式 API 的中心化设计避免了多组件并发修改的冲突，为 Pod 热迁移提供了标准化的扩展接口。

遵循“一切皆资源”的设计理念，本研究将热迁移定义为一种自定义资源，通过声明式 API 描述其生命周期。热迁移 API 的设计遵循 Kubernetes 的核心原则，包括声明式配置、可观测性、一致性和可扩展性，确保与云原生生态的兼容性。API 包含两部分内容：系统信息和自定义信息。以下为热迁移 API 的核心字段伪代码：

```
1 apiVersion: migration.k8s.io/v1
2 kind: PodMig
3 metadata:
4   name: pod-migration-example
5   namespace: default
6 spec:
7   sourcePod: pod-name
8   targetNode: node-name
9   maxDowntime: 100ms
10  checkpointStrategy:
11    frequency: 1s
```

```
12     dirtyPageThreshold: 10MB
13     networkSlice:
14         sliceId: slice-001
15         qos: high
16     status:
17         phase: Running
18         conditions:
19             - type: MigrationReady
20             status: True
```

系统信息基于 Kubernetes 内置资源的核心属性,定义热迁移资源的基础结构,包括 API 版本 (apiVersion)、资源种类 (kind)、命名空间 (namespace) 和元数据 (metadata)。通过模板化规范 (spec.template), API 支持与调度器和容器运行时 containerd 的直接交互。为增强可观测性, API 设计了状态反馈字段,记录迁移阶段 (Pending、Running、Completed) 和条件状态 (MigrationReady), 由控制器实时更新。最后, 系统信息部分保留了与 K8S 标签机制的集成, 支持通过标签选择实现多 Pod 迁移的灵活性。这种设计不仅保证了 API 与 K8S 生态的兼容性, 还为底层容器迁移技术的调用提供了标准化接口。

自定义信息部分针对 5G 核心网的特殊需求,扩展了 API 的功能性和适应性。首先, 为满足服务质量 (QoS) 要求, API 引入迁移配置参数: 最大停机时间 (maxDowntime) 和检查点策略 (checkpointStrategy)。参数允许用户根据业务优先级动态调整迁移行为, 提高面对不同场景的灵活性。其次, API 支持与网络切片高可用性的集成, 预留字段存储切片标识和 QoS 策略, 确保迁移决策符合 5G 网络的动态需求。

该 API 通过自定义资源组 (migration.k8s.io) 实现, 完整定义见附录 A。热迁移任务创建后, API Server 将其存储至 etcd, 由热迁移控制器负责执行, 触发 CRI 扩展的 C/R 操作。

3.4.4.2 热迁移控制器

在深入探讨热迁移控制器的具体实现之前, 有必要先对 Kubernetes 控制器模式及其运行原理进行简要介绍, 以奠定理解热迁移控制器设计的基础。控制器模式是 Kubernetes 核心架构的关键组成部分, 负责管理集群资源的生命周期并协调其状态一致性。在 5G 核心网网元热迁移场景中, 热迁移控制器需动态管理 Pod 的迁移过程, 确保服务无中断并优化资源调度效率。

控制器模式基于反馈控制理论，通过持续对比资源的实际状态（status）与期望状态（spec），执行调谐操作以收敛至目标状态。控制器通过 API Server 实时监控 etcd 中的资源对象，采用事件驱动或定时轮询机制感知变化。以 Deployment 为例，其控制器根据 spec 中的副本数调整 Pod 数量，状态变更通过 API Server 透明完成。控制器模式具有三大特性：幂等性确保多次操作结果一致，解耦性通过 API 实现模块化交互，容错性支持自动重试以应对故障。这些特性为热迁移的复杂协调提供了高可靠性和稳定性。

本研究提出的热迁移控制器即基于 Operator 模式，利用 PodMig 资源实现 Pod 的动态迁移。热迁移控制器负责协调 PodMig 资源的生命周期，确保 Pod 在节点间的无缝迁移。方案的核心在于控制循环的设计与资源监控配置。在初始化阶段，控制器通过 SetupWithManager 注册 PodMig 为监控资源，同时指定其创建的 Pod 类型一并监控。为便于管理，控制器按 PodMig 名称对 Pod 建立索引，确保高效查询相关资源。这一配置利用了 kubernetes 提供的功能，使控制器能够实时感知状态变化。

控制循环由 Reconcile 方法驱动，当 PodMig 或其 Pod 发生变化时触发。该方法设计为幂等，仅依赖可观察状态运行，支持控制器的无状态特性，从而提升扩展性和高可用性。图3.9展示了热迁移控制器单个调谐周期的具体流程。

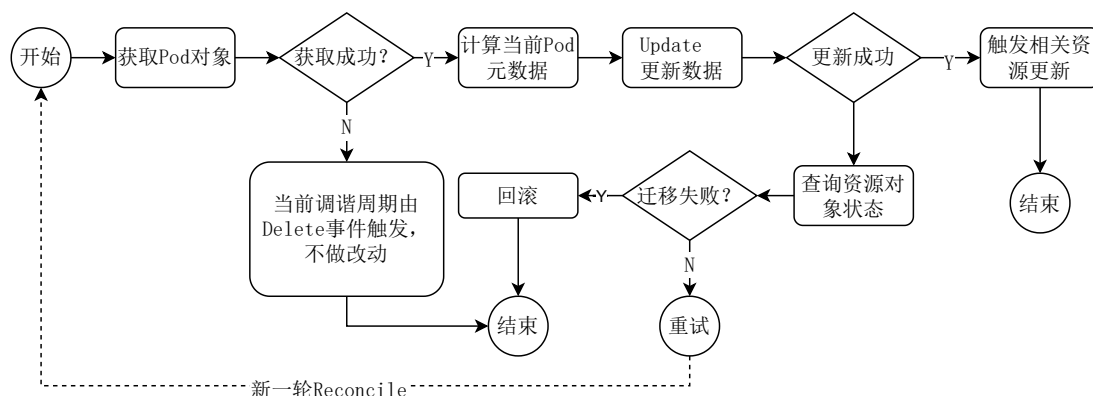


图 3.9 热迁移控制器 Reconcile 循环

图 3.10展示了 PodMig 资源的状态转换，从 Creating 到 Completed，清晰呈现迁移的阶段流转。状态管理包括 State 和 ActivePod 两个状态，其中 State 反映 PodMig 的阶段，ActivePod 指向当前活跃 Pod。

实际操作根据当前状态分步执行。若无 Pod 存在，则创建初始 Pod 作为源 Pod；若源 Pod 被标记删除且无目标 Pod，则创建目标 Pod 触发迁移；若迁移完成，则清理

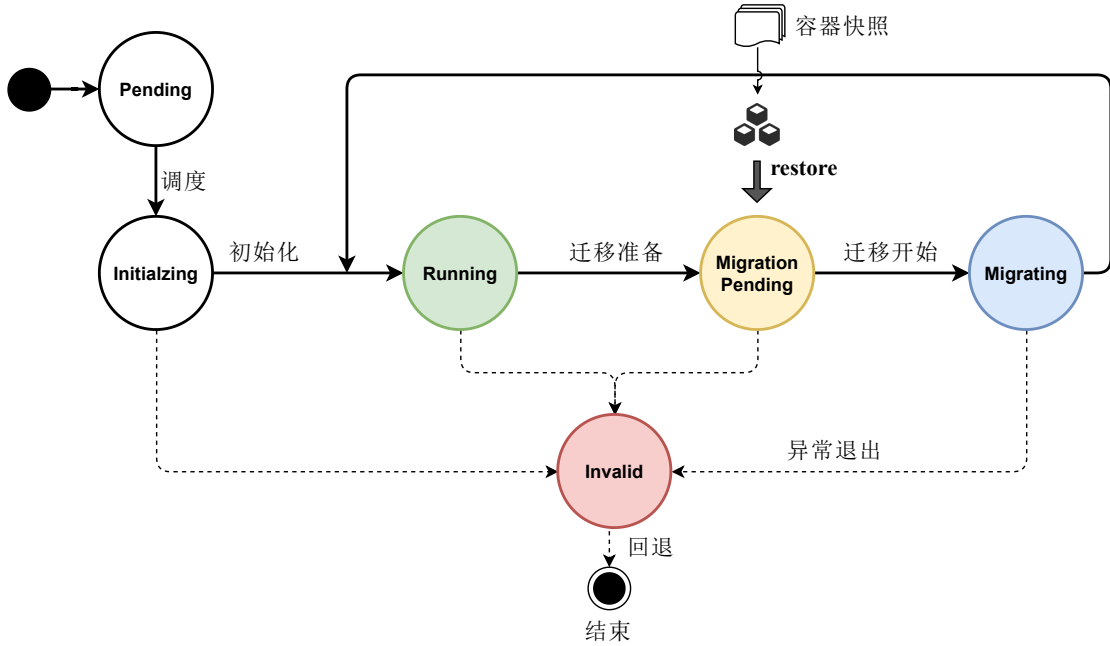


图 3.10 热迁移 Pod 状态转换

相关资源。控制器遵循先更新状态再修改资源的惯例，操作完成后循环再次触发，确保状态与行为的逐步协调。

通过控制器模式与 PodMig 的结合，本研究实现了热迁移控制器的设计与开发。API 的模块化设计提供了标准化接口，控制器模式提供了闭环反馈和幂等性支持，确保迁移过程的稳定性和一致性。热迁移控制器通过状态管理和分步操作，配合 CRIU 技术实现了 Pod 的无中断迁移。方案不仅满足了 5G 核心网网元的低时延和高可靠性需求，还为 Kubernetes 生态的热迁移功能提供了可扩展基础，具体性能验证见第 3.5 节。

3.5 实验分析

为验证本章提出的热迁移方案在 5G 核心网场景中的性能，本节对原型系统进行了系统性评估。通过模拟接入与移动性管理功能网元的迁移，测试方案在不同负载和网络条件下的服务中断时间、总迁移时间、QoS 请求成功率和资源开销，分析迁移前后状态的完整性，并评估其对 5G 核心网低时延和高可靠性的支持。

3.5.1 实验环境

3.5.1.1 软硬件资源

实验采用高性能服务器和专用网络设备搭建测试集群，确保硬件资源满足 5G 核心网网元的高并发需求。表 3.3 和表 3.4 分别列出了硬件和软件资源的详细配置。

表 3.3 实验硬件资源

硬件	数目	配置
Dell PowerEdge T640	2	处理器：Intel Xeon Gold 6248R CPU @ 3.00GHz 内存：512GB 以太网：Broadcom NetXtreme E-Series Advanced 10GBASE-T Ethernet LOM
vMotion Router	1	网络带宽：2.5 Gigabit Ethernet
Gateway Router	1	网络带宽：2.5 Gigabit Ethernet

表 3.4 实验软件资源

软件工具	说明	来源
VMware ESXi	分布式硬件层虚拟化 OS	https://www.vmware.com/
Kubernetes	分布式容器编排系统	https://kubernetes.io/
Containerd	容器运行时	https://containerd.io/
Prometheus	系统指标时序数据抓取	https://prometheus.io/
Open Source MANO	核心网网元管理	https://osm.etsi.org/
iPerf	网络性能测试	https://iperf.fr/

3.5.1.2 测试环境

测试集群部署在 T640 服务器（图3.11）上，采用虚拟化集群节点的方式构造由一个主节点和两个工作节点组成的 Kubernetes 集群，运行于 Ubuntu Linux 20.04 操作系统。主节点部署 kube-apiserver、etcd 和 kube-controller-manager，负责集群管理和状态存储。工作节点运行 kubelet、kube-proxy 和 containerd（集成第3.4.3节的 CRI 扩展），执行 Pod 调度和容器操作。网络插件采用 Calico，默认配置 BGP 模式，支持节点间 2.5Gbps 带宽，确保低时延通信。

时性和运维效率。该指标反映了迁移过程的完整延迟，直接影响方案对于网元稳定性的影响。

测量方法：通过迁移控制器日志记录时间节点，从控制器接收迁移指令开始计时，至目标 Pod 状态转为 Running 且会话操作成功率 >99% 为止。

(3) **停机时间**：停机时间是指在 Pod 迁移过程中，容器服务不可用的时间段，即从源 Pod 进入终止状态至目标 Pod 完成恢复并首次成功响应外部请求之间的时间间隔。在 5G 核心网场景中，控制面网元服务的中断将直接影响用户体验和网络性能。该指标直接体现了方案对网络中断的影响程度，停机时间越短，迁移的业务透明性越高，服务连续性越好。停机时间的测量方法：在源 Pod 和目标 Pod 上部署就绪探针，记录最后一个成功请求离开源 Pod 到第一个成功请求到达目标 Pod 的时间差。

(4) **CPU 资源开销**：资源开销指标评估热迁移对系统资源的额外消耗，反映方案对集群负载的影响程度。本实验聚焦于 CPU 使用率的测量，分析迁移期间源节点和目标节点的 CPU 负载变化。理想的热迁移方案应在保证迁移效果的同时，尽量降低资源开销，并保持 CPU 使用的平缓性，避免剧烈波动，减少对系统性能的瞬时冲击以导致因资源竞争而影响其他服务的风险。

测量方法：通过 Prometheus 采集测试环境中系统层级和服务层级的 CPU 使用率数据，记录迁移期间源节点和目标节点的 CPU 使用率峰值、平均值及变化趋势，分析资源开销的动态特性。

3.5.3 实验过程

本实验在中国电子科技集团公司某所的测试网络环境中进行，基于云原生环境适配的 5G 独立组网（SA）核心网原型，包含接入与移动性管理功能（AMF）、会话管理功能（SMF）、网络功能注册功能（NRF）、用户平面功能（UPF）、统一数据管理（UDM）、认证服务器功能（AUSF）、统一数据存储（UDR）和网络切片选择功能（NSSF）共 8 个网元。实验通过模拟 AMF 网元的迁移，验证热迁移方案在高并发、低时延场景下的性能表现。

3.5.3.1 测试场景

实验选取 AMF 网元作为测试对象，因其在 5G 核心网中负责用户设备（UE）的注册、连接、移动性和认证授权，直接影响会话建立和移动性事件的处理效率。AMF 的迁移性能能够直观反映热迁移方案的适用性：一方面，AMF 需处理频繁的 UE 接入

请求，对停机时间和服务连续性敏感；另一方面，其会话状态（如 UE 上下文、NAS 认证数据）复杂且关键，迁移需确保状态完整性以避免中断或重连引发的延迟和拥塞。

为模拟真实负载，实验采用 UERANSIM 生成 UE 和 gNB 的接入流量，覆盖会话建立和数据传输场景。负载配置通过调整 UERANSIM 实例数和 UE 数量，实现每秒请求数（QPS）从 100 到 400 的递增测试。每个 UE 在注册成功后以 1 秒周期发送 SR 请求，构建持续、均匀的控制面负载。实验在 100、200 和 400 UE 规模下分别进行三轮测试，每轮实验中第五秒开始触发热迁移或删除重建操作。分布式存储系统 Ceph 提供检查点文件的高可用存储，支持快速快照传输。网络插件采用 Calico，配置 BGP 模式，确保 Pod 间 2.5Gbps 带宽的低时延切换。

表 3.5 AMF 负载测试配置

UE 总数	UERANSIM 实例	QPS	AMF 分配内存 (GB)
100	1	100	0.5
400	2	400	1
400	4	400	2

3.5.3.2 基线测试

基线测试采用传统 Pod 删除-重建方式（Delete-Recreate），模拟 AMF 网元的常规迁移，作为热迁移方案的性能参照。具体步骤如下：

- D1. 在节点 A 部署 AMF Pod（amf-pod-0），通过 UERANSIM 初始化 UE 会话，验证服务正常运行。
- D2. 使用 `kubectl delete` 命令删除 amf-pod-0，触发 Kubernetes 调度器的自动恢复机制（Autohealing）在节点 B 重建 Pod（amf-pod-1）。
- D3. 记录停机时间（Downtime）：重建过程中，记录从 Pod 删除到新 Pod（amf-pod-1）恢复服务的时间，作为停机时间。
- D4. 记录总迁移时间（Total Migration Time）：从删除指令发出到 amf-pod-1 状态转为 Running 且 QoS 请求成功率 > 99% 的耗时，通过 PodMig 控制器日志提取。
- D5. 测量资源开销：通过 Prometheus 采集节点 A 和 B 的 CPU 与内存使用率，分析峰值与平均值。

3.5.3.3 热迁移测试

热迁移测试基于本章的 Clone-Restore 方案，通过 PodMig 控制器协调 AMF 网元的无中断迁移，验证低时延和高可靠性的实现效果。具体步骤如下：

- C1. 在节点 A 初始化 AMF Pod (amf-pod-0) 及其他网元 Pod, 通过 UERANSIM 建立 UE 会话, 验证服务正常运行 (QoS 请求成功率 > 99%)。调用 CRI 的 Checkpoint 方法, 生成 AMF 基础快照, 并存储至 Ceph。
- C2. 使用 kubectl apply 命令创建 PodMig CR 资源, 指定源 Pod (amf-pod-0)、目标节点及检查点策略。
- C3. PodMig 控制器检测到状态变更 (MigrationPending), 调用 CRI 的 Checkpoint 方法, 生成增量快照直至达到终止条件。
- C4. 快照传输至节点 B, 控制器调用 Restore 方法重建目标 Pod (amf-pod-1), 并执行最终同步。
- C5. 记录停机时间 (Downtime): 从源 Pod 冻结到目标 Pod 首次成功响应 SR 请求的间隔, 使用就绪探针测量。
- C6. 记录总迁移时间 (Total Migration Time): 从 PodMig 资源创建到 amf-pod-1 状态转为 Running 且 QoS 请求成功率 > 99% 的耗时。
- C7. 验证状态一致性: 比对迁移前后的 NAS Session ID, 评估状态留存性能。
- C8. 测量资源开销: 通过 Prometheus 采集节点 A 和 B 的 CPU 与内存使用率, 分析 CRIU 快照操作与数据同步带来的负载。

3.5.4 实验结果分析

基于第3.5.3节的实验过程, 本小节分析热迁移方案 (Clone-Restore) 与传统方案 (Delete-Recreate) 在 100、200 和 400 UE 规模下的性能表现, 重点对比停机时间 (Downtime)、QoS 请求成功率 (Request Success Rate)、总迁移时间 (Total Migration Time) 和资源开销 (Resource Overhead), 验证方案在 5G 核心网 AMF 网元迁移中的低时延和高可靠性优势。

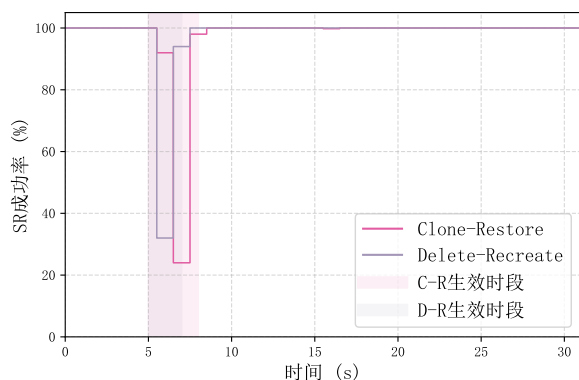


图 3.12 低负载-SR 成功率

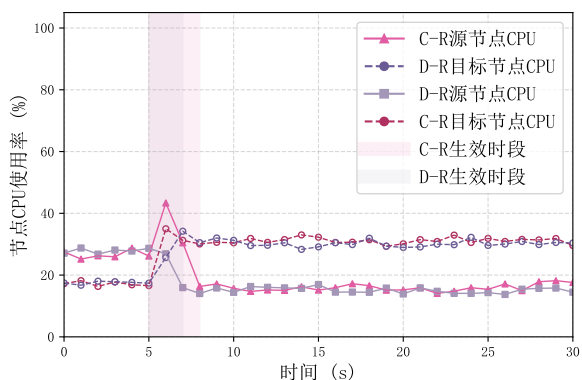


图 3.13 低负载 - 节点 CPU

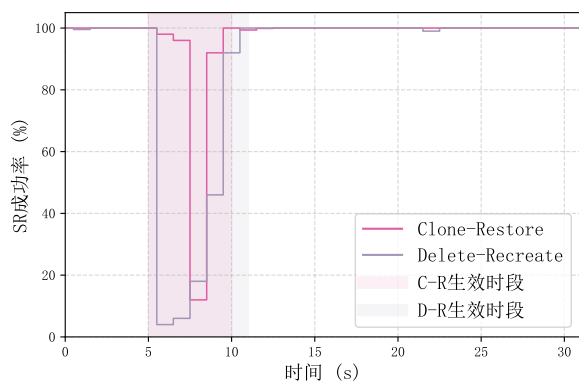


图 3.14 中负载-SR 成功率

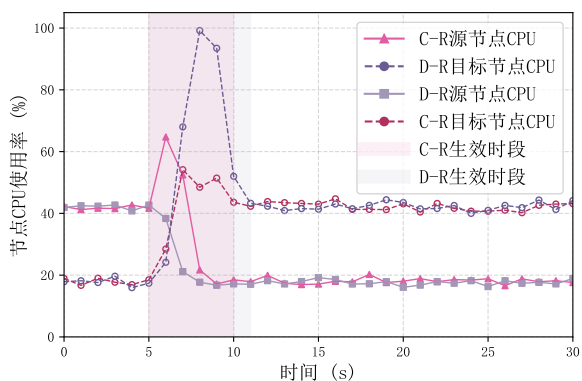


图 3.15 中负载 - 节点 CPU

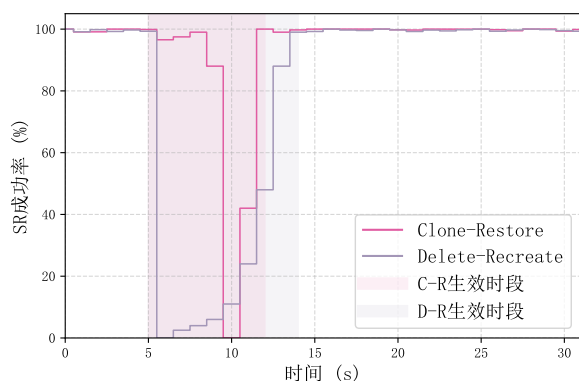


图 3.16 高负载-SR 成功率

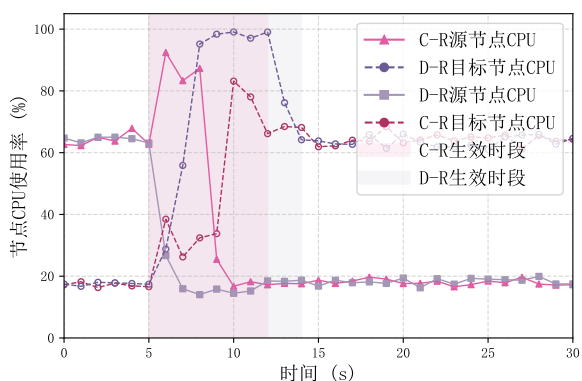


图 3.17 高负载 - 节点 CPU

图 3.16 和图 3.17 展示了在 400 QPS 高负载下，Clone-Restore 方案与 Delete-Recreate 方案的 SR 请求成功率和 CPU 占用率随时间的变化。需要说明的是，受限于 Kubernetes 组件限制，SR 成功率统计和 CPU 占用率采样周期为 1 秒，更细粒度的采样实现较为复杂，且对工作节点性能影响较大，SR 成功率由 UERANSIM 日志统计，CPU 占用率通过 Prometheus 时序数据库获取。

SR 请求成功率：Clone-Restore 方案在迁移前（0-5 秒）保持 100% 的成功率，迁移阶段（10-15 秒）短暂下降至最低约 36%，并在 1.82 秒内恢复至 100%，此后稳定运行。相比之下，Delete-Recreate 方案在触发 Pod Autohealing 后由于原 Pod 处于不可用状态（Downtime），在新 Pod 准备就绪的 2.3 秒内成功率为 0%，新 Pod 就绪后，由于此前会话状态随原 Pod 被销毁，大量 UE 发送重连请求的瞬时流量导致 CPU 占用打满，此时由于大部分 UE 会话未建立，AMF 无法处理 SR 请求导致成功率无法上升，新 Pod 就绪后因会话重建导致“平台期”（成功率低至 10%）和“激增期”（瞬时流量冲击），至 19 秒才恢复至 100%。Clone-Restore 方案通过动态增量快照和 Pod 带状态恢复机制显著减少了服务恢复时间，体现了 PodMig 控制器对状态继承的高效支持。

CPU 占用率：Clone-Restore 方案在系统资源开销方面优于 Delete-Recreate，展现了更高的资源效率。图3.13，3.15，3.17展示了三种负载规模下两种方案迁移期间的节点 A 与节点 B 的 CPU 使用率变化。Clone-Restore 节点 A 在迁移开始后出现峰值，对应克隆阶段增量快照生成过程，达到终止条件后原 AMF 停止服务，进行“last-sync”后进入销毁过程，此时节点 A 的 CPU 占用率恢复无网元负载的水平平稳运行。Delete-Recreate 方案在新网元就绪后，节点 B 的 CPU 使用率快速升至峰值约 99%，且持续时间更长，主要原因是 Delete-Recreate 的冷启动机制：Pod 重建后完全丢失原有 AMF 上下文，所有 UE 需重新完成注册、建立 NAS 安全上下文和 PDU 会话流程，导致大量并发会话请求集中涌入，引发控制面拥塞，进而造成 UE 多次注册重传、部分请求超时或失败，进一步拉低服务稳定性。两方案在迁移后的稳定状态下 CPU 使用率基本趋于一致，但 Clone-Restore 的总 CPU 占用时间更短，且避免了突发性资源占用飙升，体现了更优的资源效率。这种平缓的资源使用特性减少了对系统性能的瞬时冲击，特别是在 5G 核心网高负载场景下，能够有效避免因资源竞争导致的其他服务受影响。Clone-Restore 的资源效率得益于其状态继承机制，减少了会话重建的开销。

迁移时间：总迁移时间指从迁移操作发起到 QoS 请求成功率恢复至 99% 以上的耗时。如表3.6所示，Clone-Restore 方案在 100、200 和 400 UE 规模下的总迁移时间分别为 854ms，1283ms，2229ms，而 Delete-Recreate 方案的总迁移时间分别为 741ms，3227ms，5871ms，增长趋势明显，特别是在 400 UE 规模下，受冷启动和会话重建开销影响，恢复时间延长了近 8 倍。相比之下，Delete-Recreate 方案在低负载的场景下表现较好，而 Clone-Restore 方案在面临高请求压力时拥有更短的总迁移时间，二者对比如图3.18所示，三种场景下 Clone-Restore 的总迁移时间分别比 Delete-Recreate

短-15.25%，60.36%，62.03%，结果体现了热迁移方案的状态保存机制在有状态网元高可用性保障方面表现出了显著优于基线方案的性能。

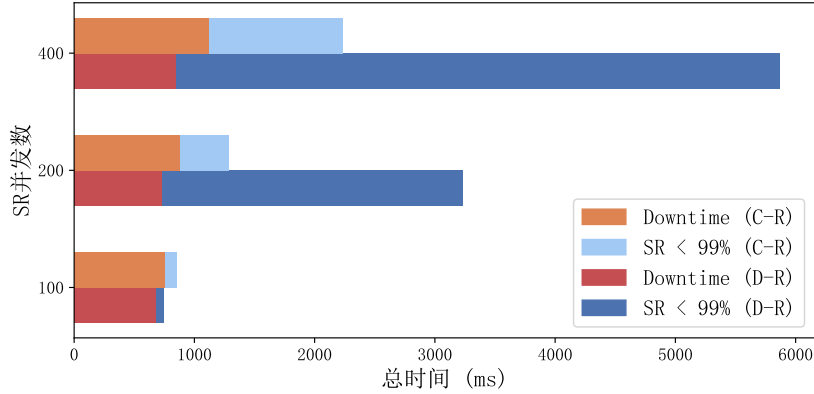


图 3.18 不同负载迁移时间对比

停机时间：Delete-Recrate 方案无法继承迁移前网元状态，每次重启都是从零开始的冷启动过程，因此不同 UE 规模下的停机时间都在 0.7s 左右，变化较小，反映了冷启动停机时间对 UE 规模的低敏感性，但整体时延远高于 Clone-Restore。三种负载下 Clone-Restore 方案的停机时间都长于 Delete-Recrate，分别长了 2.80%，19.86 和 31.41%，Clone-Restore 的停机时间随负载增加而略有延长，主要是由于快照数据量和同步时延的增加，但仍保持在秒级范围内。

表 3.6 不同迁移方案在不同 UE 规模下的性能对比

UE	total-CR	down-CR	SR<99%-CR	total-DR	down-DR	SR<99%-DR
100	854	762	92	741	680	61
200	1283	881	402	3237	735	2502
400	2229	1121	1108	5871	853	5018

本研究聚焦于缩短服务迁移总体时间以及减少服务重配置对服务稳定性的影响。Clone-Restore 方案在 3 种负载下的综合性能显著优于 Delete-Recrate。总迁移时间均低于传统方案；SR 成功率恢复效率更高；CPU 占用率峰值低于 Delete-Recrate，且持续时间短，波动小，体现出更优的资源效率。该方案通过 PodMig 的状态管理、和 CRI 的 C/R 操作，将 AMF 网元的中断时间压缩至秒级，快速恢复服务请求成功率至 99% 以上，满足 5G 核心网的低时延和高可靠性需求。虽然目标 Pod 的恢复时间随内存规模增大，但在实际部署中，网元实例内存通常控制在 2GB 以下，Clone-Restore 的综合

优势更加明显。在 5G 核心网低时延场景中，快速恢复服务的能力比单纯缩短停机时间更具实际意义，Clone-Restore 在此方面具有更好的性能。未来可通过提升带宽或优化 CRIU 加载效率进一步缩短大内存场景下的迁移时间。

3.6 本章总结

本章提出了一种云原生 5G 核心网场景下有状态网元热迁移方案。在原有云原生基础设施 Kubernetes 的基础上对 CRI 等多个关键组件进行拓展，实现了 Pod 运行中容器状态与非容器状态的增量克隆与恢复，赋予了云原生集群 Pod 热迁移的能力；同时，设计并实现了网元热迁移控制体系，通过协调整个迁移过程，确保了状态迁移的完整性、一致性，以及安全会话状态的恢复。为了评估热迁移方案性能，本章通过系统性实验，对比基于 Clone-Restore 的热迁移方案 and 传统 Delete-Recreate 方案在 5G 核心网 AMF 网元迁移中的性能表现。实验结果表明，Clone-Restore 方案在低时延和高可靠性方面显著优于 Delete-Recreate 方案。在 400 QPS 负载下，Clone-Restore 的总迁移时间为 2.23 秒，远低于 Delete-Recreate 的 5.87 秒，SR 请求成功率在 1.10 秒内恢复至 >99%，体现了对网元服务连续性的高效支持。CPU 占用率峰值较低且持续时间短，迁移后波动小，展现出更优的资源效率。该方案通过 PodMig 控制器实现状态管理，结合动态增量快照和 CRI 的 C/R 操作，有效减少了迁移中断时间，满足了 5G 核心网秒级中断和高连续性的需求，为实现既高效又安全的网元动态管理奠定了坚实基础。

第 4 章 云原生 5G 核心网智能切片编排算法

4.1 引言

网络切片（Network Slicing）作为 5G 核心网的关键特性，为支持多样化的业务场景提供了灵活的资源分配与服务隔离能力。3GPP 标准中根据不同场景定义了 3 个具有代表性的切片类别：增强移动宽带 eMBB、超可靠低时延通信 uRLLC、海量机器通信 mMTC^[2]。网络切片普遍被认为是解决 5G 及未来网络中多样化使用场景和连接服务需求的关键工具。在云原生架构加持下，5G 核心网得以通过容器化技术实现网元的微服务化部署，利用 Kubernetes 等云原生基础设施进行更加灵活的资源管理和调度。然而，当前云原生基础设施在适配多租户切片场景时仍面临诸多挑战，特别是在涉及切片网元调度的智能化编排方面，Kubernetes 原生编排机制并不能很好地适配 5G 核心网的特定化场景需求，难以满足控制面 QoS 与资源高效利用的双重目标。

Kubernetes 传统调度机制主要基于节点的 CPU 和内存利用率，倾向于将微服务部署到资源充足且负载均衡的节点，旨在提升系统资源利用效率，但对服务 QoS 需求缺乏感知。在核心网切片运维场景中，这种机制存在以下局限性：

（1）**网元间拓扑与时延约束**。同一切片内的网元需部署在链路时延较低的节点以降低控制面信令传输时延，而传统调度机制无法感知节点间网络时延；

（2）**多维资源利用不足**。调度策略忽视网络带宽和磁盘 I/O 等资源，导致网络密集型或 I/O 密集型应用运行时节点负载不均，进而引发资源碎片化和集群整体利用率偏低；

（3）**动态迁移能力缺失**。Kubernetes 原生机制无法对运行中的服务实例进行动态迁移，仅支持“删除-重建”方式调整实例位置，导致服务不可用时间过长，运行状态丢失进一步影响 QoS 满足率。

针对上述问题，本章提出了一种名为 Deep Reinforcement Slice Scheduling (DRSS) 的智能切片编排算法。DRSS 基于深度强化学习技术，以马尔可夫决策过程为建模基础，针对云原生环境下的多租户网络切片部署提供优化方案，旨在实现切片控制面端到端时延和资源负载均衡的双重优化目标。DRSS 针对单个网元的调度与迁移设计离散动作空间，并结合第三章提出的 Checkpoint-Restore 迁移方案，通过 Clone 快照大小和节点间时延量化迁移开销，在 QoS 收益与资源最优化配置间寻求平衡。通过引入

节点间时延和切片信令端到端时延，DRSS 提升了云原生环境与 5G 核心网场景的适配性；通过综合考虑 CPU、内存和网络带宽等多维资源，缓解负载不均与资源碎片化问题；此外，通过将 DRSS 智能体嵌入云原生切片编排框架并结合热迁移机制，增强系统的高可用性与容灾能力，为 5G 核心网切片编排提供理论与技术支持。相较于传统方法，DRSS 能够自适应学习动态环境特征，实现切片网元的智能调度与迁移，从而提升系统性能。相较于传统方法，该策略能够自适应地学习动态环境特征，从而实现对切片网元的智能调度与迁移，提升系统的高可用性与容灾能力。

本章后续内容安排如下：第 4.2 节阐述切片调度问题的形式化建模；第 4.3 节详述 DRSS 的设计细节，包括网络结构与训练策略；第 4.4 节通过仿真实验评估 DRSS 的效率；第 4.5 节总结本章工作。

4.2 问题建模与算法设计

4.2.1 场景定义

本章聚焦云原生架构下 5G 核心网切片调度的优化问题，旨在解决资源异构、服务多样化和拓扑动态场景中网元动态部署与重配置的挑战，以降低切片信令传输时延并降低基础设施提供商运营成本。为此，本节从云原生服务调度机制和核心网控制面功能组成出发，对目标场景的关键要素进行建模，包括 Kubernetes 调度机制、核心网切片网元、集群拓扑以及调度与迁移机制，为后续马尔可夫决策过程的形式化定义奠定基础。

4.2.1.1 Kubernetes 服务调度机制

在云原生架构下，5G 核心网通过容器化技术实现网元的微服务化部署，以 Kubernetes 作为基础设施管理平台。Kubernetes 通过 Kube-scheduler 调度网元到集群节点，主要依据节点的 CPU 和内存资源可用性，并通过亲和性规则或手动标签实现简单优化，其具体调度流程已在第 2 章第 2.2.3 节中详细阐述。然而，这种机制在核心网切片调度场景中存在局限性：一是未充分考虑网元间的拓扑关系和时延需求，例如同一切片内网元需低时延连接以保障实时性；二是忽略网络带宽和磁盘 I/O 等关键资源，导致负载分布不均；三是缺乏动态迁移支持，无法自适应调整已部署网元的位置以应对时延变化。为此，本研究在 Kubernetes 基础上设计智能编排策略，通过引入深度强化学习技术，增强对切片复杂需求的适配性。

4.2.1.2 核心网切片网元

5G 核心网由多个功能网元组成，根据 3GPP TS 23.501^[2]标准，网元可分为共享网元和调度网元两大类，其分类及特性如表 4.1所示。共享网元因功能特性需全网统一部署或与控制面解耦，不参与动态调度；调度网元则需根据切片需求独立部署，属于本研究的优化对象。

表 4.1 控制面网元分类

类型	网元	特性
共享网元	NRF	全网服务注册与发现，统一部署
	NSSF	切片选择与映射，集中维护
	UPF	部署于边缘网络，控制面解耦
调度网元	UDR	用户数据存储
	UDM	数据管理
	AUSF	用户认证
	AMF	接入与移动性管理
	SMF	会话管理

(1) **共享网元**：包括 NRF、NSSF 和 UPF。NRF 提供全网服务注册与发现功能，需统一部署以确保一致性；NSSF 负责切片选择和映射，需集中维护；UPF 通常部署于边缘网络，与控制面解耦，不参与控制面切片调度。

(2) **调度网元**：包括 UDR、UDM、AUSF、AMF 和 SMF。这些网元直接服务于特定切片，需根据切片信令时延动态调度和迁移，以优化控制面性能。

网元建模以切片为单位展开，强调网元间的功能依赖和部署顺序，而非孤立调度单个网元。为确保服务一致性和效率，网元部署遵循以下顺序：首先部署 UDR 存储用户数据，随后部署 UDM 进行数据管理，依次部署 AUSF 完成用户认证、AMF 处理接入与移动性管理，最后部署 SMF 管理会话。这一顺序基于 5G 核心网的功能依赖关系^[2]，即 UDR 为 UDM 提供数据支持，AUSF 依赖 UDM 完成认证，AMF 需认证后接入用户，SMF 则依赖 AMF 提供会话上下文。通过优先满足前序网元的需求，这种设计有效降低了后续部署的时延风险，同时简化了切片部署过程。

4.2.1.3 集群拓扑

一个切片示例由两个部分组成，即底层硬件资源和上层网络切片。底层硬件资源包括计算、存储和网络等资源，在云原生架构下，5G 核心网网元通过容器化技术实现网元的微服务化部署，以 Kubernetes 作为基础设施管理平台；上层网络切片则由多个功能网元组成，可以被描述为通过 NF 组成的服务功能链（Service Function Chain, SFC），每个网元负责实现特定的网络服务，例如 AMF 负责接入和移动性管理，SMF 负责会话管理等。切片内的网元通过网络连接进行通信，因此它们的部署位置和时延特性对切片的性能有着重要影响。图 4.1 展示了一个示例集群拓扑，其中节点间的时延矩阵 L 表示了不同节点之间的网络延迟。为了降低切片内网元间的通信时延，后续部署的网元优先安排在与前序网元低时延连接的节点上，以达到切片端到端时延的优化。

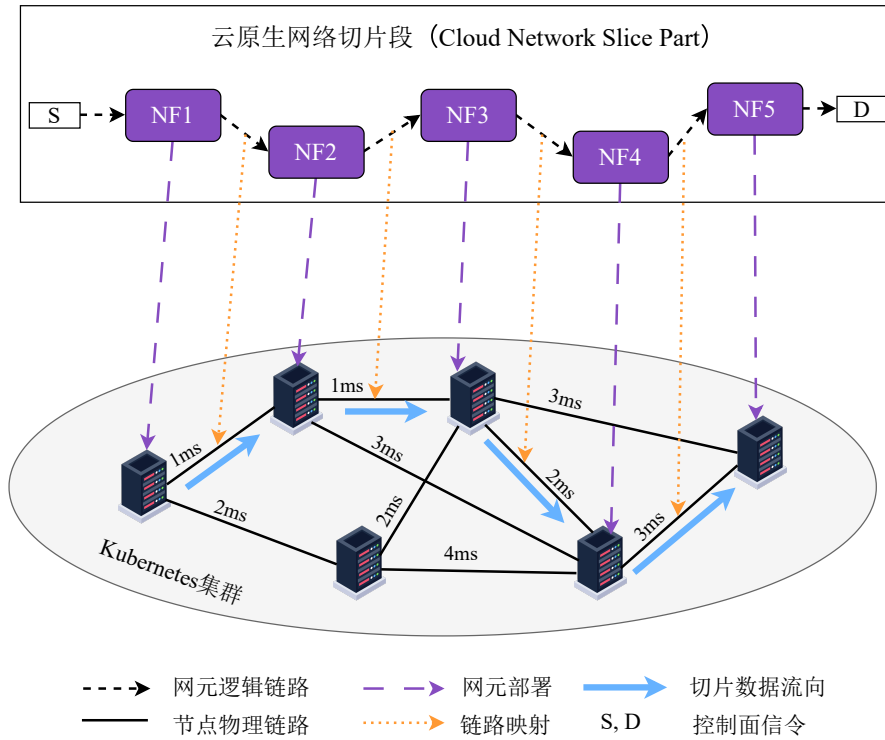


图 4.1 集群拓扑示例

集群由 N 个计算节点组成，各节点通过网络拓扑相互连接，节点间传输时延用矩阵 $L \in \mathbb{R}^{N \times N}$ 表示，其中 $L_{i,j}$ 表示节点 i 与节点 j 之间的往返时延，单位为毫秒。问题场景中，网络拓扑变化相对决策时间较缓慢^[65]，因此本研究假设时延矩阵 L 在调度周期内保持静态不变。为降低切片内网元间的通信时延，本研究引入拓扑感知

(Topology-Aware) 特性, 优化网元部署策略。具体而言, 后续部署的网元优先安排在与前序网元低时延连接的节点上, 以减少切片内部信令交互时延, 例如将 SMF 部署在与 AMF 时延较低的节点。

4.2.1.4 调度与迁移机制

在云原生 5G 核心网切片编排中, 网元的部署与重配置是在动态计算环境中维持服务质量和资源利用率的关键环节。本研究设计了包含初始调度和动态迁移两种操作的机制, 以实现网元的高效部署与自适应调整, 相较于传统静态调度方法, 动态迁移显著提升了系统对负载变化和时延变化的响应能力。

网元部署与重配置包含以下两类操作:

(1) **初始调度**。当切片部署过程中到达尚未部署的网元时, 该网元将被调度至某一节点。调度决策综合考虑节点的资源可用性、拓扑时延, 确保网元初次部署即满足基本服务需求。

(2) **动态迁移**: 若已部署网元无法持续满足 QoS 要求, 则触发网元迁移评估。迁移操作通过将网元从当前节点热迁移至另一节点, 动态优化其部署位置, 以在可接受的服务稳定性影响下提升 QoS 满足率和资源利用效率。

第三章的 3.3.2 小节详细介绍了网元热迁移的原理, 迁移过程如图 3.4 所示。切片触式调度器的请求由切片用户提交, 而迁移操作的请求由第三章实现的迁移控制器负发起, 控制器定期评估切片时延, 基于端到端时延和迁移代价计算, 决定是否执行热迁移。

4.2.2 问题建模

基于上一节的场景定义, 本研究将 5G 核心网切片中网元的调度与迁移问题形式化为一个马尔可夫决策过程 (Markov Decision Process, MDP), 并通过深度强化学习方法求解最优部署策略。相关符号如表 4.2 所示。

在 MDP 框架下, 智能体 (Agent) 在每个时间步 t 接收环境状态 $s_t \in \mathcal{S}$, 选择调度或迁移动作 $a_t \in \mathcal{A}$, 执行动作后转移至下一状态 s_{t+1} , 并获得即时奖励 $r_t = \mathcal{R}(s_t, a_t)$ 。整个过程构成一个 MDP, 定义为五元组:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle, \quad (4.1)$$

其中 $\mathcal{S}$ 为状态空间, $\mathcal{A}$ 为动作空间, $\mathcal{P}$ 为状态转移概率函数, $\mathcal{R}$ 为奖励函数, $\gamma \in [0, 1)$

表 4.2 符号说明表

符号	含义
$s_t, \mathcal{S}_t, \mathcal{C}_t$	状态向量; 切片信息; 节点负载
$\widehat{\mathcal{L}}_t, \mathcal{R}_t, \mathcal{P}_t$	节点时延拓扑; 当前网元资源需求; 前序网元位置
$\mathcal{D}_t$	当前切片总时延
$a_t, \mathcal{A}_t$	时间步 t 的动作; 动作空间
$r_t, \mathcal{R}$	时间步 t 的即时奖励; 奖励函数
$\mathcal{M}, \mathcal{P}, \gamma$	马尔可夫决策五元组; 状态转移概率函数; 折扣因子
$\pi, \pi^*, \mathbb{E}_\pi$	策略; 最优策略; 策略 π 下的期望
n, N, v	集群中的节点总数; 网元总数; 网元部署顺序索引
$C_k^i, (\text{Util}_k)_t^i$	节点 i 的资源 k 的容量; 节点 i 的资源 k 的利用率

为折扣因子。本小节将详细阐述各组成部分的设计。

4.2.2.1 状态空间

根据马尔可夫性质，集群在时间片 t 的状态仅与前一时刻的状态和待调度网元的资源需求相关。因此，我们定义状态向量 s_t 以全面刻画集群的资源利用状况和待调度网元的特性，为智能体提供充分的决策依据。状态空间由三部分组成：

$$s_t = (\mathcal{S}_t, \mathcal{C}_t, \widehat{\mathcal{L}}_t), \quad (4.2)$$

其中 $\mathcal{S}_t$ 表示当前切片信息， $\mathcal{C}_t$ 表示集群节点当前负载情况， $\widehat{\mathcal{L}}_t$ 表示节点时延拓扑。

当前切片信息 $\mathcal{S}_t = (\mathcal{R}_t, \mathcal{P}_t, \mathcal{D}_t)$ 包括当前调度网元的资源需求、前序网元位置和当前总时延，其中 $\mathcal{R}_t$ 表示资源需求，考虑多维资源，包括 CPU、内存和网络带宽，为确保稳定性，这些资源需求基于历史运行数据计算平均值，并归一化为 $[0, 100]$ 区间； $\mathcal{P}_t$ 表示前序网元的位置，用于反映网元间的依赖关系； $\mathcal{D}_t$ 表示当前切片的总时延，基于已部署网元间的通信时延累加计算，用于评估端到端时延是否满足要求。

集群节点当前负载情况 $\mathcal{C}_t = (C_t^1, C_t^2, \dots, C_t^n)$ 表示集群中 n 个节点的负载，其中每个节点 C_t^i 包含多维资源利用率，涵盖 CPU、内存和网络带宽资源，为反映资源的动态变化，我们采用过去一段时间的平均利用率，并归一化为 $[0, 100]$ 区间。

$\widehat{\mathcal{L}}_t$ 表示节点时延拓扑，即节点间的时延矩阵，用于衡量网元间通信延迟，以支持端到端时延优化。时延矩阵的引入基于 5G 核心网切片对低时延的严格要求，智能体可通过优化网元部署位置减少信令交互延迟。

4.2.2.2 动作空间

在强化学习框架下，调度器作为智能体，在接收到网元的部署请求时执行动作。智能体的动作是从集群中选择一个节点来部署或迁移当前网元，因此动作空间包含集群中所有可能的节点。对于包含 n 个节点的系统，动作空间定义为：

$$\mathcal{A}_t \in \{\text{node}_i \mid i = 0, 1, \dots, n-1\}, \quad (4.3)$$

其中 $\mathcal{A}_t$ 表示时间片 t 的动作，即选择节点 node_i 部署或迁移网元。动作的具体含义由网元的部署状态决定：若网元未部署，则表示将网元调度至节点 node_i ；若网元已部署，则表示将网元迁移至节点 node_i 。这种设计通过状态空间中的网元部署信息判断动作类型，避免了动作空间的冗余划分，简化了智能体的决策过程。

调度过程包括节点选择和绑定两个阶段。在节点选择阶段，智能体根据状态 s_t 选择一个节点；随后进入绑定阶段，将网元绑定到所选节点。若因为节点资源不足等原因绑定失败，网元将被加入重调度队列，等待下一次调度，重调度的时间间隔将随失败次数逐步增加。

4.2.2.3 状态转移机制

状态转移概率 $\mathcal{P}(s_{t+1} \mid s_t, \mathcal{A}_t)$ 描述了在状态 s_t 下执行动作 $\mathcal{A}_t$ 后，系统状态从 s_t 转移到 s_{t+1} 的规律。根据状态空间 $s_t = (\mathcal{S}_t, \mathcal{C}_t, \widehat{\mathcal{L}}_t)$ 的定义，状态转移主要来源于以下因素：

(1) **网元位置更新**：执行动作 $\mathcal{A}_t$ 选择节点 node_i 后， $\mathcal{S}_t$ 中的前序网元位置 $\mathcal{P}_t$ 更新为目标节点编号 i ，反映网元的调度或迁移。

(2) **资源占用变化**： $\mathcal{C}_t$ 中的节点负载信息根据网元的资源需求 $\mathcal{R}_t$ 更新，若为调度，则目标节点资源利用率增加；若为迁移，则原节点资源释放，目标节点资源占用。

(3) **时延信息调整**： $\mathcal{S}_t$ 中的总时延 $\mathcal{D}_t$ 根据网元的新位置和 $\widehat{\mathcal{L}}_t$ 重新计算，反映端到端时延的变化。节点时延拓扑 $\widehat{\mathcal{L}}_t$ 保持不变。

由于面临资源竞争、网络波动等环境复杂性， $\mathcal{P}$ 难以通过解析方式建模。本研究

采用“黑箱”方式，通过仿真环境的 **step** 函数模拟状态转移，假设转移为确定性，即给定 s_t 和 $\mathcal{A}_t$ ， s_{t+1} 唯一，以简化计算和训练过程。

4.2.2.4 奖励函数

奖励函数 $\mathcal{R}(s_t, a_t, v)$ 反映了调度算法的优化目标，引导智能体生成高效的网元部署策略。本研究聚焦切片端到端时延、资源利用率和负载不均衡度三个目标。

(1) **平均资源利用率**。集群资源利用率 AvgUtil_t 定义为各节点资源利用率的平均值，考虑 CPU、内存和网络带宽三种资源：

$$\text{Util}_t^i = \frac{(\text{Util}_{\text{CPU}})_t^i + (\text{Util}_{\text{Mem}})_t^i + (\text{Util}_{\text{Width}})_t^i}{3}, \quad (4.4)$$

$$\text{AvgUtil}_t = \frac{1}{n} \sum_{i=1}^n \text{Util}_t^i, \quad (4.5)$$

其中 n 为节点总数。

(2) **负载不均衡度**。负载不均衡度 ImBalance_t 通过各类资源利用率的标准差加权计算：

$$\text{ImBalance}_t = \frac{1}{3} \sum_{k \in \{\text{CPU}, \text{Mem}, \text{Width}\}} \text{std}((\text{Util}_k)_t^1, (\text{Util}_k)_t^2, \dots, (\text{Util}_k)_t^n), \quad (4.6)$$

其中 $\text{std}(\cdot)$ 表示标准差。

(3) **切片端到端时延**。切片端到端时延通过测量切片内网元间的信令交互时延来评估 QoS 满足程度。对于切片 i 在部署第 v 个网元时的时延满足度 $q_i^{(v)}$ ，定义为：

$$T_t = \sum_{i=1}^q \left(1 - \frac{L_a^{(i,v)} - L_t^{(i)}}{L_t^{(i)}}\right), \quad (4.7)$$

其中 q 为网元链路总数， $L_a^{(i,v)}$ 和 $L_t^{(i)}$ 分别为切片 i 在部署第 v 个网元时的实际端到端时延和目标时延。若 $L_a^{(i,v)} \leq L_t^{(i)}$ ，则 $q_i^{(v)} = 1$ ；否则 $q_i^{(v)}$ 线性下降，最小为 0。

最终，奖励函数可表示为：

$$\mathcal{R}(s_t, a_t, v) = \begin{cases} \beta_v \text{AvgUtil}_t - \delta_v \text{ImBalance}_t, & \text{if } v < V_i, \\ \alpha_v T_t + \beta_v \text{AvgUtil}_t - \delta_v \text{ImBalance}_t, & \text{if } v = V_i, \end{cases} \quad (4.8)$$

其中 V_i 表示切片 i 的网元总数，该奖励受切片上所有关联网元间的时延影响，因此仅在切片部署完毕后才能计算得到，因此将总奖励进行分段拆分，对于最后一个网元 ($v = V_i$) 引入切片端到端时延因子。

4.2.2.5 优化目标

智能体的优化目标是寻找最优策略 π^* ，以最大化长期累积期望奖励：

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t \cdot \mathcal{R}(s_t, a_t) \right], \quad (4.9)$$

其中 $\gamma \in (0, 1)$ 为折扣因子。优化目标需满足以下约束：

(1) **动作互斥性约束**。动作空间包含调度和迁移两种操作，每个网元 v 在时间步 t 只能选择一种动作。令 $a_t^{v,s}$ 表示网元 v 的调度动作， $a_t^{v,m}$ 表示迁移动作，则：

$$a_t^{v,s} + a_t^{v,m} \leq 1, \quad \forall v \in \{1, 2, \dots, N\}, \quad (4.10)$$

其中 $a_t^{v,s}, a_t^{v,m} \in \{0, 1\}$ ， N 为网元总数。

(2) **资源容量约束**。每个节点的 CPU、内存、网络带宽资源不得超载。令 C_k^i 为节点 i 的资源 k 容量， Util_k^i 为资源利用率，则：

$$(\text{Util}_k)_t^i \leq C_k^i, \quad \forall i \in \{1, 2, \dots, n\}, \quad \forall k \in \{\text{CPU}, \text{Mem}, \text{Width}\}, \quad (4.11)$$

其中资源利用率在执行动作 a_t 后更新。

4.3 算法设计

针对云原生 5G 核心网切片中网元的调度与迁移优化问题，本研究提出了一种名为 Deep Reinforcement Slice Scheduling (DRSS) 的智能调度算法。该算法以马尔可夫决策过程为理论基础，基于深度强化学习技术，旨在提升切片端到端时延、集群资源利用率，同时降低负载不均衡性。本节详细阐述 DRSS 的设计细节。

4.3.0.1 神经网络结构

传统 DQN^[66]因单一网络同时负责动作选择和 Q 值估计，易导致 Q 值高估问题^[66]，从而影响策略学习的稳定性。为解决这一问题，DRSS 采用了 Double DQN 结构^[67]，通过分离动作选择与目标 Q 值计算来缓解这一问题。具体而言，DRSS 使用两个独立的神经网络：一个用于动作选择的评估网络，另一个用于计算目标 Q 值的目标网络。这种设计有效降低了 Q 值高估的风险，提升了策略学习的稳定性。目标 Q 值 y_t 的计算公式如下：

$$y_t = r_t + \gamma \cdot Q_{\text{target}} \left(s_{t+1}, \arg \max_a Q_{\text{eval}}(s_{t+1}, a); \theta_{\text{target}} \right), \quad (4.12)$$

其中 r_t 为即时奖励, γ 为折扣因子, $Q_{\text{eval}}(s, a; \theta)$ 为当前策略网络估计的 Q 值, Q_{target} 为目标网络估计的 Q 值, θ 和 θ_{target} 分别为策略网络和目标网络的参数。Double DQN 通过当前网络选择最优动作, 目标网络计算对应 Q 值, 有效降低了 Q 值高估的风险, 提升了策略学习的稳定性。

为防止梯度爆炸问题, 我们引入梯度裁剪机制^[68], 在参数更新时限制梯度范数。当梯度范数超过阈值 $\delta = 1.0$ 时, 按如下方式裁剪:

$$\nabla_{\theta} \mathcal{L} \leftarrow \frac{\delta}{\|\nabla_{\theta} \mathcal{L}\|_2} \cdot \nabla_{\theta} \mathcal{L}, \quad \text{if } \|\nabla_{\theta} \mathcal{L}\|_2 > \delta, \quad (4.13)$$

其中 $\mathcal{L}$ 为损失函数, $\nabla_{\theta} \mathcal{L}$ 为梯度。这种方法有效保障了训练过程中的数值稳定性。

参数更新采用 Adam 优化器^[69], 初始学习率设为 1×10^{-4} 。Adam 通过一阶矩和二阶矩估计动态调整学习率, 能够适应非平稳策略学习任务, 提高收敛效率。

为平衡探索与利用, 我们采用 ϵ -greedy 策略进行动作选择:

$$a_t = \begin{cases} \text{random action,} & \text{with probability } \epsilon, \\ \arg \max_a Q(s_t, a; \theta), & \text{with probability } 1 - \epsilon, \end{cases} \quad (4.14)$$

其中 ϵ 初始值为 0.9, 并随训练轮次逐步衰减至 0.01。这种策略在训练初期鼓励探索新动作, 随着训练深入逐渐倾向于利用已学习的最优策略。

4.3.0.2 优化策略

算法通过最小化均方误差 (Mean Squared Error, MSE) 损失函数更新策略网络参数:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} [(y_t - Q_{\theta}(s_t, a_t))^2], \quad (4.15)$$

其中 $\mathcal{D}$ 为经验回放缓冲区, y_t 为目标 Q 值, $Q_{\theta}(s_t, a_t)$ 为当前网络估计的 Q 值。该损失函数驱动 DRSS 逼近真实 Q 值分布, 从而优化网元调度和迁移策略。

状态空间包含节点资源利用率、时延矩阵、网元状态等多个异构字段, 其量纲和数值范围差异显著, 可能导致某一维度因数值尺度较大而主导模型训练。为此, 我们引入层归一化 (Layer Normalization, LayerNorm) 对输入状态进行标准化。对于输入状态向量 $\mathbf{x} = [x_1, x_2, \dots, x_D]^{\top} \in \mathbb{R}^D$, LayerNorm 的计算过程如下:

$$\mu = \frac{1}{D} \sum_{i=1}^D x_i, \quad \sigma^2 = \frac{1}{D} \sum_{i=1}^D (x_i - \mu)^2, \quad (4.16)$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y_i = \gamma_i \cdot \hat{x}_i + \beta_i, \quad (4.17)$$

其中 μ 和 σ^2 分别为均值和方差, $\epsilon = 10^{-5}$ 为防止除零的小常数, γ_i 和 β_i 为可学习的缩放和平移参数。归一化后的状态向量 $\mathbf{y} = [y_1, y_2, \dots, y_D]^\top$ 被送入策略网络, 用于后续特征提取和 Q 值估计。LayerNorm 增强了模型对不同量纲输入的鲁棒性, 尤其适用于高维异构状态空间。

为提升训练过程的稳定性与效率, 本文通过经验回放缓冲区存储历史经验样本, 并从中随机采样批量数据, 打破时间相关性, 提高样本利用效率。缓冲区容量设为 10^4 , 批量大小为 256。这种机制有效缓解了强化学习中样本间的时序依赖问题, 提升了模型的泛化能力^[70]。

目标网络参数 θ_{target} 通过软更新方式逐步接近策略网络参数 θ :

$$\theta_{\text{target}} \leftarrow \tau\theta + (1 - \tau)\theta_{\text{target}}, \quad (4.18)$$

其中 $\tau = 0.005$ 为更新率。软更新通过平滑调整目标网络参数, 减少目标 Q 值的震荡, 从而提升训练稳定性^[71]。

4.3.1 智能体调度架构

图 4.2 展示了 DRSS 智能体嵌入切片调度控制流程中的具体架构。

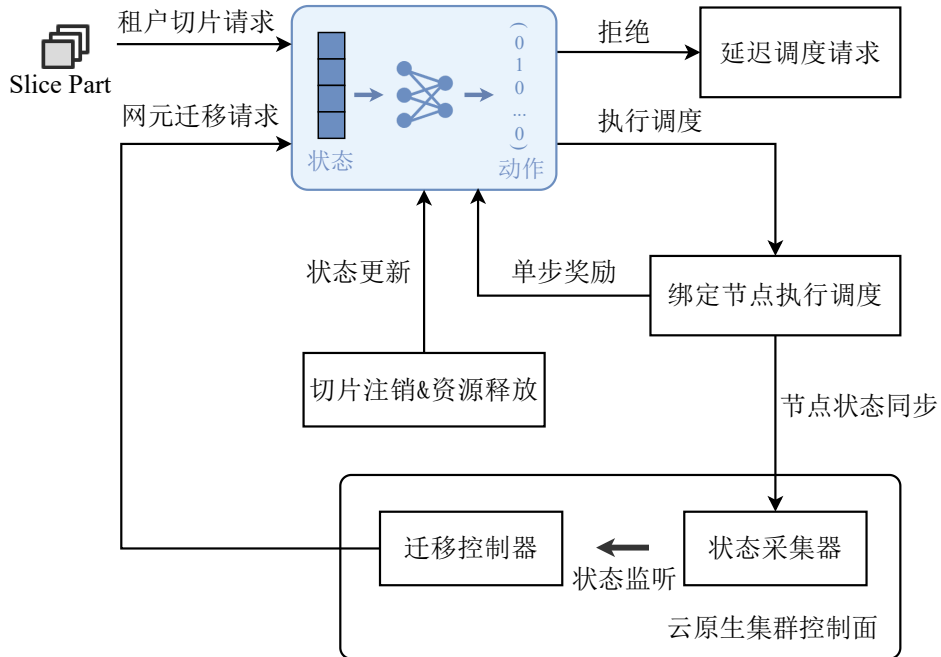


图 4.2 算法调度控制循环

调度过程首先由事件输入触发。事件包括两种类型：一是租户切片请求，即新切片到来时，系统接收切片启动请求，包含网元部署需求；二是网元迁移请求，在运行过程中，根据端到端时延或资源波动触发对某些网元的迁移需求。这些请求作为环境输入，送入 DRSS 决策单元进行处理。

决策单元作为智能体，负责状态感知和动作选择。在状态表示阶段，决策单元从集群控制面获取当前状态 $s_t = (\mathcal{S}_t, \mathcal{C}_t, \hat{\mathcal{L}}_t)$ 。在动作输出阶段，智能体通过 DQN 神经网络基于状态 s_t 映射到动作 $\mathcal{A}_t \in \{\text{node}_i \mid i = 0, 1, \dots, n-1\}$ 。动作选择存在两种可能：若资源不足或时延无法保障，则拒绝本次调度或迁移请求，进入延迟调度流程；若条件满足，则为网元选择目标节点 node_i ，并下发调度或迁移命令。动作类型由网元部署状态决定，若网元未部署则为调度；若已部署则为迁移，若目标节点与原节点相同则不迁移。

动作选择完成后，系统进入执行与监控阶段。调度或迁移指令首先下发至 Kubernetes API Server 执行网元绑定。若绑定失败，网元将被加入重调度队列，等待下一次调度，重调度间隔随失败次数增加。同时，状态采集器持续监听集群中各节点的运行状态，并将最新状态同步回决策单元作为后续决策的环境输入。对于迁移动作，迁移控制器协调容器运行时执行 Clone-Restore 操作，确保网元跨节点迁移的平滑性，并报告迁移进度和结果。

执行阶段结束后，系统进行奖励计算。每完成一次调度或迁移动作，仿真环境根据资源利用率均衡性、端到端时延满足度和迁移开销计算即时奖励，反馈给决策智能体，驱动 DQN 模型优化策略。资源利用率均衡性基于 $\mathcal{C}_t$ 的负载分布，时延满足度基于 $\mathcal{D}_t$ 。当切片生命周期结束时，系统释放切片占用的资源，并将最新状态推回决策单元，作为下一时间片的环境输入。

整个调度过程形成闭环控制，决策单元根据当前状态 s_t 选择动作 $\mathcal{A}_t$ ，云原生控制面通过状态采集器和迁移控制器执行调度或迁移，并持续回报运行状态。即刻奖励驱动智能体优化策略，提升长期调度性能，而状态更新通过状态转移和资源释放形成新的环境输入 s_{t+1} ，进入下一时间片。

4.4 实验评估

本节通过仿真实验评估所提的 Deep Reinforcement Slice Scheduling (DRSS) 算法在云原生 5G 核心网切片调度中的性能，重点对比其与 Kubernetes 原生调度器 (Kube-

scheduler) 和轮询调度器 (Round Robin, RR) 的表现, 以验证 DRSS 算法在不同负载场景下的端到端时延优化能力、负载均衡性和调度效率。

4.4.1 环境设定

4.4.1.1 测试环境

实验基于 VMware ESXi 7.0 构建了一个包含 9 台虚拟机的 Kubernetes 集群, 集群版本为 1.23.4。其中一台虚拟机作为控制节点, 除部署 Kubernetes 集群必需组件外, 还部署了第 3 章实现的热迁移控制器。其余 8 台虚拟机分为两组: 第一组包含 4 台工作节点, 部署了支持热迁移的改造版 containerd 组件, 用于网元调度; 第二组包含 4 台高配置工作节点, 专门用于部署 UERANSIM 生成 UE 测试流量, 不参与网元调度。节点间通过 VMware vSwitch 连接, 带宽限制为约 5MB/s, 模拟实际网络环境。集群节点配置如表 4.3 所示, 各节点资源配置不同, 反映了实际场景中的资源异构性。

表 4.3 集群节点配置

节点	CPU (Core)	内存 (GB)	带宽 (MB/s)	角色
控制节点	4	8	5.2	控制节点
工作节点 1	2	4	5.4	网元调度
工作节点 2	4	4	5.1	网元调度
工作节点 3	2	4	5.6	网元调度
工作节点 4	2	2	5.1	网元调度
测试节点 1	4	8	8.3	虚拟 UE
测试节点 2	4	8	8.2	虚拟 UE
测试节点 3	4	8	8.5	虚拟 UE
测试节点 4	4	8	8.4	虚拟 UE

4.4.1.2 工作负载

为贴近真实场景, 本实验在第 3 章测试场景的基础上, 引入了切片特性完整的 5G 核心网网元配置。由于中电某所内部测试网元的容器化改造成本较高, 我们采用 OpenAirInterface 开源项目 OAI-5G-CN^[72]进行测试。

通过 OAI 提供的内置信令负载 API 可以便捷地生成大规模虚拟 UE 注册/释放请求用于构建工作负载。图 4.3 展示了控制面信令流量的生成过程: 每个切片部署完成

后，通过 Kubernetes Job 和 Pod PostStart Hook 自动唤起一个 Job 任务以部署生成测试流量的 UERANSIM 实例；每个实例周期性地生成多个 UE 实例，与控制面网元完成交互过程。

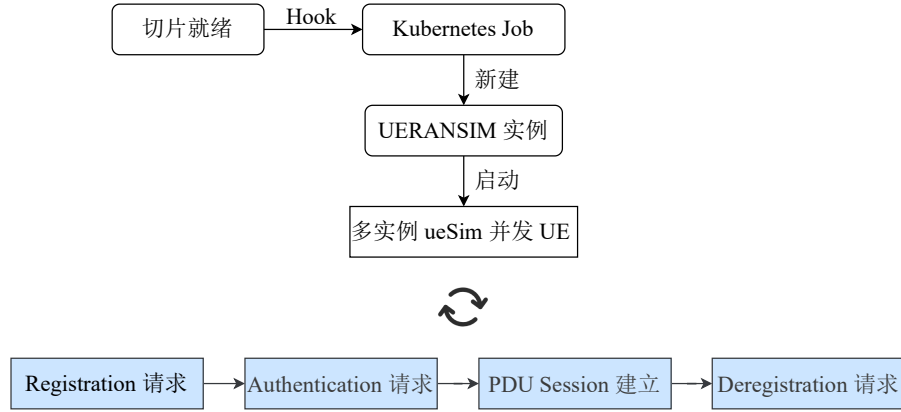


图 4.3 控制面信令负载生成过程

图 4.4 展示了 5 种网元（UDR、UDM、AUSF、AMF、SMF）在单台 2 核 4GB、带宽 5MB/s 虚拟机上承接不同 UE 负载（500UE/s、1000UE/s、1500UE/s）时的资源平均使用率。结果表明，随着 UE 负载增加，AMF 和 SMF 的 CPU 和带宽需求显著升高，而 UDR 和 UDM 的资源消耗较为平稳。

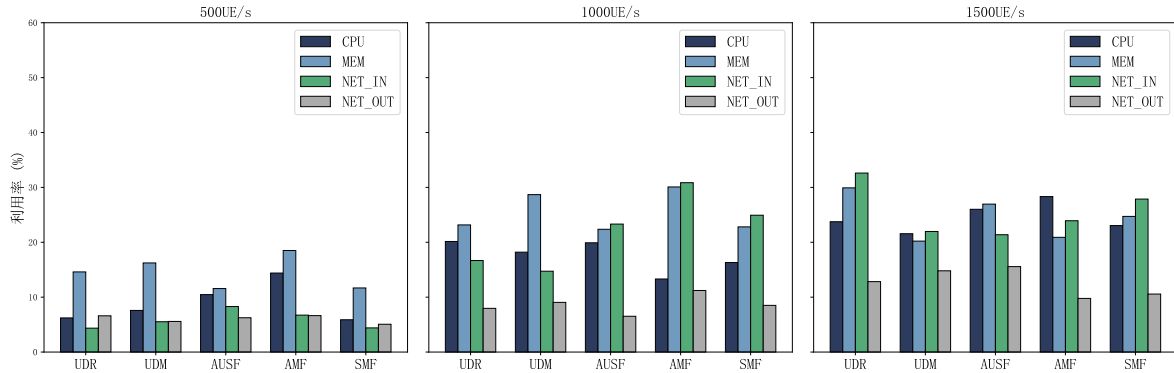


图 4.4 不同 UE 负载下网元资源消耗

基于上述资源消耗特性，实验构建动态工作负载以测试调度算法的性能。对于三种策略使用相同随机种子模拟多租户切片请求随机到达，调度请求以切片部分（Slice Part）为单位到达，切片间到达时间服从均值 20s、标准差 5s 的正态分布。每个切片部分包含 5 个网元独立部署网元，切片就绪后自动唤起一个 Job 任务，生成 800-1000UE/s 的信令流量，模拟负载的动态变化。

4.4.2 实验结果

实验比较了 DRSS 调度器与 Kubernetes 原生调度器和轮询两种基准策略的奖励值变化。实验部署 4000 个 Slice-Part 作为 2000 个 episodes，每 50 个 episodes 更新一次目标网络参数。图 4.5 展示了三种调度策略在动态负载场景下的奖励值变化，奖励值定义见公式 (4.8)。

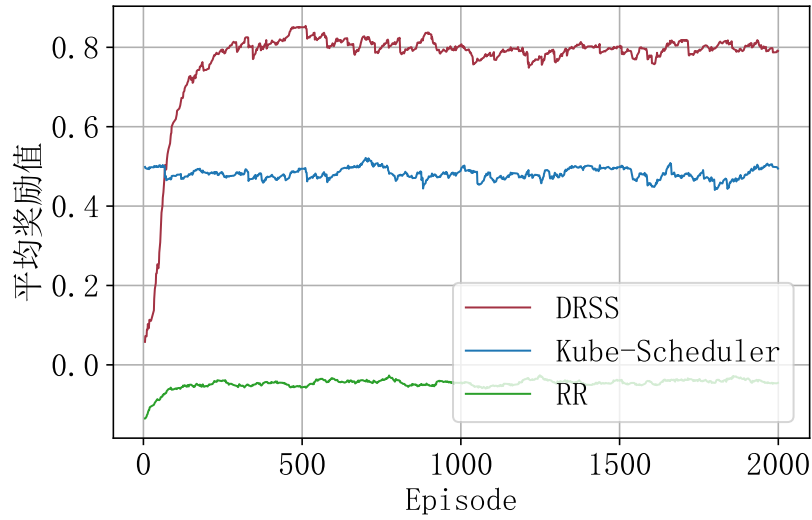


图 4.5 不同调度策略的奖励值对比

DRSS 在前 500 个 episodes 内，奖励值快速上升，表明 DQN 迅速学习到基础调度策略；在 800 至 2000 个 episodes 间，奖励值波动幅度逐渐减小，反映出 DRSS 平衡资源利用率与负载不均衡度的能力逐渐优化；最终奖励值稳定在约 0.8，表明策略已收敛至较优水平。Kube-Scheduler 基于静态规则无训练过程，因此奖励值波动较小，保持在 0.48 左右。Kube-Scheduler 的调度策略仅考虑 CPU 和内存资源的分配，难以优化综合成功率和多维度不均衡度的复合指标。RoundRobin 的奖励值则维持在 -0.2 左右，大幅低于其他两种策略，主要原因是其循环分配策略无法动态选择最优节点，节点选择时可能导致某些节点资源过载，而其他节点资源闲置，引发较大的不均衡度惩罚。

图 4.6 显示了动态工作负载下三种调度策略的平均资源利用率曲线。由于使用了相同的切片调度请求序列，三种策略平均资源利用率随时间变化的趋势大致相同，但 DRSS 策略的均值更高。DRSS 在前 500 个 episodes 内，资源利用率快速上升至 0.31，表明其快速学习到高效的资源分配策略；在 500 至 1000 个 episodes 间，利用率继续提升至 0.32，平均值约为 0.31，展现出较好的资源优化能力。Kube-Scheduler 的资源利用率波动较小，整体维持在 0.30 左右。其静态调度策略仅关注 CPU 和内存分配，难

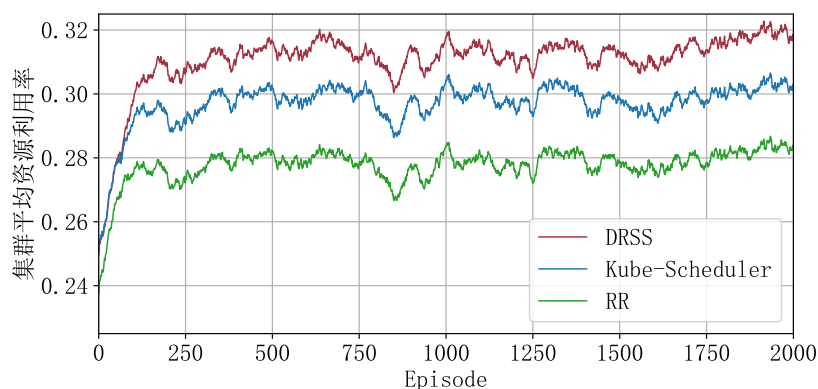


图 4.6 不同调度策略的平均资源利用率对比

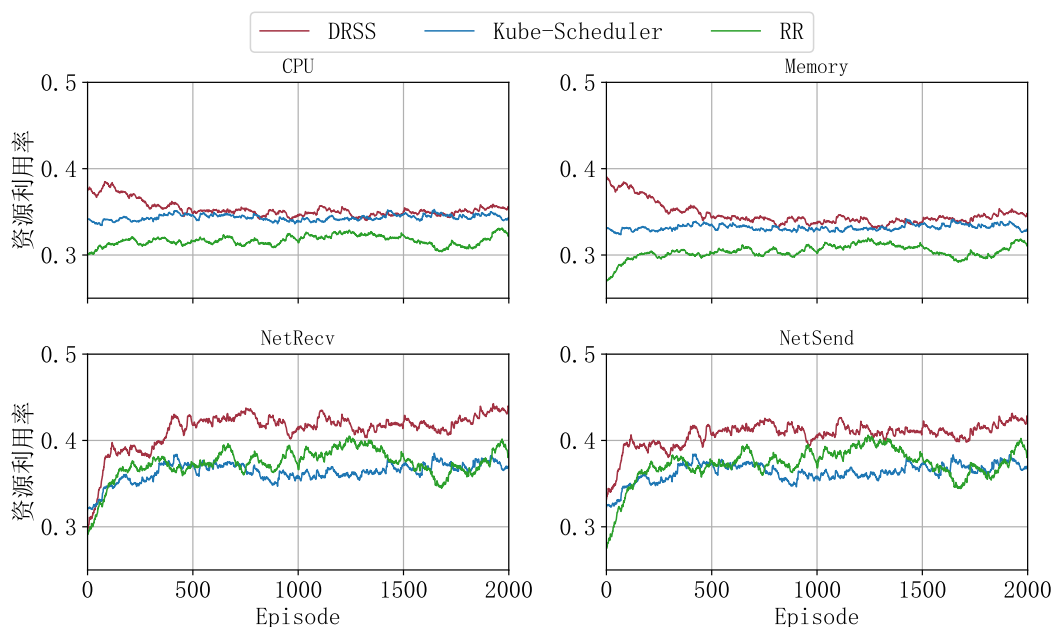


图 4.7 不同调度策略各资源利用率对比

以适应动态负载的波动，资源利用率提升有限。RoundRobin 的资源利用率最低，平均值约为 0.28。RR 的循环分配策略无法动态调整节点选择，导致资源分配不均，整体利用率较低。

图 4.7展示了三种调度策略在 CPU、Memory、NetRecv（网络接收）和 NetSend（网络发送）四种资源上的利用率。在 CPU 和 Memory 利用率方面，DRSS 和 Kube-Scheduler 表现相近，均波动在 0.30 至 0.35 之间，平均值分别为 0.38 和 0.36，而 RR 的利用率较低，平均值为 0.32。Kube-Scheduler 因其静态调度策略在 CPU 和 Memory 分配上稍占优势，但整体差异不大。在网络资源利用率（NetRecv 和 NetSend）方面，DRSS 表现出显著优势。DRSS 在前 500 个 episodes 内，NetRecv 和 NetSend 的利用率均快速上升至 0.43，之后稳定在 0.40 至 0.43 之间，平均值约为 0.42。相比之下，

Kube-Scheduler 的 NetRecv 和 NetSend 利用率波动在 0.35 至 0.40 之间；RR 的利用率更低且波动更大。

综合来看，DRSS 在动态工作负载场景下展现出更优的资源利用效率。其在平均资源利用率上优于 Kube-Scheduler 和 RR，尤其是在网络资源上的提升更为显著。这表明 DRSS 通过深度强化学习能够动态优化多维资源分配，适应动态负载的波动，特别是在网络密集型任务中表现突出，而在 CPU 和 Memory 利用率上与 Kube-Scheduler 相当，优于 RR。

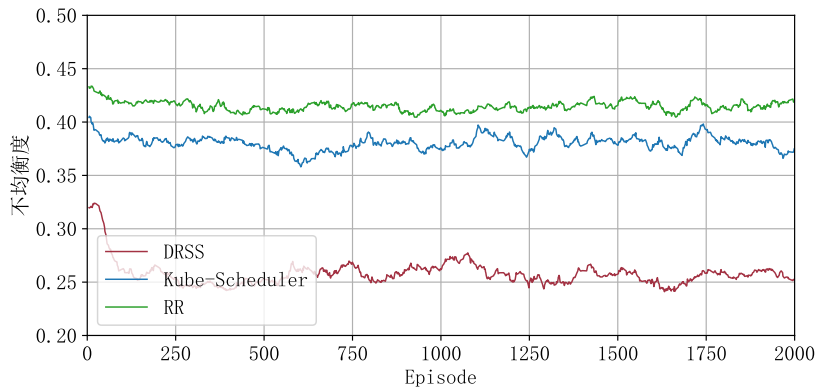


图 4.8 不同调度策略的不均衡度对比

图 4.8显示了动态工作负载下三种调度策略的不均衡度曲线。DRSS 在前 250 个 episodes 内，不均衡度从 0.32 迅速下降至 0.25，表明其快速学习到均衡的资源分配策略；在 250 至 2000 个 episodes 间，不均衡度进一步降低并稳定在 0.25 左右，波动较小，展现出优秀的资源均衡能力。Kube-Scheduler 的不均衡度波动较小，整体维持在 0.35 至 0.40 之间，平均值约为 0.38。其静态调度策略仅关注 CPU 和内存分配，未考虑网络 I/O 资源，且由于切片场景中网络 I/O 较 CPU 和内存波动较大，导致其负载不均衡度较高。RoundRobin 的不均衡度最高，稳定在 0.40 至 0.45 之间，平均值约为 0.42。RR 的循环分配策略无法动态调整节点选择，导致部分节点资源过载而其他节点闲置，资源分配不均问题较为严重。DRSS 在动态工作负载场景下展现出更低的资源不均衡度。通过深度强化学习，DRSS 动态优化调度策略，有效应对负载波动，显著提升资源分配的均衡性，优于 Kube-Scheduler 和 RR。

图 4.9显示了 DRSS 在不同参数配置下的表现。当 $\alpha = 0.5, \beta = 1, \delta = 1$ 时，平均资源利用率最高（约 30%），但不均衡度较高（约 28%），端到端时延也较长（约 8ms）。这表明在强调资源利用率（ $\beta = 1$ ）和不均衡度惩罚（ $\delta = 1$ ）时，DRSS 优先优化资源使用，但牺牲了时延性能。当 $\alpha = 1, \beta = 1, \delta = 0.5$ 时，端到端时延降低

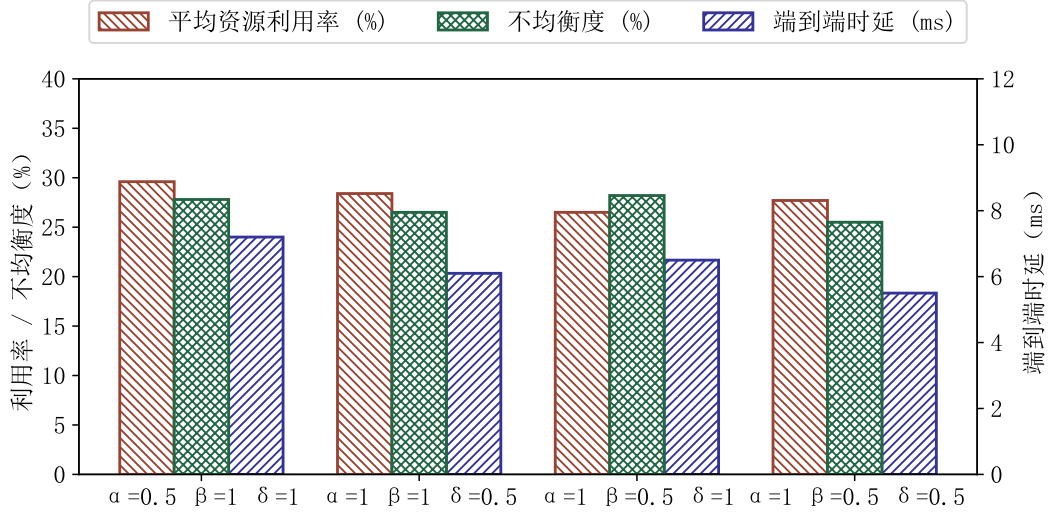


图 4.9 不同策略偏好下的表现

至约 6ms，不均衡度降至 25%，但资源利用率略减至 28%，反映了更高的 QoS 权重 ($\alpha = 1$) 改善了时延，同时较低的不均衡度惩罚 ($\delta = 0.5$) 减少了资源分配的不均。当 $\alpha = 1, \beta = 0.5, \delta = 1$ 时，端到端时延略有升高，不均衡度为 24%，但资源利用率降低至 27%。这表明降低资源利用率权重 ($\beta = 0.5$) 使 DRSS 更关注时延和均衡性，而非资源使用效率。最佳综合表现出现在 $\alpha = 1, \beta = 0.5, \delta = 0.5$ 时，端到端时延最低 (约 5.4ms)，不均衡度最低 (约 24%)，资源利用率为 26%。这表明在动态工作负载场景下，适当降低资源利用率和均衡度的权重，优先优化 QoS，可以实现更低的时延和更好的资源均衡性。在动态负载场景下，较高的 α 有助于显著降低端到端时延，而适当降低 β 和 δ 可以进一步改善不均衡度，但可能以牺牲部分资源利用率为代价。

4.4.2.1 小结

实验结果表明,DRSS 调度器在动态工作负载场景下表现出色,优于 Kube-Scheduler 和 RoundRobin (RR)。在奖励值方面, DRSS 快速收敛并显著优于 Kube-Scheduler 和 RR, 展现了其在资源分配和负载均衡优化上的高效性。不均衡度分析显示, DRSS 有效降低了资源分配不均, 远优于 Kube-Scheduler 和 RR, 体现了其在资源均衡性上的优势。资源利用率分析进一步验证了 DRSS 的优越性, 其平均资源利用率高于 Kube-Scheduler 和 RR, 尤其在网络资源上表现突出, 而在 CPU 和 Memory 利用率上与 Kube-Scheduler 相当, 均优于 RR。这表明 DRSS 在多维资源优化和网络密集型任务中具有显著优势, 为智能切片编排提供了有力支持。

4.5 本章总结

本章提出了基于深度强化学习的 Deep Reinforcement Slice Scheduling (DRSS) 智能编排算法，针对云原生 5G 核心网切片调度问题展开研究。我们以马尔可夫决策过程为理论基础，设计了 DRSS 算法，通过优化状态空间、动作空间和奖励函数，提升了调度效率，并引入 Double DQN 增强了算法性能。

实验评估基于真实环境搭建的仿真环境，对比了 DRSS 调度器与 Kube-Scheduler 和 RoundRobin 在动态工作负载场景下的性能。DRSS 展现了较强的适应能力，在资源分配、负载均衡和多维资源优化上均优于 Kube-Scheduler 和 RR，尤其在网络密集型任务中表现突出。通过调整奖励函数参数，DRSS 能够灵活适应不同优化目标，在高 QoS 权重配置下显著降低端到端时延并改善不均衡度，适合时延敏感场景；而在高资源利用率权重配置下优先优化资源使用，适合资源密集型场景。

相较于 Kube-Scheduler，DRSS 综合考虑了 CPU、内存和网络 I/O 等多维资源，避免了静态调度导致的资源分配不均；相较于 RR，DRSS 通过强化学习克服了循环调度的局限性，显著提升了资源利用效率和均衡性。总体而言，DRSS 智能编排算法在动态负载下的云原生 5G 核心网切片调度中展现了优越的性能和适应性，为智能切片编排提供了有力支持。

第 5 章 面向云原生 5G 核心网的智能调度器

5.1 引言

本文第 3 章提出了一种基于 C-R 方案的网元热迁移机制，通过增量快照传输实现了 Pod 级别的低开销迁移，为动态资源重配置提供了技术基础。第 4 章进一步针对 5G 核心网切片调度问题进行了形式化建模，揭示了 Kubernetes 原生调度器在动态 QoS 保障、网络拓扑感知以及服务连续性保障方面的不足，并基于优化的 DQN 算法设计了一种自适应的切片编排策略 Deep Reinforcement Slice Scheduling (DRSS)，显著提升了调度效率和资源利用率。本章将两部分工作嵌入云原生网络切片调度体系，为实现高效切片管理和动态优化提供技术支撑。

本章在前文工作的基础上，提出并实现了一种面向云原生 5G 核心网的智能调度器框架，旨在将迁移策略与热迁移能力深度融入网络切片调度体系，解决原生调度器在动态资源调度和网络感知方面的局限性。智能调度器通过 Kubernetes 扩展机制嵌入集群调度流程，集成了第 3 章的热迁移机制和第 4 章的 DRSS 优化算法，构建了一个集调度与迁移决策于一体的系统。系统设计上，调度器通过状态采集、策略决策和动作执行三大核心模块，实现了集群状态的实时感知、调度策略的智能生成以及调度指令的高效执行；在实验评估中，通过搭建支持热迁移的 Kubernetes 集群，验证了所提方案在资源利用率、负载均衡、调度延迟、信令处理效率以及响应稳定性等关键维度上的性能优势。

本章内容安排如下：第 5.2 节介绍智能调度器的系统架构以及调度器核心模块的设计与实现；第 5.3 节通过实验对所提方案进行性能评估；第 5.4 节对本章内容进行总结。

5.2 智能调度器设计

智能调度器的目标是实现 5G 核心网网元的高效调度与动态迁移，优化 QoS 满足率、资源利用率和负载不均衡度等关键指标。为此，本研究设计了一个模块化的调度架构，基于第 4 章提出的 DRSS 算法生成调度和迁移决策，并通过 Kubernetes 的扩展机制实现与集群的集成。本节将介绍智能调度器的总体架构以及关键技术实现。

5.2.1 总体架构

智能调度器以 Kubernetes 扩展调度器的形式实现，替换原生调度器，利用 DRSS 算法生成调度和迁移决策。智能调度器在第 3 章扩展了迁移功能的 Kubernetes 集群中部署，作为第二个调度器运行，遵循第 2.2.3 节介绍的 Kubernetes 调度流程。

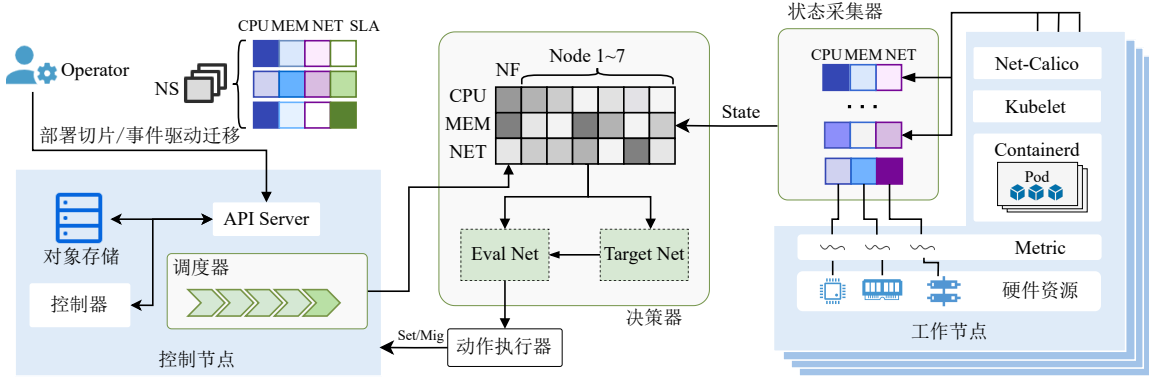


图 5.1 智能调度器架构

总体架构如图 5.1 所示。我们使用 Kubernetes 调度框架在拓展了迁移功能的集群中部署 DRSS 调度器作为辅助调度组件。主要包含以下 3 个关键模块：

- **状态采集模块：**运行在集群中每个工作节点上，负责从 Kubernetes 集群中实时采集状态信息，它持续监视和记录节点上 CPU、内存、网络 and 磁盘的使用情况，并在接收到状态获取请求时返回集群的当前状态。
- **DRSS 决策模块：**加载预训练的 DRSS 模型（策略网络 $\pi(a|s;\theta)$ ），结合状态采集模块提供的集群状态，响应 DRSS 调度器的请求，生成动作概率分布，并通过动作掩码选择最优动作。
- **动作执行模块：**该模块对接 Kubernetes API Server，负责将策略网络输出的动作转化为具体的集群控制指令。对于调度动作 a_i^{sched} ，通过调度绑定接口将待部署网元分配至节点 i ；对于迁移动作 a_j^{migr} ，则构造 PodMig 资源对象，触发第三章所实现的热迁移控制器完成迁移操作。

使用 DRSS 调度器完成切片部署的流程包括以下几个关键步骤：

首先，用户指定网络切片的 QoS 需求和各核心网网元配置，包括每个网元的容器镜像地址与对应的资源需求等，完成后向集群 API Server 提交部署切片的请求，请求被提交至 Kubernetes 主节点后将进入调度队列，等待 DRSS 调度器处理。

在调度周期的过滤阶段，调度器会根据当前待处理网元的资源需求，从集群中筛除掉不满足资源要求或存在调度冲突的节点。这一步骤确保后续评分仅在可行节点集合中进行，从而减少决策空间并提高调度效率。

进入评分阶段后，调度器将当前待调度网元的资源需求、当前可调度节点的状态集合、切片 QoS 需求及拓扑约束一并封装为调度请求，发送给 DRSS 决策器。决策器进一步通过节点探测接口与各节点上的状态采集模块交互，实时收集节点的当前资源利用率，并将这些状态指标与其余状态信息共同编码为一个归一化环境状态向量 s_t 。

在 DRSS 调度器中，策略网络 π_θ 接收该状态向量 s_t 并输出一个候选动作向量 a_t ，代表对目标节点的选择决策。该动作表征了推荐的初始调度或迁移目标节点索引。随后，该动作连同动作概率和价值估计结果一并返回给调度器。动作执行模块将该动作映射为真实节点，并基于选中节点的预期得分进行最终确认。如无冲突，调度器将网元绑定至选定节点，并通过 Kubernetes 控制平面触发实际部署过程。

DRSS 调度器将该动作映射为真实节点，并基于选中节点的预期得分进行最终确认。如无冲突，调度器将网元绑定至选定节点，并通过 Kubernetes 控制平面触发实际部署过程。当网元部署完成并启动成功后，该核心网网元实例正式上线，参与网络切片中的信令流程。整个过程随后会被记录为一个调度经验条目用于后续策略网络的增量训练与性能优化。

5.2.2 关键技术实现

本小节将详细描述状态采集模块、决策模块、动作执行模块的设计与实现细节。

5.2.2.1 状态采集模块

状态采集模块作为智能调度编排器的核心基础组件，部署于 Kubernetes 集群的每个工作节点，为 DRSS 决策模块提供实时、可靠的状态输入。

状态采集模块主要负责以下功能：首先，监控节点资源使用情况，包括 CPU 利用率、内存使用率和网络带宽；其次，测量节点间网络时延，构建时延矩阵；再次，采集本节点上的网元状态信息；最后，集成 QoS 指标，为切片性能评估提供依据。状态采集模块采用 FastAPI 框架构建 RESTful 服务接口，每个工作节点运行一个独立实例，通过多线程并行采集数据以提升效率。其核心功能包括以下几个方面：

(1) **节点资源监控**。通过 psutil 库实现对 CPU、内存和网络资源的实时监控，除 CPU 和内存资源使用率外，通过 `psutil.net_io_counters()` 统计单位时间内接

收和发送的字节数评估网络带宽。

(2) **节点间时延测量**。利用 `ping3` 库对集群中的其他节点执行 Ping 操作，测量往返时延 (RTT)，并构建时延矩阵 $L \in \mathbb{R}^{N \times N}$ ，以毫秒为单位存储，时延数据每 5 秒更新一次。

(3) **网元状态采集**。通过 Kubernetes Python Client 与 API Server 交互，获取网元数据，包括网元类型、资源请求、当前部署位置和 Clone 快照大小。

(4) **QoS 指标采集**。通过 Prometheus API 查询切片相关的 QoS 指标，并根据第 4.2.4 节的分级标准归一化，形成 s_t^{QoS} 。

(5) **数据接口**。模块提供 `/stats` 端点，以 JSON 格式返回包含节点名、时间戳、资源利用率、时延矩阵、网元状态和 QoS 指标的响应，供 DRSS 决策模块调用。

为确保采集效率和实时性，状态采集模块的采集频率设置为每秒一次，时延测量频率为每 5 秒一次，并通过 FastAPI^[73] 的异步接口实现低延迟响应。采集到的状态数据通过 gRPC 接口传输至 DRSS 决策模块，支持后续决策过程。状态采集模块的工作流程如图 5.2 所示。

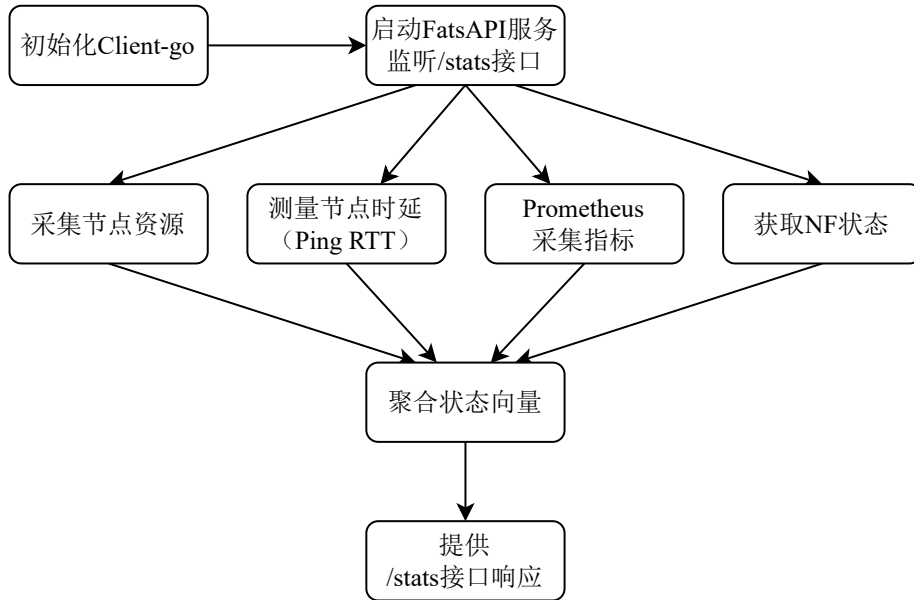


图 5.2 状态采集模块流程图

5.2.2.2 DRSS 决策模块

DRSS 决策模块是智能调度编排器的核心组件，其主要功能是基于第 4 章预训练的 DRSS 模型，结合状态采集模块提供的集群状态，生成调度和迁移决策，并为动作

执行模块提供指令。本模块通过高效的推理和动作选择机制，确保在动态的网络切片环境中实现优化的网元部署和迁移管理。

DRSS 决策模块的核心功能包括以下几个方面：首先，加载第 4 章中预训练的 DRSS 模型，对状态采集模块提供的归一化状态向量 $\tilde{\mathbf{s}}_t$ 进行推理，生成动作概率分布 $\pi(a|s_t; \theta)$ 。其次，通过动作掩码屏蔽无效动作，并从修正后的概率分布中选择概率最高的有效动作 a_t ，动作类型包括调度动作 a_i^{sched} 和迁移动作 a_j^{migr} 。最后，模块将生成的动作 a_t 通过 gRPC 接口传递至动作执行模块，同时记录状态-动作对 (s_t, a_t) ，供在线学习模块进行后续策略更新。

DRSS 决策模块利用 ONNX Runtime^[74]进行优化部署，以提升推理效率。动作选择过程通过动作掩码机制确保决策的有效性，例如屏蔽资源不足的节点或不可迁移的网元，从而提升决策的鲁棒性。具体工作流程如 4.3 节所述。

5.2.2.3 动作执行模块

动作执行模块负责将 DRSS 决策模块生成的动作 a_t 转换为实际的 Kubernetes 操作指令，并通过与 Kubernetes API Server 的交互实现网元的部署与迁移操作。动作执行使用 client-go 库与 Kubernetes API Server 交互，以独立服务形式运行，接收 DRSS 决策模块的 gRPC 请求并执行动作，功能主要包括以下几个方面：

(1) **动作接收**：通过 gRPC 接口接收 DRSS 决策模块下发的动作 a_t ，解析其类型（调度/迁移）及目标节点信息；

(2) **调度动作执行**：对于调度类动作 a_i^{sched} ，使用 Kubernetes 的 Binding API 将新 Pod 绑定至指定节点 i ；

(3) **迁移动作触发**：对于迁移动作 a_j^{migr} ，构造自定义资源对象 PodMig 实例（定义见第 3 章），提交至 API Server，由热迁移控制器监控并完成迁移操作；

错误处理与状态反馈：处理执行过程中可能出现的异常，返回执行状态并通过 gRPC 反馈集群状态 s_{t+1} ，支持奖励计算与策略更新。

动作执行模块的工作流程如图 5.3 所示。

5.3 实验评估

本节通过实验评估智能调度编排器的性能，验证其在动态切片场景下的调度和迁移效果，并与基线方法进行对比。评估指标包括资源利用率、负载不均衡度、调度延

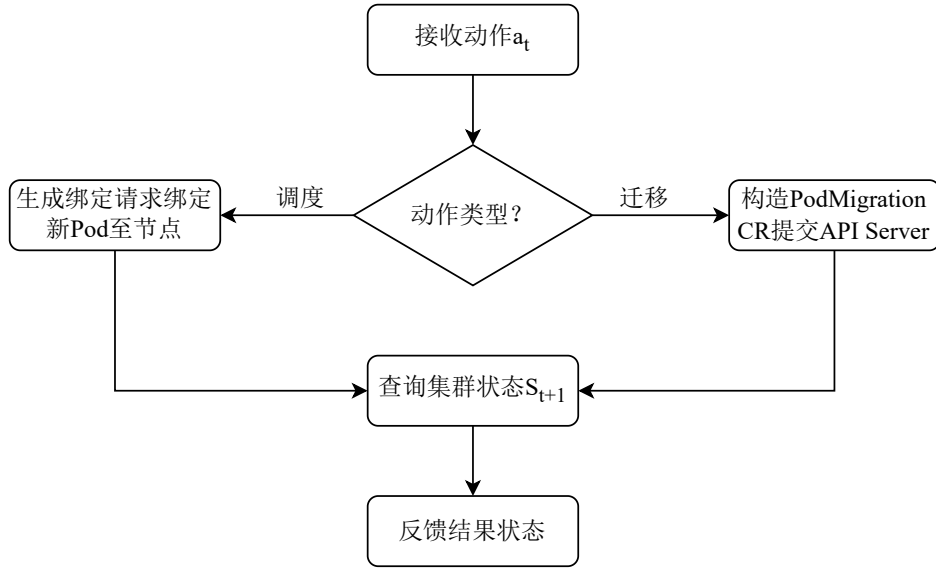


图 5.3 动作执行模块流程

迟等关键指标上的表现。以下将从实验设置、评估指标、实验结果等方面展开详细阐述。

5.3.1 实验设置

实验设置旨在为智能调度编排器的性能评估提供可控、可重复的环境，涵盖硬件配置、集群规模、实验参数和基线方法。实验设置的细节如下：

实验环境的配置主要包括以下几个方面：

(1) **硬件与集群环境**。实验基于一个由 1 个控制节点和 6 个工作节点组成的 Kubernetes 集群展开，其中控制节点负责运行 K8S 控制平面及智能调度编排器，工作节点用于部署切片网元，并在每个节点上运行状态采集模块。所有节点通过 VMware ESXi 创建，作为虚拟机运行于两台 Dell PowerEdge T640 服务器上，每台服务器配备 Intel Xeon Gold 6248R CPU、512GB 内存和 Broadcom NetXtreme E-Series Advanced 10GBASE-T Ethernet LOM。为体现资源异构性，集群中各节点的 CPU 核心数和内存大小有所不同，具体配置如表 5.1 所示。此外，DRSS 决策模块的推理任务运行于配备 NVIDIA RTX 4090 GPU 的独立服务器，通过 gRPC 与 K8S 集群通信；集群存储采用 Ceph 分布式存储系统，支持第 3 章提出的 C-R 方案的增量快照传输。

(2) **切片工作负载**。为全面评估所提基于深度强化学习的网元调度方法在多业务场景下的性能表现，本文构建了三种具有代表性的 5G 网络切片类型作为测试工作负载，分别对应 3GPP 标准定义的三大主要应用场景：增强型移动宽带（eMBB）、超可

表 5.1 实验集群节点配置

Node	Node Type	CPU	Memory	Bandwidth
Node0	Master	8× Intel Xeon Gold 6248R@3.00GHz	16GB	8MB/s
Node1	Worker	4× Intel Xeon Gold 6248R@3.00GHz	8GB	7MB/s
Node2	Worker	4× Intel Xeon Gold 6248R@3.00GHz	8GB	6MB/s
Node3	Worker	2× Intel Xeon Gold 6248R@3.00GHz	4GB	9MB/s
Node4	Worker	2× Intel Xeon Gold 6248R@3.00GHz	4GB	5MB/s
Node5	Worker	2× Intel Xeon Gold 6248R@3.00GHz	2GB	8MB/s
Node6	Worker	4× Intel Xeon Gold 6248R@3.00GHz	4GB	7MB/s

靠低时延通信（URLLC）与海量机器类通信（mMTC），每个切片段（Slice Part）实例包含所有五类独立部署核心网网元：UDR、UDM、AUSF、AMF、SMF，不同切片类型在网元资源请求、克隆迁移代价、QoS 目标等方面具有显著差异，以体现调度策略在多目标平衡中的适应能力。详细信息如表 5.2 所示。每次切片部署任务提交以切片为单位，但系统对切片中的单个网元逐一做出调度或迁移决策，决策过程考虑切片的整体 QoS 需求。切片部署就绪后 UERANSIM 构造 UE 和 gNB 的接入流量，与第四章 4.4 节图 4.3 所述方案一致。

（2）**调度负载**。为贴近真实场景中多租户切片部署请求场景，实验采用动态工作负载。三类切片调度请求以 1:2:2 的比例到达，切片间到达时间服从均值 10s，标准差 5s 的正态分布。网元以 Service 形式部署，共享网元通过预先配置的共享机制实现资源复用，所有工作节点预先拉取网元镜像。

5.3.2 评估指标

为全面评估智能调度编排器的性能，实验从多个维度定义了评估指标，与第 4 章的优化目标和奖励函数一致。评估指标的定义和计算方法如下：

实验的评估指标主要包括以下几个方面：

（1）**资源利用率**。该指标用于衡量调度器对集群资源的整体利用效率，通过对各节点的资源使用情况取平均计算得出。定义如下：

表 5.2 三类切片中各网元的资源请求与迁移代价

切片	网元	资源请求 (CPU, Mem, Net)	快照尺寸	依赖节点
URLLC	UDR	(0.3, 0.2, 0.1)	0.1	-
	UDM	(0.4, 0.3, 0.2)	0.2	UDR
	AUSF	(0.3, 0.3, 0.3)	0.2	UDM
	AMF	(0.6, 0.5, 0.4)	0.3	AUSF
	SMF	(0.5, 0.4, 0.5)	0.3	AMF
eMBB	UDR	(0.4, 0.3, 0.4)	0.2	-
	UDM	(0.5, 0.4, 0.5)	0.3	UDR
	AUSF	(0.4, 0.4, 0.4)	0.2	UDM
	AMF	(0.6, 0.5, 0.6)	0.4	AUSF
	SMF	(0.8, 0.6, 0.8)	0.5	AMF
mMTC	UDR	(0.3, 0.2, 0.2)	0.3	-
	UDM	(0.3, 0.3, 0.2)	0.3	UDR
	AUSF	(0.4, 0.3, 0.2)	0.3	UDM
	AMF	(0.4, 0.4, 0.3)	0.4	AUSF
	SMF	(0.4, 0.4, 0.3)	0.4	AMF

$$\text{AvgUtil}_t = \frac{1}{N} \sum_{i=1}^N \text{Util}_i \quad (5.1)$$

其中, Util_i 表示第 i 个节点的资源利用率, 计算方式为该节点上 CPU 利用率、内存使用率与网络带宽使用率的算术平均值。

(2) **负载不均衡度**。该指标用于衡量集群中各节点资源利用情况的离散程度, 通过计算各节点资源利用率与整体平均利用率之间的方差来刻画调度器对负载均衡的控制能力。定义如公式 (4.6) 所示。

其中, Util_i 表示第 i 个节点的资源利用率, AvgUtil_t 为时刻 t 所有节点资源利用率的平均值, 如公式 (5.1) 所示。该指标值越小, 说明集群资源负载越均衡。

(3) **调度延迟**。测量从调度请求发出到动作执行完成的时间 (ms), 包括 DRSS 决策延迟和动作执行延迟, 反映系统的实时性。

(4) **核心网控制面信令处理时延**。该指标聚焦控制面网元的信令处理效率，包含两个子指标：(a) **网元间 RTT**：测量典型网元对 AMF-SMF 的平均往返时延，反映调度决策对网元部署位置的优化效果；(b) **UE 注册时延**：通过使用 UERANSIM 模拟 1000 个 UE 执行 1000 次注册流程，统计时延分布，评估控制面整体信令处理效率。

(5) **响应稳定性**。包括以下子指标：(a) **延迟波动**：对 1000 次 UE 注册时延计算标准差，反映调度器在动态环境下的稳定性；(b) **最大/最小延迟**：统计 1000 次注册中的最大和最小时延，反映延迟波动范围。

5.3.3 实验结果与分析

5.3.3.1 资源利用率与负载不均衡度

资源利用率和负载不均衡度是衡量调度器资源分配效率和负载均衡能力的重要指标。根据 5.3.2 节的定义，计算所有节点的 CPU、内存和网络带宽的平均利用率作为资源利用率 ($AvgUtil_t$)；负载不均衡度 ($ImBalance_t$) 通过计算每种资源在每个节点上的利用率标准差求和。实验在 5 个 URLLC 切片的部署场景下进行，对比 DRSS 调度器和原生调度器 Kube-scheduler。图 5.4 展示了两种调度器的资源利用率和负载不均衡程度。

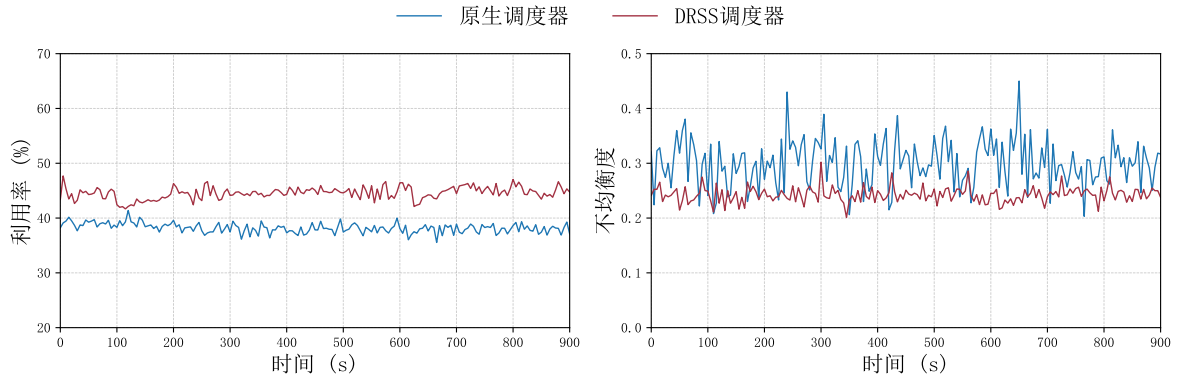


图 5.4 平均资源利用率与负载不均衡度

DRSS 调度器工作的资源利用率为 44.56%，而 Kube-scheduler 的资源利用率为 38.20%。DRSS 调度器的平均负载不均衡度为 0.042，远低于 Kube-scheduler 的 0.099。相比之下，DRSS 调度器的资源利用率高且较为平衡。DRSS 调度器通过动态感知集群状态，综合考虑节点 3 类资源状态和网元间的信令交互需求，将网元部署到资源利用更均衡的节点，能够更高效地利用集群资源，避免部分节点过载而其他节点资源闲置的情况。Kube-scheduler 在调度决策时仅考虑 CPU 和内存余量资源，依赖静态的资

源余量规则，对 CPU 密集型服务能表现出较好的资源利用率和均衡度，但对于高网络 IO 的服务则容易导致负载不均衡。

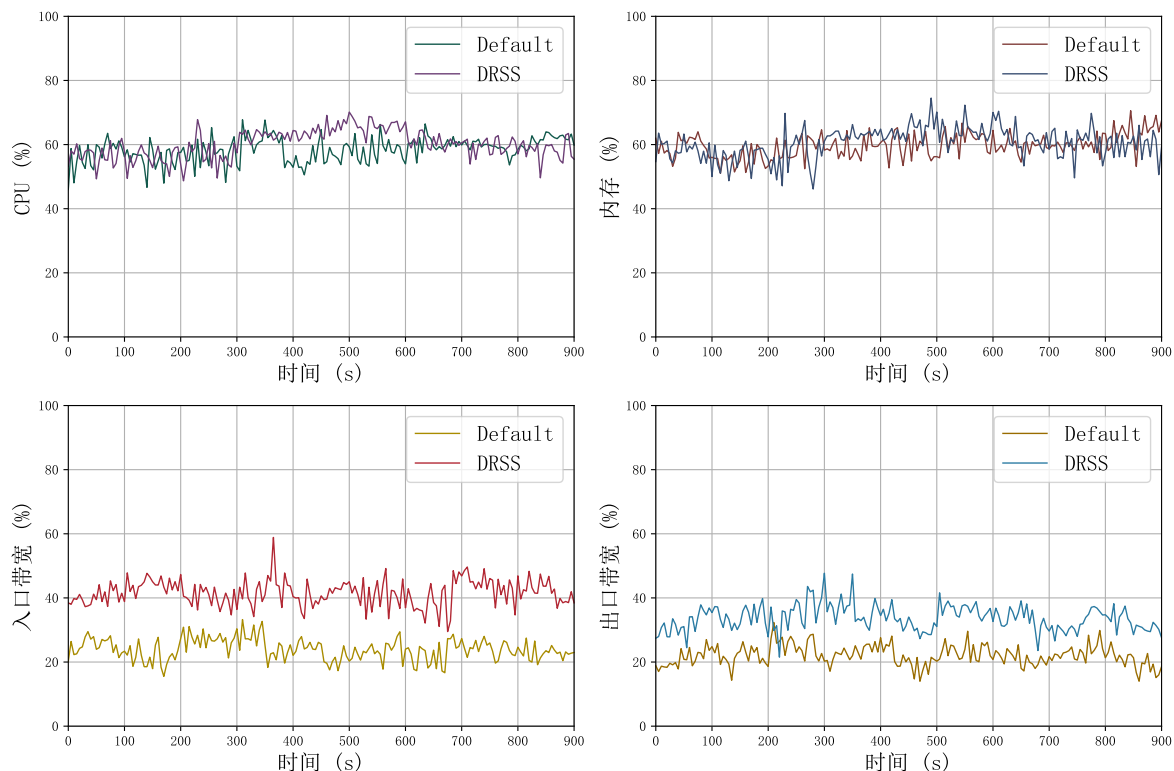


图 5.5 资源利用率

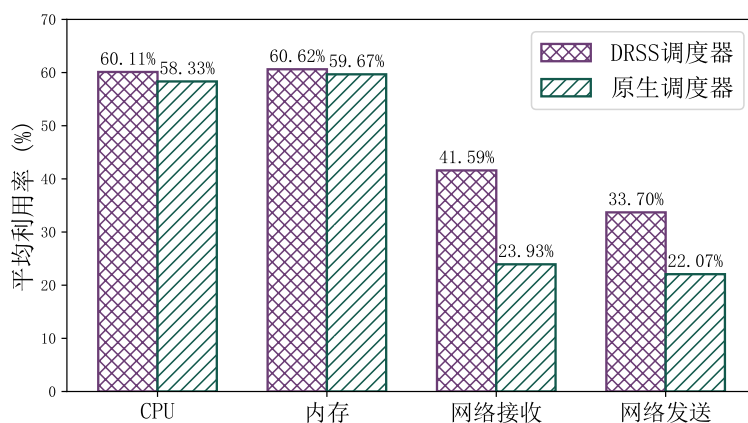


图 5.6 平均资源利用率对比

图5.5详细展示了两种调度器在动态调度负载下各类资源的利用率随时间变化情况。图 5.6展示了测量周期内各资源使用率的平均值，使用 DRSS 调度器的集群 CPU 和内存平均利用率分别为 60.11% 和 60.62%，网络接收带宽占用率和网络发送带宽占用率分别为 41.59% 和 33.70%。使用原生调度器的集群 CPU 和内存利用率分别为 58.33% 和 59.67%，网络接收带宽占用率和网络发送带宽占用率分别为 23.93% 和 22.07%。

为 58.33% 和 59.67%，网络接收带宽占用率和网络发送带宽占用率分别为 23.93% 和 22.07%。

5.3.3.2 调度时延分析

为评估智能调度编排器的实时性，实验首先对比了 DRSS 调度器与原生调度器在调度延迟上的表现。调度延迟根据前文定义，分解为决策延迟（DRSS 模型推理或原生调度器的评分时间）、通信延迟和网元启动延迟。实验在不同调度器调度 5 个 URLLC 切片部署场景下进行，采用动态请求模式，以突出调度延迟的差异。实验结果如表 5.3 所示。

表 5.3 调度延迟对比

调度器	类型	时延 (ms)		百分比 (%)
DRSS 调度器	调度延迟	13608		100.00
	决策延迟	77		0.56
	通信延迟	51		0.37
	网元启动延迟	13480		99.06
原生调度器	调度延迟	13549		100.00
	决策延迟	23		0.17
	通信延迟	36		0.27
	网元启动延迟	13490		99.56

实验结果表明，DRSS 调度器和原生调度器的总调度延迟较为接近，差距仅为 1%。具体来说：

（1）**决策延迟**。DRSS 调度器的决策延迟为 77ms，占总延迟的 0.56%，远高于原生调度器的 23ms（占 0.17%）。这一差异主要源于 DRSS 模型的推理开销，DRSS 调度器需通过神经网络推理生成动作概率分布，而原生调度器仅需执行简单的过滤和评分逻辑。

（2）**通信延迟**。DRSS 调度器的通信延迟为 51ms（占 0.37%），高于原生调度器的 36ms（占 0.27%）。这是因为 DRSS 调度器涉及模块间通信，尤其是状态采集模块与 DRSS 决策模块之间的 gRPC 请求，而原生调度器直接运行于 K8S 控制平面，通信开销较小。

(3) **网元启动延迟**。两者的网元启动延迟相近，DRSS 调度器为 13480ms（占 98.5%），原生调度器为 13490ms（占 99.6%），表明网元启动延迟是调度延迟的主要组成部分，且与调度器类型无关，主要受 K8S Pod 创建和启动过程的影响。

DRSS 调度器在决策延迟和通信延迟上的额外开销是其总延迟略高的主要原因，但总体调度延迟差距在可接受范围内，DRSS 调度器以可忽略的延迟开销提高了集群的资源利用率。

5.3.3.3 控制面性能分析

控制面信令处理时延用于衡量调度器在动态网络切片场景下性能，直接反映调度决策对网元部署的优化效果。测试方式如 5.3.2 节所述，图 5.7 展示了两种调度器下网元对 AMF 和 SMF 之间信令传输时延以及两种方式部署后使用 UERANSIM 模拟 1000 个 UE 注册请求的时延核密度估计。

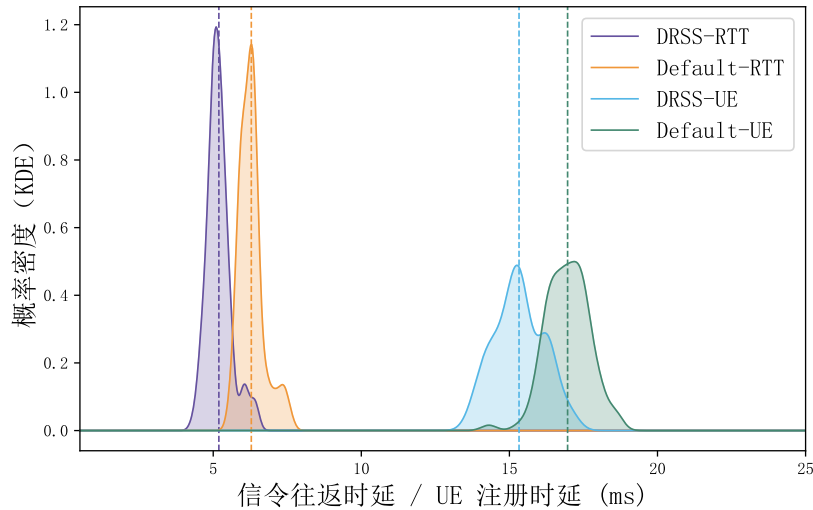


图 5.7 AMF-SMF 网元 RTT 和 UE 注册时延的核密度估计

DRSS 调度器的平均网元间 RTT 为 5.19ms，相比 Kube-scheduler 的 6.29ms 低 17.49%，DRSS 调度器在控制面信令处理时延上显著优于基线方法，这一优势源于 DRSS 调度器通过更加全面的状态感和决策依据优化网元部署决策，将交互频繁的网元对部署在低时延的节点对上，而 Kube-scheduler 仅基于静态规则和 CPU、内存两项资源余量选择节点。DRSS 调度器的平均 UE 注册时延为 15.32ms，相比 Kube-scheduler 的 16.96ms 低 9.67%。UE 注册时延的优化得益于 DRSS 调度器对控制面网元整体部署的优化，通过调度和迁移降低网元间 RTT，DRSS 调度器减少了信令交互的累计延迟，而 Kube-scheduler 在高负载下因缺乏动态适应性，导致时延显著增加。

5.3.3.4 响应稳定性分析

响应稳定性是衡量调度器在动态信令负载下维持控制面网元性能一致性的关键指标，尤其对于 URLLC 切片的高可靠性需求至关重要。根据 5.4 节的定义，响应稳定性包括延迟波动、最大延迟和最小延迟。实验设置如 5.3.2 节所述，实验结果如图 5.8 所示。

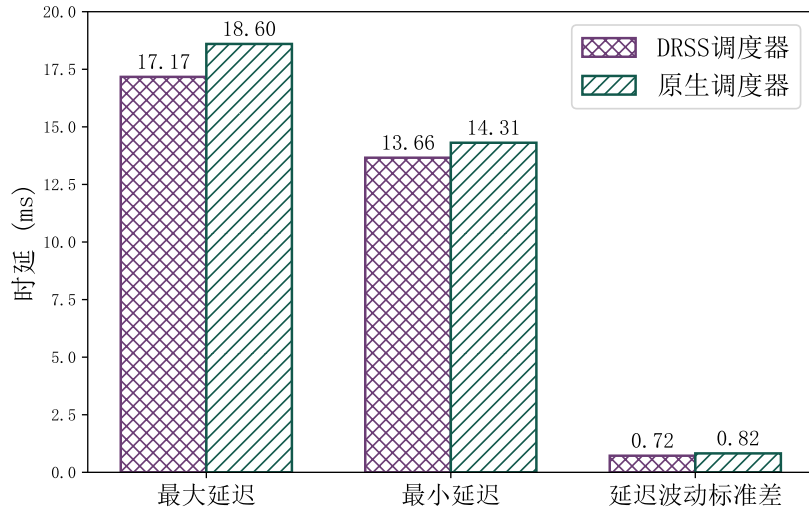


图 5.8 响应稳定性对比

从表中可以看出，DRSS 调度器在响应稳定性上表现出优势，DRSS 调度器的标准差延迟波动仅为 0.721ms，相比 Kube-scheduler 的 0.822 低 12.29%，最大延迟和最小延迟分别为 13.66ms 和 17.17，波动范围 3.51ms，小于 Kube-scheduler 的 4.29ms，这表明 DRSS 调度器能够更稳定地维持 UE 注册时延。与更优秀的 UE 注册时延类似，DRSS 调度器通过更加丰富的状态感知和重配置机制提供了更大的调度灵活性。DRSS 调度器通过实时采集集群状态和在线更新策略网络，能够主动调整网元重配置，缓解高负载下的资源竞争和网络拥塞。相比之下，Kube-scheduler 依赖静态规则，无法适应动态信令负载，导致延迟波动较大。

5.4 本章总结

本章围绕云原生 5G 核心网控制面的高效调度问题，设计并实现了一种基于深度强化学习的智能调度编排器。针对原生 Kubernetes 调度器在资源动态调度、网络拓扑感知以及服务连续性保障方面存在的不足，本研究基于前文提出的网元热迁移机制和 DRSS 优化算法，构建了一个集调度与迁移决策于一体的调度系统。

在系统设计部分，智能调度器通过 **Kubernetes** 扩展机制嵌入集群调度流程，整体架构包括状态采集、策略决策与动作执行三个核心模块。其中，状态采集模块实现了集群资源使用情况、网元运行状态及网络时延的实时采集，并构建状态向量输入策略模型；决策模块利用预训练的 **DRSS** 网络生成调度与迁移动作，并通过动作掩码机制保证决策的有效性；动作执行模块负责将调度结果转换为 **Kubernetes** 操作指令，完成实际的部署或迁移操作。系统通过模块解耦、接口标准化的方式实现了良好的可扩展性与工程落地能力。

在实验评估部分，搭建了拓展热迁移能力的 **Kubernetes** 集群，模拟部署 5G 切片任务，围绕资源利用率、负载均衡、调度延迟、信令处理效率与响应稳定性五个维度开展性能对比。实验结果表明，与原生调度器相比，智能调度器能够更充分利用系统资源，显著降低节点负载不均衡问题，同时优化关键路径信令时延与 UE 注册的整体延迟表现，且具备更高的时延稳定性，能够更好地适应多变业务负载与集群状态。

结论

本文针对云原生 5G 核心网控制面运维中的关键挑战，系统研究了有状态网络功能的热迁移机制与网络切片的智能调度策略。研究旨在提升控制面服务的高可用性与部署灵活性，同时优化网络资源利用效率与用户侧关键路径时延。

本研究的主要创新点包括：

(1) **基于 Clone-Restore 的网元热迁移方案**：本文提出了一种基于 Clone-Restore 机制的控制面网元热迁移方案，通过拓展容器运行时接口与实现迁移控制器，实现了 Pod 运行时状态与非容器状态的增量保存和跨节点传输恢复。得益于增量状态保存机制，该方案在确保迁移状态一致性的同时，规避了常规服务冷启动的时间消耗，显著降低了服务变更的停机时间。该方案通过完整克隆和恢复包括用户认证会话、安全上下文在内的关键状态，确保了 AMF 等执行核心安全功能的网元在迁移过程中的连续性，有效降低了因状态丢失可能引发的安全风险，增强了控制面服务的连续性与安全韧性。

(2) **DRSS 智能调度算法**：针对原生 Kubernetes 调度器在网络状态感知、时延优化及切片安全隔离方面的不足，本文提出了一种深度强化学习切片调度（Deep Reinforcement Slice Scheduling, DRSS）算法。该算法以控制路径时延最小化与集群资源负载均衡为联合优化目标，通过多维状态感知与在线自适应学习，动态优化调度策略。DRSS 不仅提升了调度灵活性与系统负载均衡能力，还通过对资源的精细化管理，为保障网络切片间的安全隔离奠定了坚实基础。

(3) **云原生智能调度器原型系统**：本文设计实现了面向 5G 核心网切片调度场景的云原生智能调度器原型系统，集成状态采集、策略决策与动作执行关键功能，为云原生赋能核心网运维提供完整高效的解决方案。通过真机模拟实验证明了智能调度器在控制路径延迟、节点资源利用率与 UE 注册稳定性等关键指标上优于默认调度器，验证了其在多切片、动态负载场景下的实际效能与工程价值。

综上所述，本研究提出的热迁移机制与智能调度框架，共同构成了面向云原生 5G 核心网控制面运维的创新解决方案。该方案不仅展现出良好的工程实践价值与显著的性能提升，还有效增强了网络切片控制面的弹性、安全与智能化管理水平，为构建高可靠、高性能的下一代 5G 运维体系提供了关键技术支撑，并对推动云原生技术与 5G

核心网控制面深度融合具有积极的促进作用。

尽管本研究在上述方面取得了阶段性成果，但仍存在一些局限性，为未来的研究指明了方向：

（1）目前热迁移机制主要面向中小规模集群进行设计和验证，在超大规模集群及多 NF 并发迁移场景下的性能与可扩展性有待进一步验证与优化，且迁移过程仍然存在可感知的停机时间。此外，对于迁移过程中状态数据的加密传输和完整性校验等安全加固措施尚未深入研究。后续研究可参考现有虚拟机迁移领域前沿技术，进一步优化增量状态保存策略与状态压缩机制，提升迁移效率与并发调度能力，降低停机时间，并集成更强的迁移过程安全防护机制。

（2）智能调度器采用了深度强化学习模型，DRSS 算法虽然提升了调度质量，但其较高的计算复杂度导致决策时延相对较长，在资源受限的边缘节点或大规模集群中的实时性与可扩展性需进一步评估。后续研究可探索模型剪枝、量化或知识蒸馏等轻量化技术，或结合传统启发式算法以平衡调度性能与计算开销。

（3）核心网运维场景复杂，影响因素多，本文提出的调度系统聚焦于网元信令时延和集群资源负载均衡优化，缺乏对能耗、显式安全策略执行、可靠性等指标的优化探索，特别是在网络切片这种典型的多租户环境下，信令安全、访问控制和策略合规性值得重点考虑。后续研究可探索将安全机制与调度流程深度融合，确保切片业务隔离与数据保护，构建更为全面的 5G 核心网安全可信运维体系。

总之，本研究为云原生 5G 核心网的高效、安全与智能运维提供了有价值的理论方法与技术实践，期待未来的研究能在此基础上取得更大突破，共同推动下一代网络基础设施的持续演进与创新。

参考文献

- [1] Al-Falahy N, Alani O Y. Technologies for 5G networks: Challenges and opportunities[J]. *It Professional*, 2017, 19(1): 12-20.
- [2] 3rd Generation Partnership Project (3GPP). TS 23.503 V17.10.0: Policy and Charging Control Framework for the 5G System (5GS); Stage 2[R]. Technical Specification TS 23.503 V17.10.0. Available at <https://www.3gpp.org/dynareport/23503.htm>. France: 3GPP, 2023: 12-13.
- [3] Kaur K, Mangat V, Kumar K. A review on Virtualized Infrastructure Managers with management and orchestration features in NFV architecture[J]. *Computer Networks*, 2022, 217: 81-100.
- [4] Sharma S, Miller R, Francini A. A cloud-native approach to 5G network slicing[J]. *IEEE Communications Magazine*, 2017, 55(8): 120-127.
- [5] Ahmad I, Shahabuddin S, Kumar T, et al. Security for 5G and beyond[J]. *IEEE Communications Surveys & Tutorials*, 2019, 21(4): 3682-3722.
- [6] Ericsson. Your quick guide to Network Functions in 5G[EB/OL]. 2019 [2025-04-23]. <https://www.ericsson.com/en/core-network/5g-core-network-functions>.
- [7] Nascimento B, Santos R, Henriques J, et al. Availability, scalability, and security in the migration from container-based to cloud-native applications[J]. *Computers*, 2024, 13(8): 192.
- [8] Nguyen K, Zhe-Tao L, Sekiya H. Virtualization for flexibility and network-aware on 5G mobile devices [C]. 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC): vol. 1. 2019: 928-929.
- [9] Arouk O, Nikaein N. Kube5g: A cloud-native 5g service platform[C]. *GLOBECOM 2020-2020 IEEE Global Communications Conference*. 2020: 1-6.
- [10] Barakabitze A A, Ahmad A, Mijumbi R, et al. 5G network slicing using SDN and NFV: A survey of taxonomy, architectures and future challenges[J]. *Computer Networks*, 2020, 167: 106984.
- [11] Luong D H, Outtagarts A, Ghamri-Doudane Y. Multi-level Resource Scheduling for network slicing toward 5G[C]. 2019 10th International Conference on Networks of the Future (NoF). 2019: 25-31.
- [12] Chen H M, Chen S Y, Wang S K, et al. Designing a reinforcement learning approach for the NFV orchestration system with energy saving optimization[C]. 2022 8th International Conference on Applied System Innovation (ICASI). 2022: 98-101.
- [13] Gifre L, Manso C, Casellas R, et al. Experimental demonstration of end-to-end NFV orchestration on top of the ADRENALINE testbed[C]. 2022 IEEE 8th International Conference on Network Softwarization (NetSoft). 2022: 281-283.
- [14] Martínez R, Vettori L, Baranda J, et al. Resource Abstractions in NFV Management and Orchestra-

- tion: Experimental Evaluation[J]. IEEE Transactions on Network and Service Management, 2022, 20(1): 608-624.
- [15] Kim H G, Yoo J H, Hong J W K. AI-based Network Function Virtualization Orchestration[C]. NOMS 2024-2024 IEEE Network Operations and Management Symposium. 2024: 1-5.
- [16] Zu J, Hu G, Yan J, et al. 网络功能虚拟化下服务功能链的资源管理研究综述[J]. 计算机研究与发展, 2021, 58(1): 137-150.
- [17] Liu Z, Shu Z, Chen S, et al. Low-latency Virtual Network function Scheduling Algorithm Based on Deep Reinforcement Learning[J]. Computer Networks, 2024, 246: 110418.
- [18] Zhu L, Huang K, Fu K, et al. A priority-aware scheduling framework for heterogeneous workloads in container-based cloud[J]. Applied Intelligence, 2022, 53(12): 152-168.
- [19] Huang X, Bian S, Gao X, et al. Online VNF Chaining and Predictive Scheduling: Optimality and Trade-Offs[J]. IEEE/ACM Transactions on Networking, 2021, 29(4): 1867-1880.
- [20] Thien P D, Wu F, Bekhit M, et al. Optimizing Placement and Scheduling for VNF by a Multi-objective Optimization Genetic Algorithm[J]. International Journal of Computational Intelligence Systems, 2024, 17: 43.
- [21] Jia J, Yang L, Cao J. Reliability-aware Dynamic Service Chain Scheduling in 5G Networks based on Reinforcement Learning[C]. IEEE INFOCOM 2021. 2021: 1-10.
- [22] Yao J, Wang J, Wang C, et al. DRL-Based VNF Cooperative Scheduling Framework With Priority-Weighted Delay[J]. IEEE Transactions on Mobile Computing, 2024, 23(12): 11375-11388.
- [23] Gharbaoui M, Martini B, Cecchetti G, et al. Resource Orchestration Strategies With Retrials for Latency-Sensitive Network Slicing Over Distributed Telco Clouds[J]. IEEE Access, 2021, 9(12): 132801.
- [24] Han B, Sciancalepore V, Costa-Perez X, et al. Multiservice-Based Network Slicing Orchestration with Impatient Tenants[J]. IEEE Transactions on Wireless Communications, 2020, 19(4): 2515-2529.
- [25] Moreira R, Silva F d O, Carvalho T C M d B, et al. Intelligent Data-Driven Architectural Features Orchestration for Network Slicing[C]. International Workshop on ADVANCES in ICT Infrastructures and Services (ADVANCE). Hanoi, Vietnam, 2024: 378-392.
- [26] Kovacevic I, Shafigh A S, Glisic S, et al. Multi-Domain Network Slicing With Latency Equalization [J]. IEEE Transactions on Network and Service Management, 2020, 17(4): 2182-2196.
- [27] Salhab N, Langar R, Rahim R. 5G network slices resource orchestration using Machine Learning techniques[J]. Computer Networks, 2021, 188: 107-129.
- [28] Mijumbi R, Serrat J, Gorricho J L, et al. Network Function Virtualization: State-of-the-Art and Research Challenges[J]. IEEE Communications Surveys & Tutorials, 2016, 18(1): 236-262.
- [29] Zakariya Saleh Z, Abbod M F, Nilavalan R. Intelligent Resource Allocation via Hybrid Reinforce-

- ment Learning in 5G Network Slicing[J]. IEEE Access, 2025, 13: 47440-47458.
- [30] Haris R M, Khan K M, Nhlabatsi A. Live migration of virtual machine memory content in networked systems[J]. Computer Networks, 2022, 209: 108898.
- [31] Vagin A. CRIU — criu.org[Z]. https://criu.org/Main_Page. [Accessed 20-04-2025].
- [32] Biswas M I, Parr G, McClean S, et al. A Practical Evaluation in Openstack Live Migration of VMs Using 10Gb/s Interfaces[C]. 2016 IEEE Symposium on Service-Oriented System Engineering (SOSE): vol. 45: 12. 2016: 346-351.
- [33] Noshay M, Ibrahim A, Ali H A. Optimization of live virtual machine migration in cloud computing: A survey and future directions[J]. Journal of Network and Computer Applications, 2018, 110: 1-10.
- [34] Stoyanov R, Kollingbaum M J. Efficient live migration of linux containers[C]. High Performance Computing: ISC High Performance 2018 International Workshops, Frankfurt/Main, Germany, June 28, 2018, Revised Selected Papers 33. 2018: 184-193.
- [35] Poggiani L, Puliafito C, Viridis A, et al. Live Migration of Multi-Container Kubernetes Pods in Multi-Cluster Serverless Edge Systems[C]. Proceedings of the 1st Workshop on Serverless at the Edge. 2024: 9-16.
- [36] Iorio M, Risso F, Palesandro A, et al. Computing Without Borders: The Way Towards Liquid Computing[J]. IEEE Transactions on Cloud Computing, 2023, 11(3): 2820-2838.
- [37] Ahmad M, Nawazish S M A, Tariq M T, et al. Neutrino: A Fast and Consistent Edge-Based Cellular Control Plane[J]. IEEE/ACM Transactions on Networking, 2022: 1-16.
- [38] Ahmad M, Nawazish S M A, Tariq M T, et al. Enabling Emerging Edge Applications Through a 5G Control Plane Intervention[C]. Proc. ACM CoNEXT 2022. 2022: 333-347.
- [39] CNCF Artificial Intelligence Working Group. Cloud Native Artificial Intelligence Whitepaper[Z]. <https://github.com/cncf/tag-app-delivery/blob/main/whitepapers/cloud-native-ai-whitepaper.md>. Cloud Native Computing Foundation. Accessed: 2025-04-21. 2023.
- [40] Cisco. Cisco Annual Internet Report (2018–2023)[R]. White Paper. San Jose, CA: Cisco Systems, 2020.
- [41] Kratzke N, Quint P C. Understanding cloud-native applications after 10 years of cloud computing-a systematic mapping study[J]. Journal of Systems and Software, 2017, 126: 1-16.
- [42] Docker Inc. What is Docker?[EB/OL]. 2025 [2025-04-23]. <https://www.docker.com/resources/what-container>.
- [43] Gannon D, Barga R, Sundaresan N. Cloud-Native Applications[J]. IEEE Cloud Computing, 2017, 4(5): 16-21.
- [44] Li C, Tao J, Chen X, et al. Homcs: A Hierarchical Orchestration Mechanism for Cloud-Native Services in 5G Core Networks[C]. Proceedings of the 2024 12th International Conference on Commu-

- nications and Broadband Networking. 2024: 178-183.
- [45] Poniszewska-Marańda A, Czechowska E. Kubernetes cluster for automating software production environment[J]. *Sensors*, 2021, 21(5): 1910.
- [46] Alibaba Cloud. Container Service for Kubernetes –Alibaba Cloud[Z]. <https://www.alibabacloud.com/en/product/kubernetes>. Accessed: 2025-04-21. 2025.
- [47] 刘景磊, 陈燕军, 牛洁, 等. 中国移动算力网络开源工作实践[J]. *通信世界*, 2022, 1(23): 30-32.
- [48] Kubernetes Authors. Kubernetes Official Documentation[EB/OL]. 2025 [2025-04-23]. <https://kubernetes.io/>.
- [49] Bauman E, Ayoade G, Lin Z. A survey on hypervisor-based monitoring: approaches, applications, and evolutions[J]. *ACM Computing Surveys (CSUR)*, 2015, 48(1): 1-33.
- [50] Luksa M. Kubernetes in Action[M]. 1st. New York: Simon, 2017: 123-124.
- [51] Cloud Native Computing Foundation. Container Network Interface (CNI) Specification[Z]. <https://github.com/containernetworking/cni>. Accessed: 2025-05-10. 2025.
- [52] Jian Z, Xie X, Fang Y, et al. DRS: A deep reinforcement learning enhanced Kubernetes scheduler for microservice-based system[J]. *Software: Practice and Experience*, 2024, 54(10): 2102-2126.
- [53] Senjab K, Abbas S, Ahmed N, et al. A survey of Kubernetes scheduling algorithms[J]. *Journal of Cloud Computing*, 2023, 12(1): 87.
- [54] Barakabitze A A, Ahmad A, Mijumbi R, et al. 5G network slicing using SDN and NFV: A survey of taxonomy, architectures and future challenges[J]. *Computer Networks*, 2020, 167: 106984.
- [55] Yılmaz Ö Z, Alagöz F. Shared network function placement for cloud native 5g core networks[C]. 2023 14th International Conference on Network of the Future (NoF). 2023: 103-107.
- [56] Ravindran S, Chaudhuri S, Bapat J, et al. Novel Adaptive Multi-User Multi-Services Scheduling to Enhance Throughput in 5G-Advanced and Beyond[J]. *IEEE Transactions on Network and Service Management*, 2024, 21(2): 2323-2338.
- [57] Aba M A, Kassis M, Elkael M, et al. Efficient Network Slicing Orchestrator for 5G Networks using a Genetic Algorithm-based Scheduler with Kubernetes: Experimental Insights[C]. 2024 IEEE 10th International Conference on Network Softwarization (NetSoft). 2024: 82-90.
- [58] 李勤, 韩柳燕, 李晗, 等. 切片分组网安全技术研究[J]. *信息通信技术与政策*, 2024, 50(08): 17-23.
- [59] Prasad R, Sunny A. Scheduling Slice Requests in 5G Networks[J]. *IEEE/ACM Transactions on Networking*, 2023, 31(6): 3025-3036.
- [60] Li C, Tao J, Shi G, et al. Migration for Cloud-Native VNF in Open Source MANO[C]. 2023 International Conference on Future Communications and Networks (FCN). 2023: 1-6.
- [61] Piao G, Oh Y, Sung B, et al. Efficient Pre-copy Live Migration with Memory Compaction and Adaptive VM Downtime Control[C]. 2014 IEEE Fourth International Conference on Big Data and Cloud

- Computing: vol. 12: 32. 2014: 85-90.
- [62] Clark C, Fraser K, et al. Live migration of virtual machines[C]. Proceedings of the 2nd USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2005: 273-286.
 - [63] Zhang W, Chen Y, et al. Adaptive snapshot scheduling for consistent cloud VM migration[J]. IEEE Transactions on Cloud Computing, 2017, 5(2): 333-345.
 - [64] Yang L, Xu C. Optimizing live VM migration via adaptive memory compression and dirty page tracking[J]. Future Generation Computer Systems, 2019, 97: 145-155.
 - [65] Chiu Y S, Yen L H, Wang T H, et al. A cloud native management and orchestration framework for 5G end-to-end network slicing[C]. 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE). 2022: 69-76.
 - [66] Osband I, Blundell C, Pritzel A, et al. Deep Exploration via Bootstrapped DQN[C]. Advances in Neural Information Processing Systems (NeurIPS): vol. 29. Curran Associates, Inc., 2016: 4026-4034.
 - [67] Van Hasselt H, Guez A, Silver D. Deep Reinforcement Learning with Double Q-learning[C]. Proceedings of the AAAI Conference on Artificial Intelligence: vol. 30: 1. AAAI Press, 2016: 2094-2100.
 - [68] Pascanu R, Mikolov T, Bengio Y. On the difficulty of training recurrent neural networks[C]. Proceedings of the 30th International Conference on Machine Learning (ICML). 2013: 1310-1318.
 - [69] Kingma D P, Ba J. Adam: A Method for Stochastic Optimization[C]. Proceedings of the 3rd International Conference on Learning Representations (ICLR). 2015.
 - [70] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning [J]. Nature, 2015, 518(7540): 529-533.
 - [71] Lillicrap T P, Hunt J J, Pritzel A, et al. Continuous control with deep reinforcement learning[J]. arXiv preprint arXiv:1509.02971, 2015.
 - [72] Nikaein N, Marina M K, Manickam S, et al. OpenAirInterface: A flexible platform for 5G research [J]. ACM SIGCOMM Computer Communication Review, 2014, 44(5): 33-38.
 - [73] Ramírez S. FastAPI: Fast, modern, web framework for building APIs with Python 3.7+[Z]. <https://fastapi.tiangolo.com/>. Accessed: 2025-05-10. 2025.
 - [74] ONNX Community. Open Neural Network Exchange (ONNX)[Z]. <https://onnx.ai/>. Accessed: 2025-05-10. 2025.

攻读学位期间发表论文与研究成果清单

- [1] Second Author. Migration for Cloud-Native VNF in Open Source MANO[C]. 2023 International Conference on Future Communications and Networks (FCN). 2023: 1-6.
- [2] Second Author. Homcs: A Hierarchical Orchestration Mechanism for Cloud-Native Services in 5G Core Networks[C]. Proceedings of the 2024 12th International Conference on Communications and Broadband Networking. 2024: 178-183.